Orientation

What we have seen:

- RLMPC, why does it work & different flavors

Orientation

What we have seen:

- RLMPC, why does it work & different flavors

What we will do now:

- The theory is not about MPC, it is about MDPs (MPC is a special case)
- It has broader implications for AI and model-based decision making
- Provide (tentative) practical ideas for in Sim2Real

AI for Operational Decisions
Some perspectives from recent research

Prof. Sebastien Gros

Department of Cybernetic
Faculty of Information Technology
NTNU

Freiburg PhD School

Outline

1. Decisions from data
2. AI models for Decision
3. Differentiating Policy Training
4. Examples



Model-Free Pathway - "Pure" Reinforcement Learning



Actions

State

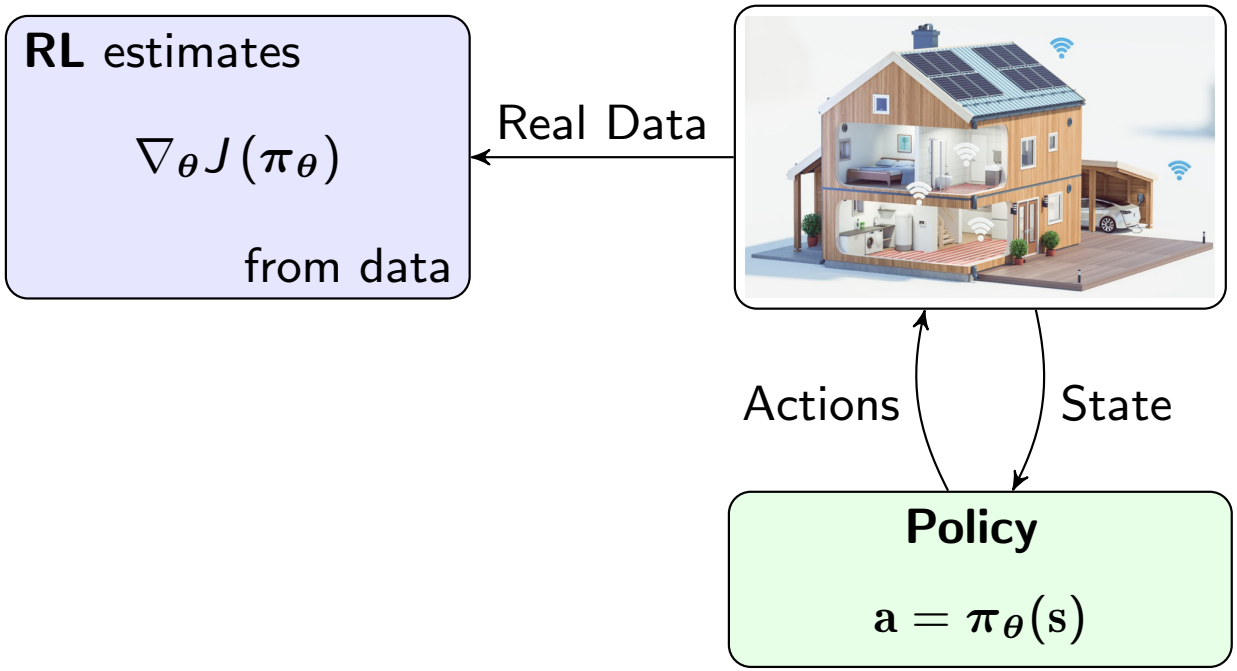
Policy

$$\mathbf{a} = \pi_\theta(\mathbf{s})$$

- Define policy π_θ as a parametrized function
- Optimize policy as:

$$\min_\theta \; J(\pi_\theta)$$

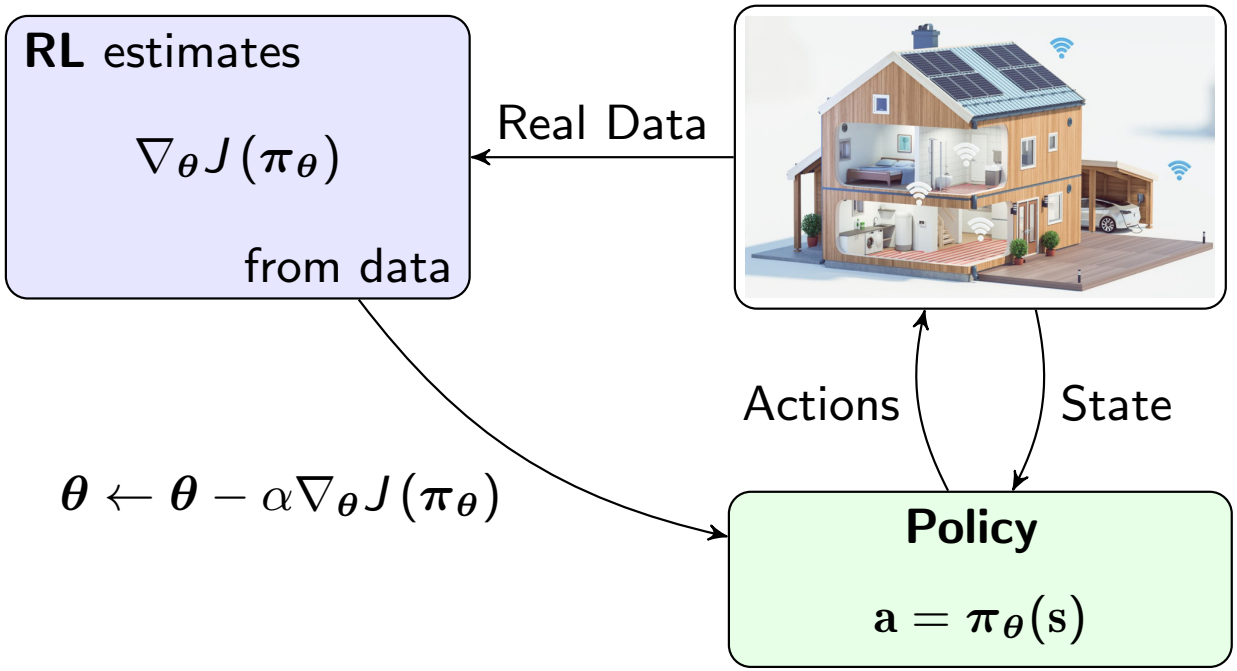
Model-Free Pathway - "Pure" Reinforcement Learning



- Define policy π_θ as a parametrized function
- Optimize policy as:

$$\min_{\theta} \; J(\pi_\theta)$$

Model-Free Pathway - "Pure" Reinforcement Learning

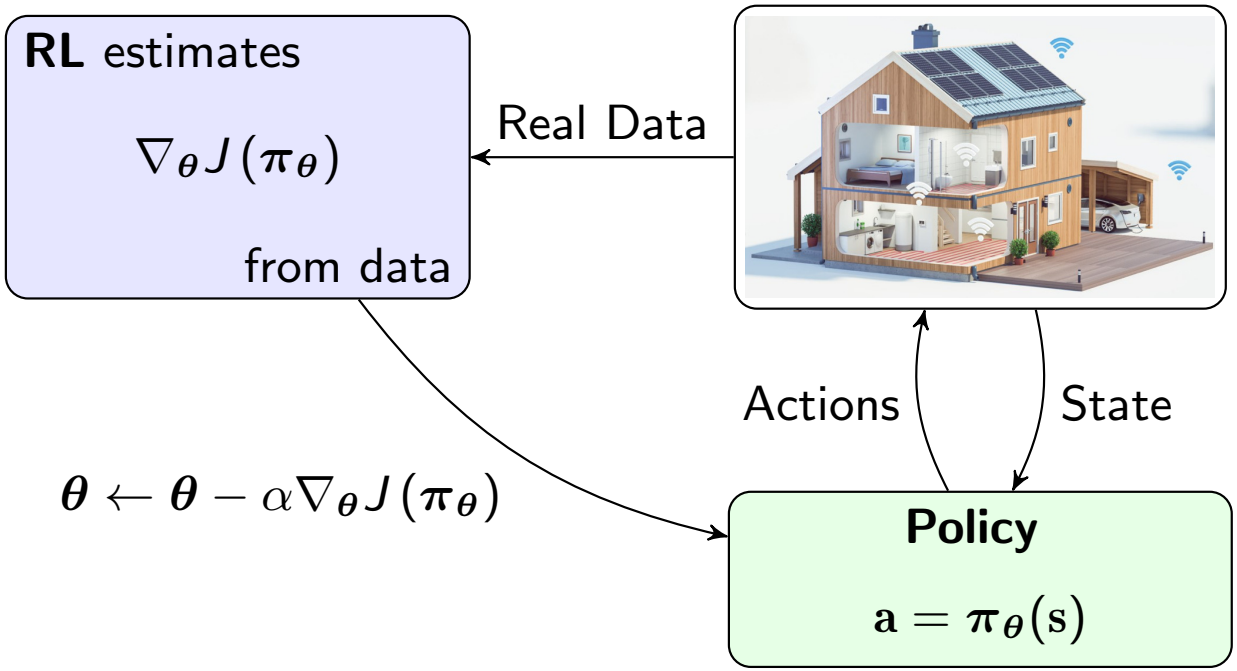


- Define policy π_θ as a parametrized function
- Optimize policy as:

$$\min_\theta \; J(\pi_\theta)$$

- Gradient descent can be used to optimize policy

Model-Free Pathway - "Pure" Reinforcement Learning



- Define policy π_θ as a parametrized function
- Optimize policy as:

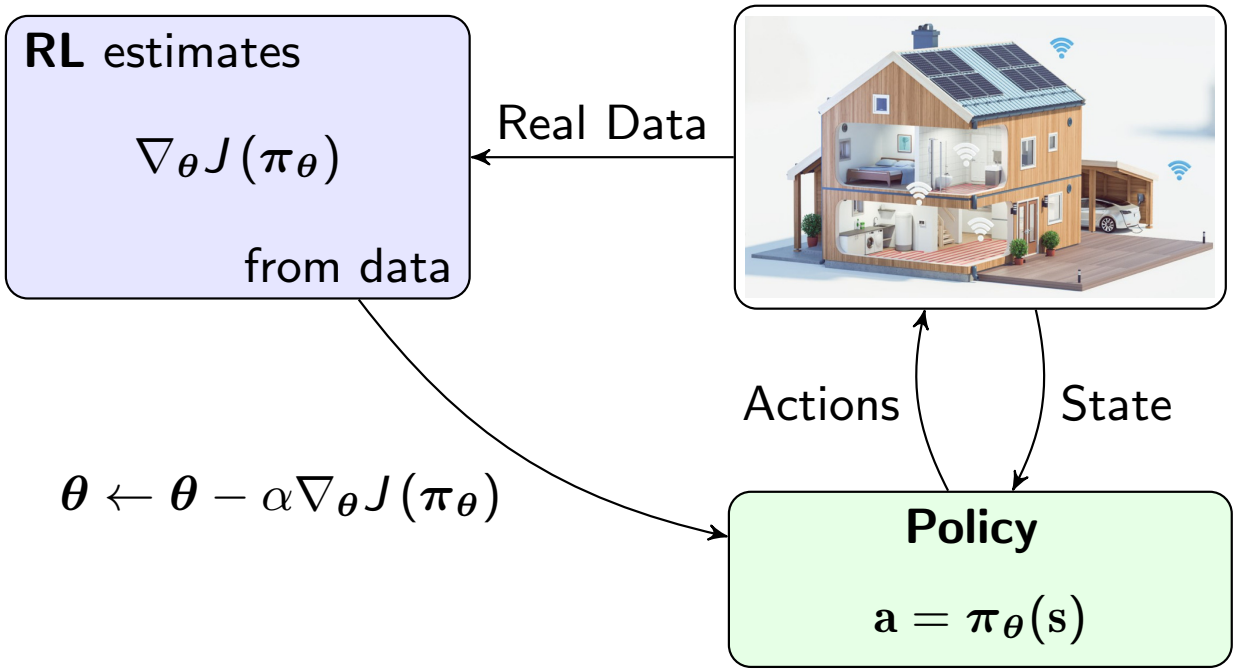
$$\min_\theta \; J(\pi_\theta)$$

- Gradient descent can be used to optimize policy

Remarks

- $\pi_\theta(\mathbf{s})$ often from DNN
- End-to-end training requires a lot of data and effort
- Difficult to provide explainability and guarantees on the policy we obtain
- Poor track record of industrial adoption (exception chatbots)

Model-Free Pathway - "Pure" Reinforcement Learning



- Define policy π_θ as a parametrized function
- Optimize policy as:

$$\min_\theta \; J(\pi_\theta)$$

- Gradient descent can be used to optimize policy

Remarks

- $\pi_\theta(\mathbf{s})$ often from DNN
- End-to-end training requires a lot of data and effort
- Difficult to provide explainability and guarantees on the policy we obtain
- Poor track record of industrial adoption (exception chatbots)

$\rightarrow$policy is often trained on **high-fidelity simulations**: embed knowledge, reduce costs, manage safety, promote explainability, control the training

Model-based Pathway - Predictive AI Models (data-driven, ML, etc)

Real world: $\mathbf{s}_{k+1} \sim \rho(\cdot \,|\, \mathbf{s}_k, \mathbf{a}_k)$

- Data with enough "richness"

Model-based Pathway - Predictive AI Models (data-driven, ML, etc)

One-step model

AI model e.g.

$$\hat{\mathbf{s}}_{k+1} \sim \rho_{\boldsymbol{\theta}}(\cdot \,|\, \mathbf{s}_k, \mathbf{a}_k)$$

fitting data

Real world: $\mathbf{s}_{k+1} \sim \rho(\cdot \,|\, \mathbf{s}_k, \mathbf{a}_k)$

- Data with enough "richness"

Here we no longer need the model $\rho_{\boldsymbol{\theta}}$ to be "optimization-friendly"

Model-based Pathway - Predictive AI Models (data-driven, ML, etc)

One-step model

AI model e.g.

$$\hat{\mathbf{s}}_{k+1} \sim \rho_{\boldsymbol{\theta}}(\cdot \,|\, \mathbf{s}_k, \mathbf{a}_k)$$

fitting data

Real world: $\mathbf{s}_{k+1} \sim \rho(\cdot \,|\, \mathbf{s}_k, \mathbf{a}_k)$

- Data with enough "richness"

Here we no longer need the model $\rho_{\boldsymbol{\theta}}$ to be "optimization-friendly"

Model parameters $\boldsymbol{\theta}$

- Selected such that model $\rho_{\boldsymbol{\theta}}$ "ressembles" reality ρ, using data (+ physics)
- Deterministic models are a special case of $\rho_{\boldsymbol{\theta}}$
- Many methods, from Least Squares and MLE to Bayesian and Adversarial Learning
- Still, in most cases $\rho_{\boldsymbol{\theta}}$ "simplifies" ρ because
  - need for huge amounts of data to decide $\boldsymbol{\theta}$ if model is very rich
  - computationally demanding simulations if $\rho_{\boldsymbol{\theta}}$ is expensive to sample from
- Then
  - biases in case of insufficiently rich model structure
  - validity is limited to some parts of the state-action space
  - only few first moments are correct (typ. mean + variance)

Model-based Pathway - Predictive AI Models (data-driven, ML, etc)

One-step model

AI model e.g.

$$\hat{\mathbf{s}}_{k+1} \sim \rho_{\boldsymbol{\theta}}(\cdot \,|\, \mathbf{s}_k, \mathbf{a}_k)$$

fitting data

Virtual data from "simulating"
$\hat{\mathbf{s}}_{k+1} \sim \rho_{\boldsymbol{\theta}}(\cdot \,|\, \mathbf{s}_k, \mathbf{a}_k)$ forward (Monte Carlo)

Here we no longer need the model $\rho_{\boldsymbol{\theta}}$ to be "optimization-friendly"

Model parameters $\boldsymbol{\theta}$

- Selected such that model $\rho_{\boldsymbol{\theta}}$ "ressembles" reality ρ, using data (+ physics)
- Deterministic models are a special case of $\rho_{\boldsymbol{\theta}}$
- Many methods, from Least Squares and MLE to Bayesian and Adversarial Learning
- Still, in most cases $\rho_{\boldsymbol{\theta}}$ "simplifies" ρ because
  - need for huge amounts of data to decide $\boldsymbol{\theta}$ if model is very rich
  - computationally demanding simulations if $\rho_{\boldsymbol{\theta}}$ is expensive to sample from
- Then
  - biases in case of insufficiently rich model structure
  - validity is limited to some parts of the state-action space
  - only few first moments are correct (typ. mean + variance)

Model-based Pathway - Predictive AI Models (data-driven, ML, etc)

One-step model

AI model e.g.

$$\hat{\mathbf{s}}_{k+1} \sim \rho_{\boldsymbol{\theta}}(\cdot \,|\, \mathbf{s}_k, \mathbf{a}_k)$$

fitting data

Virtual data from "simulating"
$\hat{\mathbf{s}}_{k+1} \sim \rho_{\boldsymbol{\theta}}(\cdot \,|\, \mathbf{s}_k, \mathbf{a}_k)$ forward (Monte Carlo)

In most cases, $\rho_{\boldsymbol{\theta}}$ is a fairly simple representation of reality ρ, easy to sample from

Here we no longer need the model $\rho_{\boldsymbol{\theta}}$ to be "optimization-friendly"

Model parameters $\boldsymbol{\theta}$

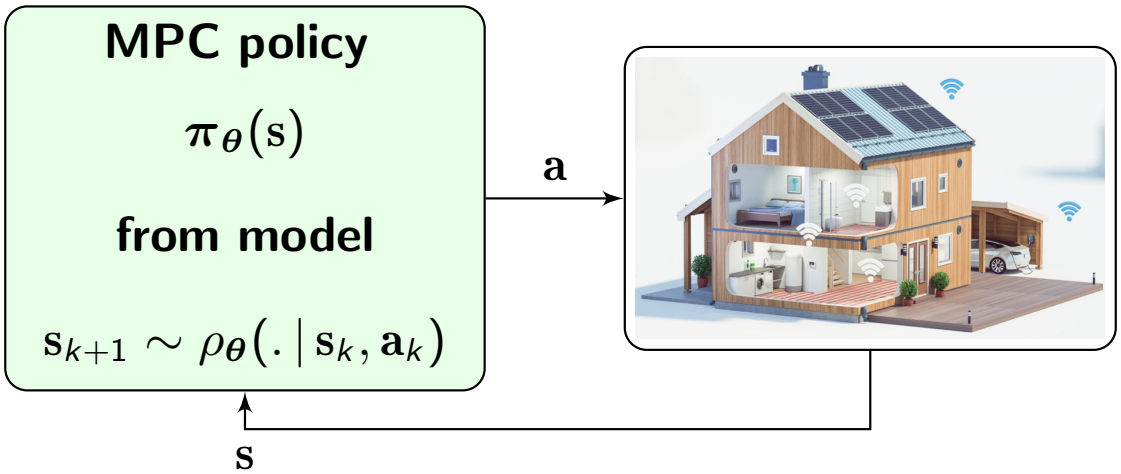
- Selected such that model $\rho_{\boldsymbol{\theta}}$ "ressembles" reality ρ, using data (+ physics)
- Deterministic models are a special case of $\rho_{\boldsymbol{\theta}}$
- Many methods, from Least Squares and MLE to Bayesian and Adversarial Learning
- Still, in most cases $\rho_{\boldsymbol{\theta}}$ "simplifies" ρ because
  - need for huge amounts of data to decide $\boldsymbol{\theta}$ if model is very rich
  - computationally demanding simulations if $\rho_{\boldsymbol{\theta}}$ is expensive to sample from
- Then
  - biases in case of insufficiently rich model structure
  - validity is limited to some parts of the state-action space
  - only few first moments are correct (typ. mean + variance)

Back to MPC - A "Classical" View on Decision Making

Policy $\pi_\theta(\mathrm{s})$ given implicitly by

$$\begin{aligned} \min_{\mathbf{x},\mathbf{u}} \quad & \mathbb{E}\left[T\left(\mathbf{x}_N\right) + \sum_{k=0}^{N-1} L\left(\mathbf{x}_k, \mathbf{u}_k\right) \right] \\ \text{s.t.} \quad & \mathbf{x}_{k+1} \sim \rho_{\boldsymbol{\theta}}\left(\mathbf{x}_k, \mathbf{u}_k\right) \\ & \mathbf{x}_0 = \mathbf{s} \end{aligned}$$

use $\pi_\theta(\mathrm{s}) = \mathbf{u}_0^\star$ on the system

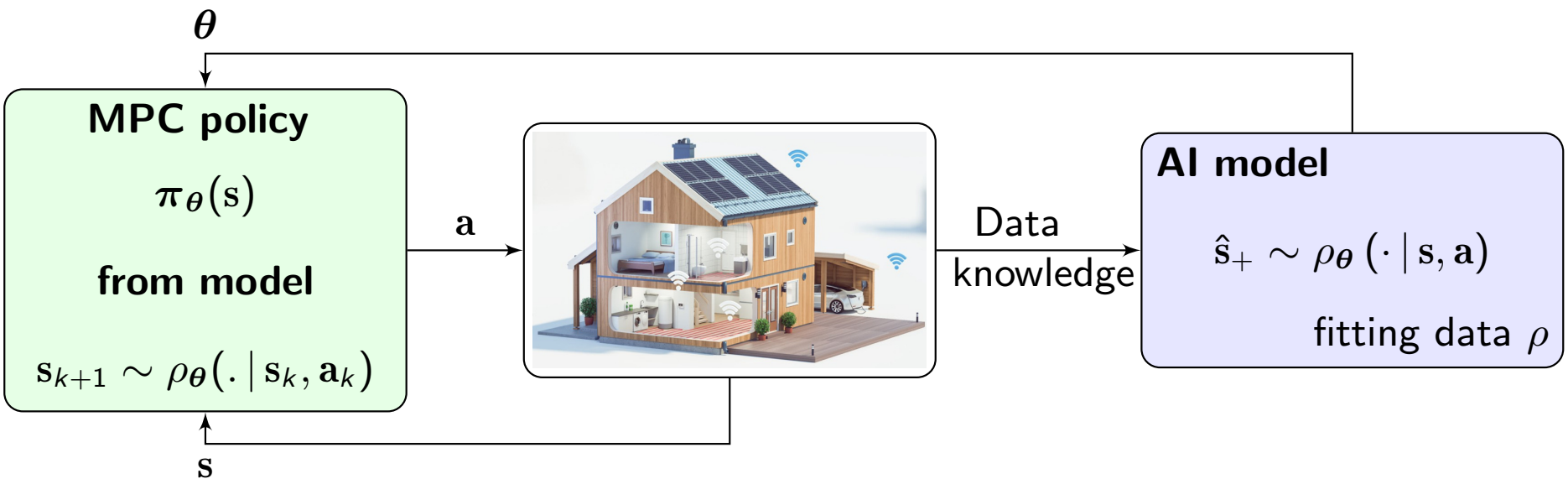


Back to MPC - A "Classical" View on Decision Making

Policy $\pi_\theta(\mathrm{s})$ given implicitly by

$$\begin{aligned} \min_{\mathbf{x},\mathbf{u}} \quad & \mathbb{E}\left[T\left(\mathbf{x}_N\right) + \sum_{k=0}^{N-1} L\left(\mathbf{x}_k, \mathbf{u}_k\right)\right] \\ \text{s.t.} \quad & \mathbf{x}_{k+1} \sim \rho_{\boldsymbol{\theta}}\left(\mathbf{x}_k, \mathbf{u}_k\right) \\ & \mathbf{x}_0 = \mathbf{s} \end{aligned}$$

use $\pi_{\boldsymbol{\theta}}(\mathrm{s}) = \mathbf{u}_0^\star$ on the system



Back to MPC - A "Classical" View on Decision Making

Policy $\pi_\theta(\mathrm{s})$ given implicitly by

$$\begin{aligned} \min_{\mathbf{x},\mathbf{u}} \quad & \mathbb{E}\left[T\left(\mathbf{x}_N\right) + \sum_{k=0}^{N-1} L\left(\mathbf{x}_k, \mathbf{u}_k\right) \right] \\ \text{s.t.} \quad & \mathbf{x}_{k+1} \sim \rho_{\boldsymbol{\theta}}\left(\mathbf{x}_k, \mathbf{u}_k\right) \\ & \mathbf{x}_0 = \mathbf{s} \end{aligned}$$

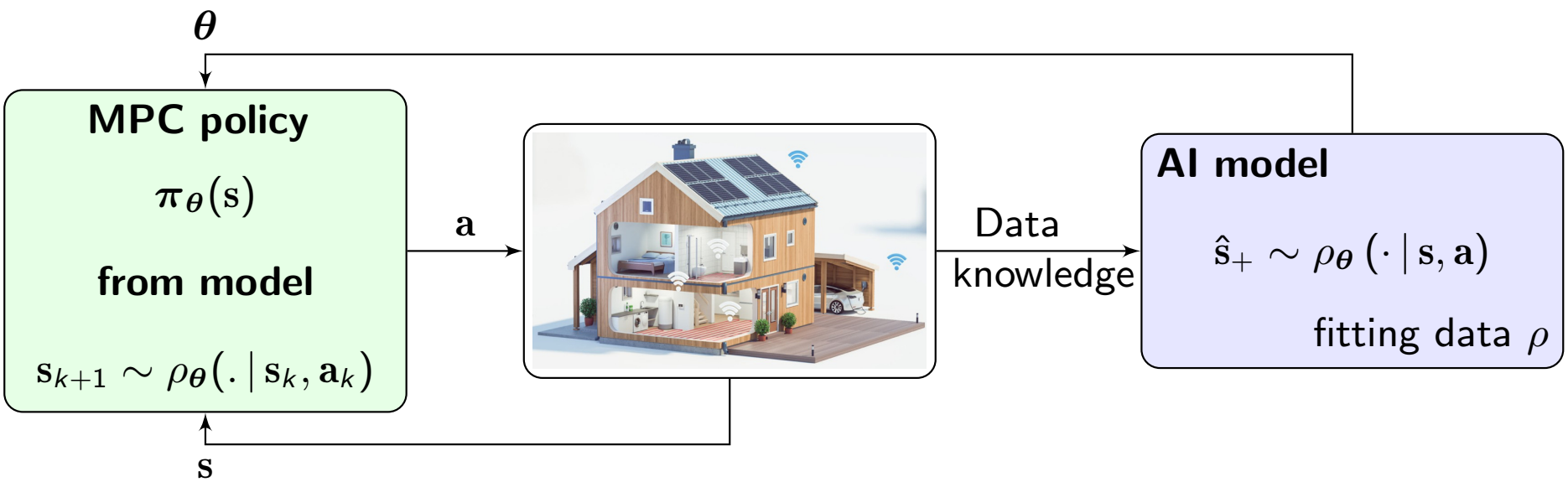
use $\pi_{\boldsymbol{\theta}}(\mathbf{s}) = \mathbf{u}_0^\star$ on the system

Defines a paradigm...

- Performance from model accuracy

  i.e. $\rho_{\boldsymbol{\theta}}$ "close enough" to ρ

- "Ignore" that we replan all the time



Back to MPC - A "Classical" View on Decision Making

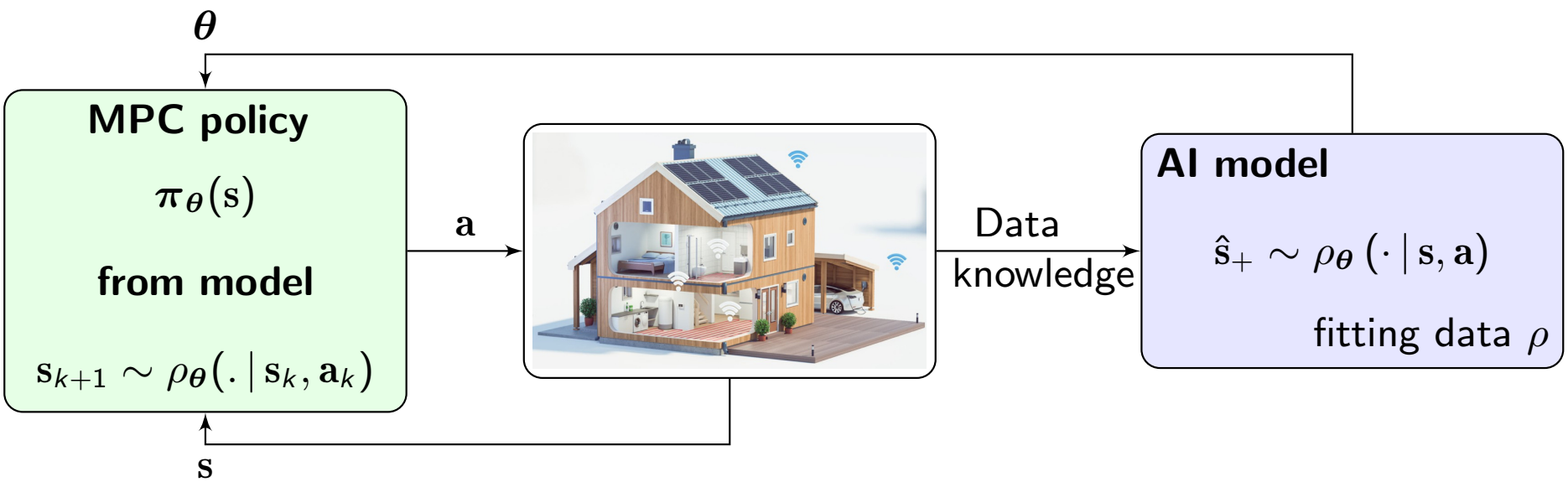
Policy $\pi_\theta(\mathrm{s})$ given implicitly by

$$\begin{aligned} \min_{\mathbf{x},\mathbf{u}} \quad & \mathbb{E}\left[T\left(\mathbf{x}_N\right)+\sum_{k=0}^{N-1} L\left(\mathbf{x}_k,\mathbf{u}_k\right)\right] \\ \text{s.t.} \quad & \mathbf{x}_{k+1} \sim \rho_{\boldsymbol{\theta}}\left(\mathbf{x}_k,\mathbf{u}_k\right) \\ & \mathbf{x}_0 = \mathbf{s} \end{aligned}$$

use $\pi_{\boldsymbol{\theta}}(\mathrm{s}) = \mathbf{u}_0^\star$ on the system

Relationship π_θ to $\pi^\star$??

- They match if $\rho_{\boldsymbol{\theta}}$ is exact and deterministic
- But it usually cannot be...



Back to MPC - A "Classical" View on Decision Making

Policy $\pi_\theta(\mathrm{s})$ given implicitly by

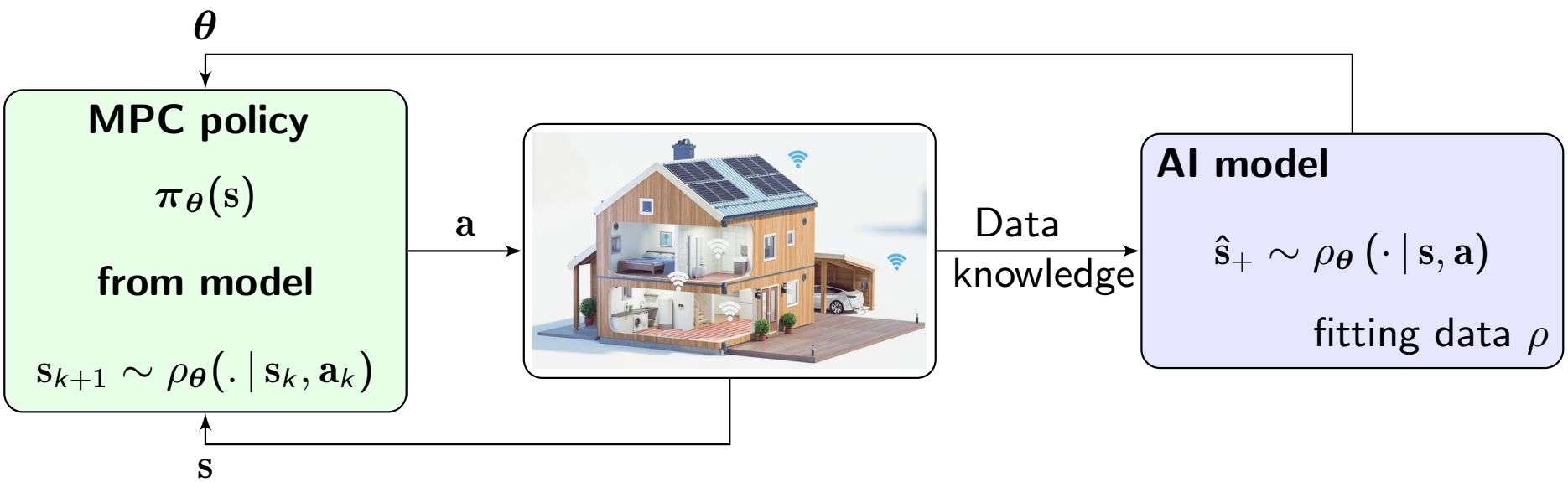
$$\begin{aligned} \min_{\mathbf{x},\mathbf{u}} \quad & \mathbb{E}\left[T\left(\mathbf{x}_N\right) + \sum_{k=0}^{N-1} L\left(\mathbf{x}_k, \mathbf{u}_k\right)\right] \\ \text{s.t.} \quad & \mathbf{x}_{k+1} \sim \rho_\theta\left(\mathbf{x}_k, \mathbf{u}_k\right) \\ & \mathbf{x}_0 = \mathbf{s} \end{aligned}$$

use $\pi_\theta(\mathrm{s}) = \mathbf{u}_0^\star$ on the system

Relationship π_θ to $\pi^\star$??

- They match if ρ_θ is exact and deterministic
- But it usually cannot be...

"Standard methods" for choosing θ in general do not yield the best π_θ.
RL over MPC can fix that, L becomes part of the model



Back to MPC - A "Classical" View on Decision Making

Policy $\pi_\theta(\mathrm{s})$ given implicitly by

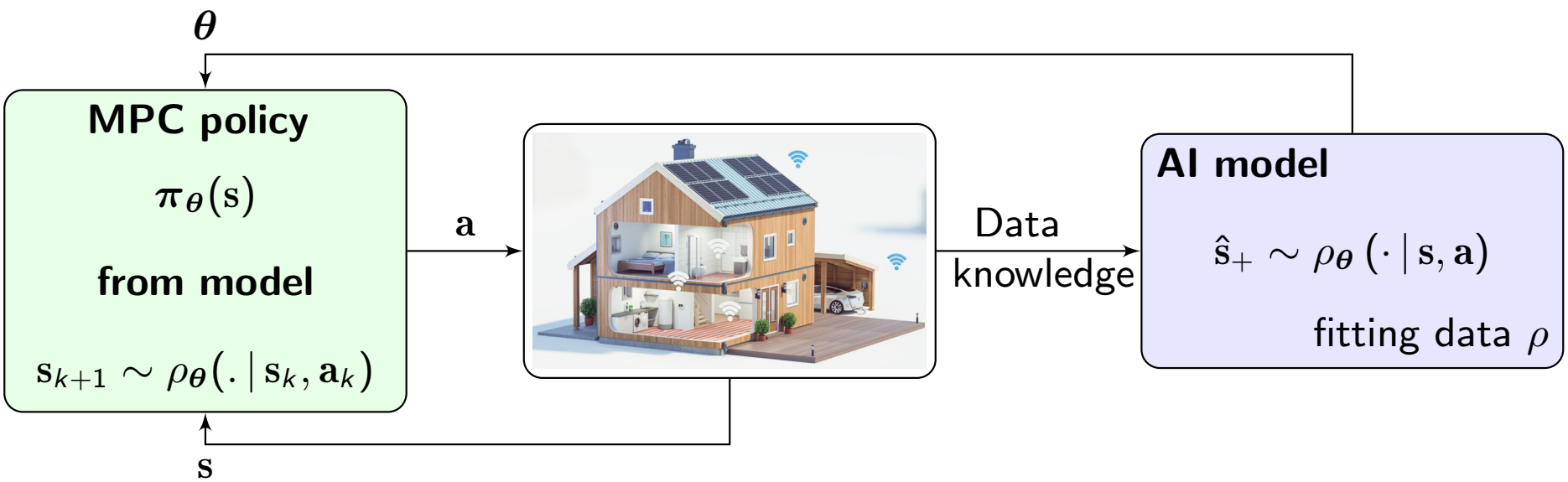
$$\begin{aligned} \min_{\mathbf{x},\mathbf{u}} \quad & \mathbb{E}\left[T\left(\mathbf{x}_N\right) + \sum_{k=0}^{N-1} L\left(\mathbf{x}_k, \mathbf{u}_k\right)\right] \\ \text{s.t.} \quad & \mathbf{x}_{k+1} \sim \rho_\theta\left(\mathbf{x}_k, \mathbf{u}_k\right) \\ & \mathbf{x}_0 = \mathbf{s} \end{aligned}$$

use $\pi_\theta(\mathrm{s}) = \mathbf{u}_0^\star$ on the system

Relationship π_θ to $\pi^\star$??

- They match if ρ_θ is exact and deterministic
- But it usually cannot be...

"Standard methods" for choosing θ in general do not yield the best π_θ. RL over MPC can fix that, L becomes part of the model



This is not the only way of taking decisions from models

Decision policy from AI models (Sim2Real)

AI model

$$\hat{\mathbf{s}}_+ \sim \rho_{\boldsymbol{\theta}}\left(\cdot \,|\, \mathbf{s}, \mathbf{a}\right)$$

fitting data ρ

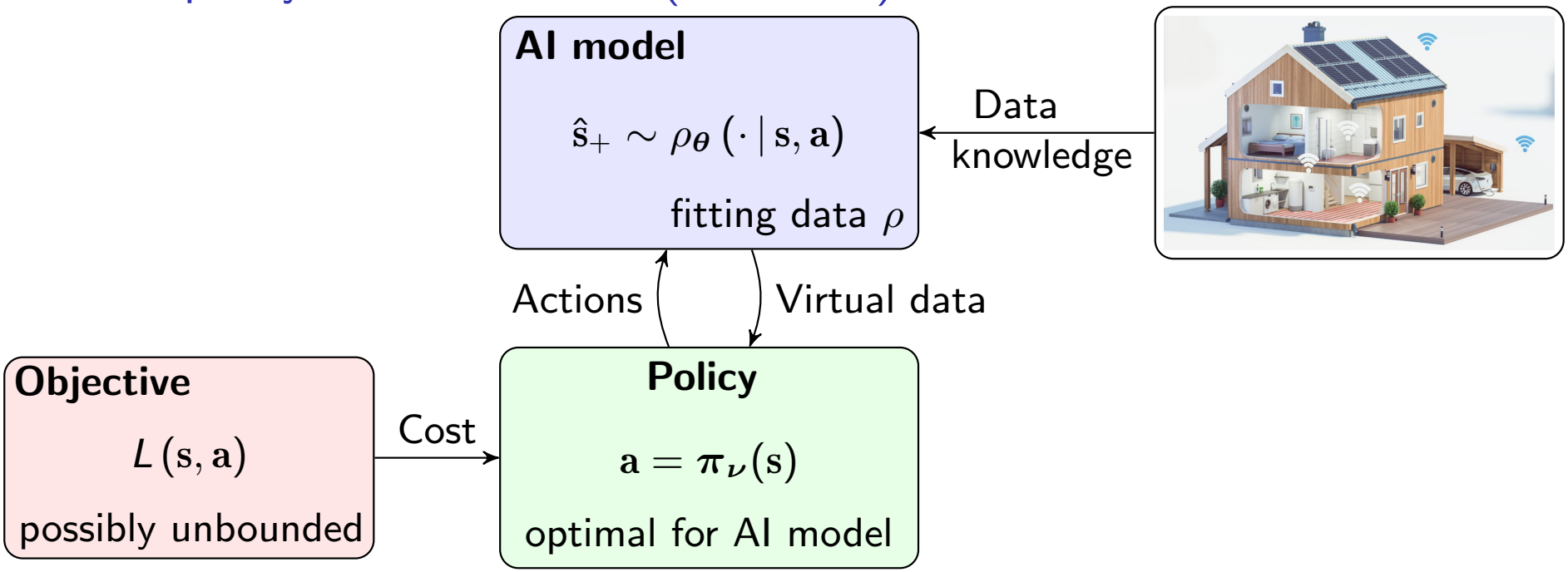
Data knowledge



Classical Process:

- Fit AI model $\rho_{\boldsymbol{\theta}}$ to real data

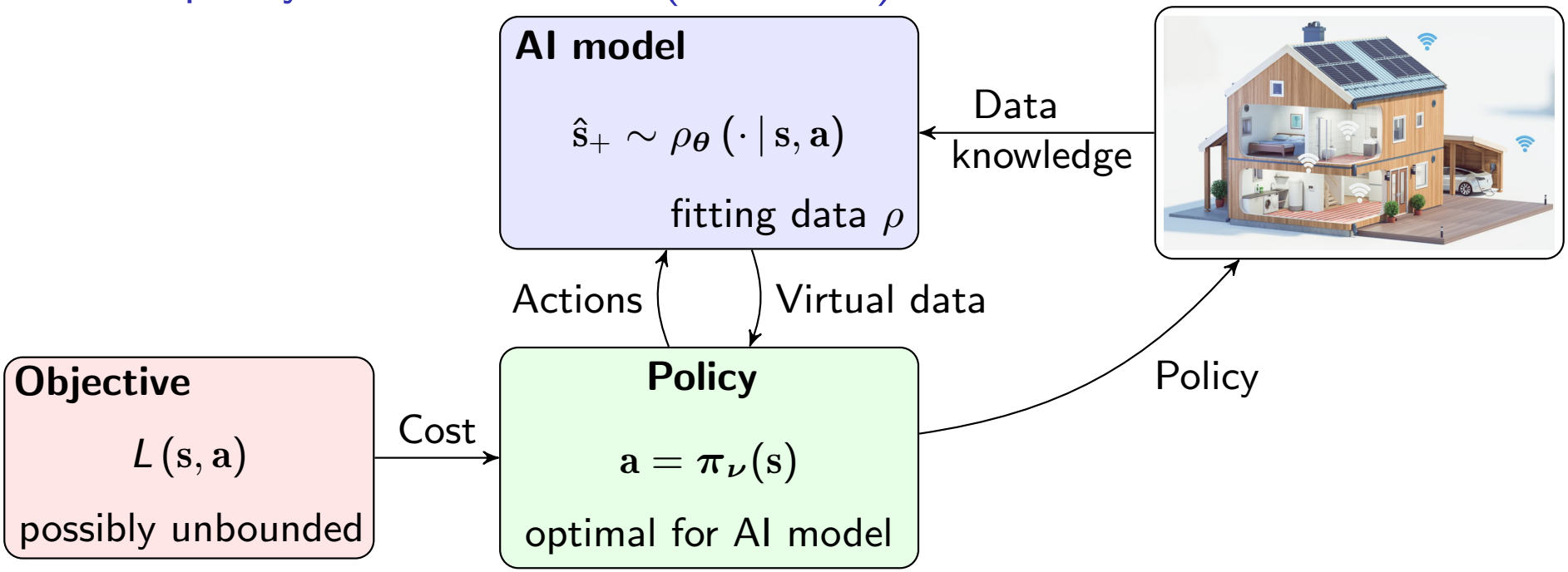
Decision policy from AI models (Sim2Real)



Classical Process:

- Fit AI model $\rho_{\boldsymbol{\theta}}$ to real data
- Develop optimal policy for L and $\rho_{\boldsymbol{\theta}}$ from AI model, e.g. using In-Sim RL
  - Define parametrized policy $\boldsymbol{\pi}_{\boldsymbol{\nu}}$
  - Optimize policy parameters $\boldsymbol{\nu}$ for performance w.r.t. AI model: $\rho_{\boldsymbol{\theta}}, L \rightarrow \boldsymbol{\nu}^\star$

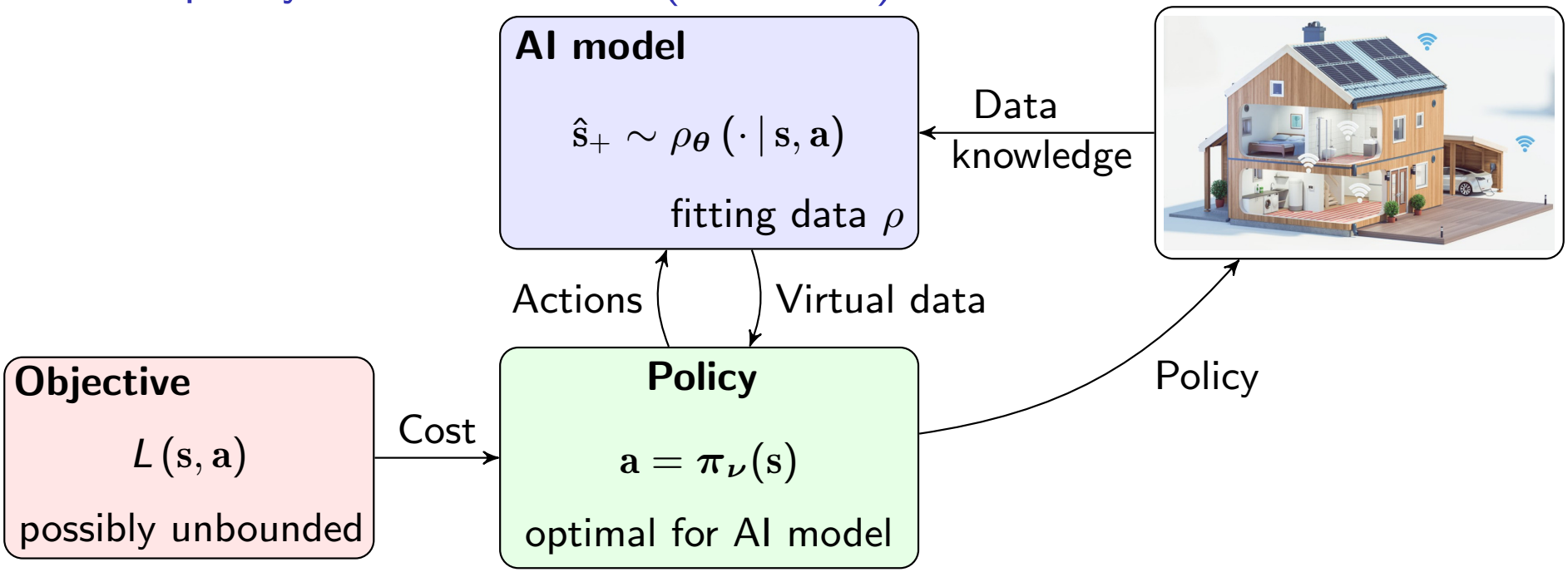
Decision policy from AI models (Sim2Real)



Classical Process:

- Fit AI model $\rho_{\boldsymbol{\theta}}$ to real data
- Develop optimal policy for L and $\rho_{\boldsymbol{\theta}}$ from AI model, e.g. using In-Sim RL
  - Define parametrized policy $\boldsymbol{\pi}_{\boldsymbol{\nu}}$
  - Optimize policy parameters $\boldsymbol{\nu}$ for performance w.r.t. AI model: $\rho_{\boldsymbol{\theta}}, L \rightarrow \boldsymbol{\nu}^\star$
- Transfer policy $\boldsymbol{\pi}_{\boldsymbol{\nu}^\star}$ into the real world

Decision policy from AI models (Sim2Real)

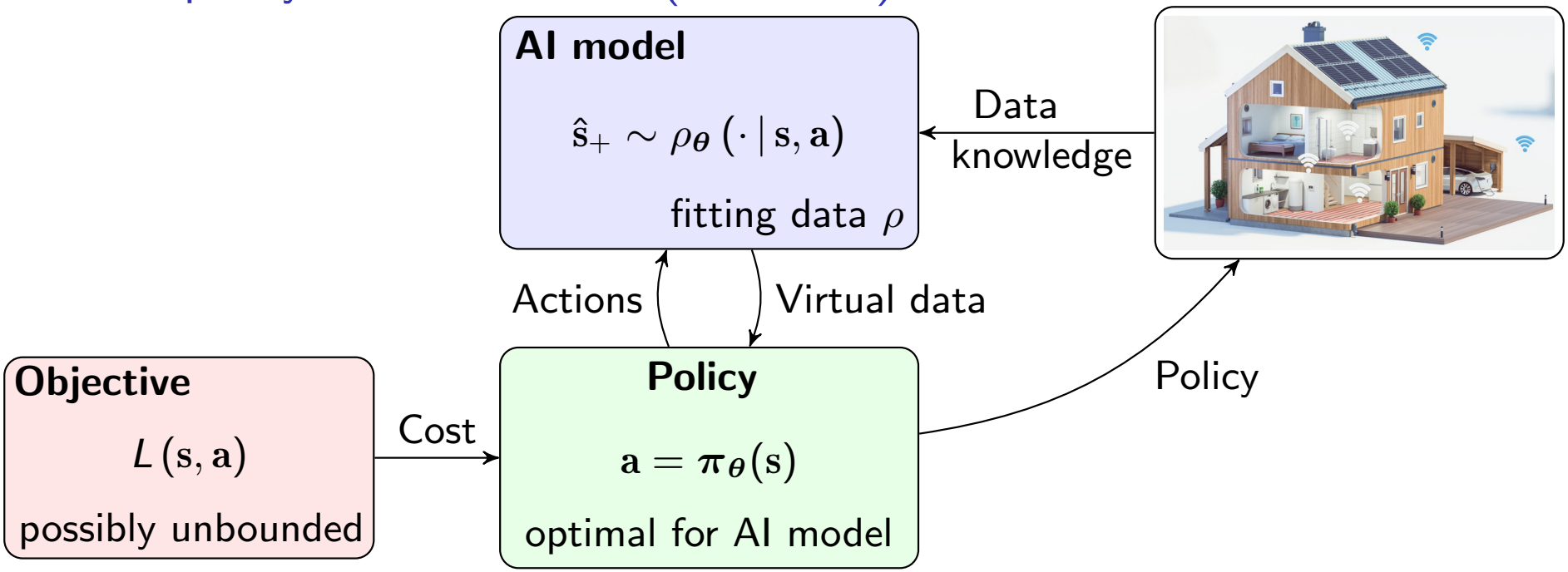


Classical Process:

- Fit AI model $\rho_{\boldsymbol{\theta}}$ to real data
- Develop optimal policy for L and $\rho_{\boldsymbol{\theta}}$ from AI model, e.g. using In-Sim RL
  - Define parametrized policy $\boldsymbol{\pi}_{\boldsymbol{\nu}}$
  - Optimize policy parameters $\boldsymbol{\nu}$ for performance w.r.t. AI model: $\rho_{\boldsymbol{\theta}}, L \rightarrow \boldsymbol{\nu}^\star$
- Transfer policy $\boldsymbol{\pi}_{\boldsymbol{\nu}^\star}$ into the real world

Note: policy parameters $\nu^\star$ optimal for AI model become function of θ
Let's then label $\pi_\theta = \pi_{\nu^\star}$ for simplicity

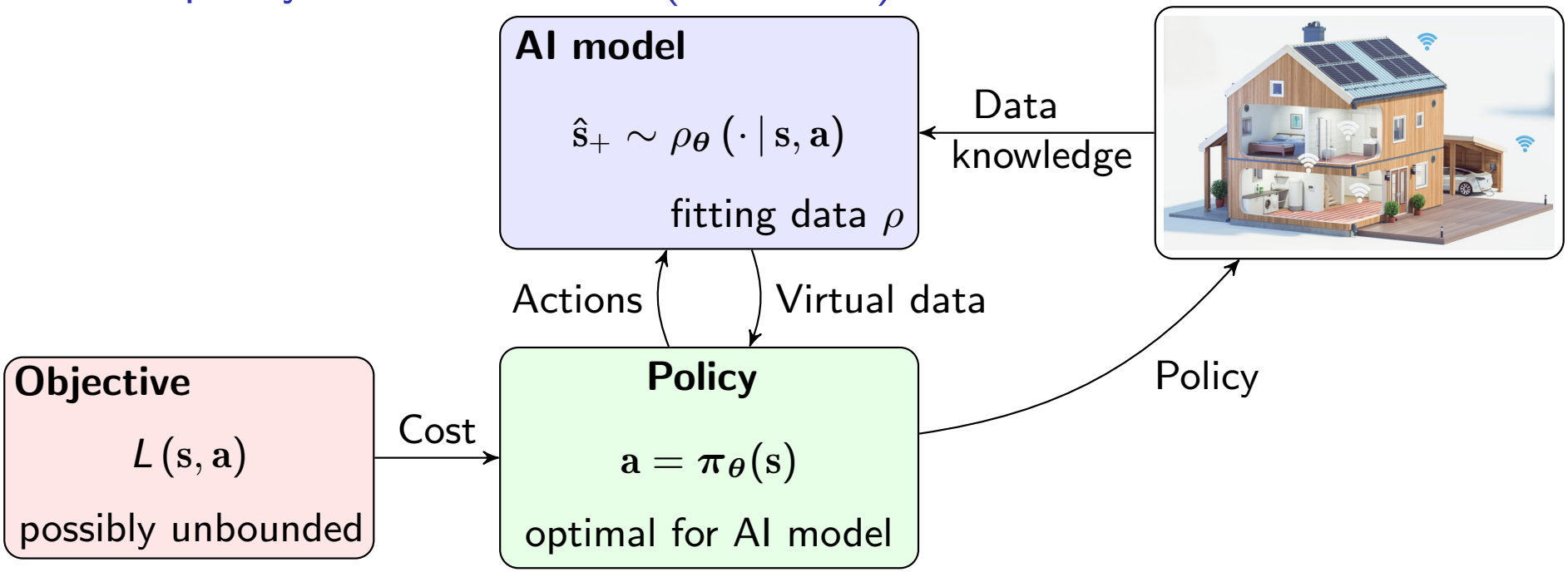
Decision policy from AI models (Sim2Real)



Remarks

- Relationship $\boldsymbol{\pi}_{\boldsymbol{\theta}}$ to $\boldsymbol{\pi}^\star$??
  - They match if $\rho_{\boldsymbol{\theta}} = \rho$, i.e. if model is exact
  - But $\rho_{\boldsymbol{\theta}}$ is (almost always) an approximation

Decision policy from AI models (Sim2Real)



Remarks

- Relationship $\boldsymbol{\pi}_{\boldsymbol{\theta}}$ to $\boldsymbol{\pi}^\star$??
  - They match if $\rho_{\boldsymbol{\theta}} = \rho$, i.e. if model is exact
  - But $\rho_{\boldsymbol{\theta}}$ is (almost always) an approximation

Are "standard methods" for choosing θ resulting in a good policy π_θ??
Empirically the answer is "no". Then how to choose θ to get a good policy?

MPC or Sim2Real - It's the same question...

MPC or Sim2Real - It's the same question...

The theory is about model-based decisions, and equivalences between MDPs

MPC or Sim2Real - It's the same question...

The theory is about model-based decisions, and equivalences between MDPs

World MDP

World MDP definition

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

MPC or Sim2Real - It's the same question...

The theory is about model-based decisions, and equivalences between MDPs

World MDP

World MDP definition

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Optimal value functions

$$V^\star(\mathbf{s}) = \min_{\mathbf{a}} Q^\star(\mathbf{s}, \mathbf{a})$$

$$Q^\star(\mathbf{s}, \mathbf{a}) = L(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}_+ \sim \rho}[V^\star(\mathbf{s}_+)|\mathbf{s}, \mathbf{a}]$$

MPC or Sim2Real - It's the same question...

The theory is about model-based decisions, and equivalences between MDPs

World MDP

World MDP definition

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Optimal value functions

$$V^\star(\mathbf{s}) = \min_{\mathbf{a}} Q^\star(\mathbf{s}, \mathbf{a})$$

$$Q^\star(\mathbf{s}, \mathbf{a}) = L(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}_+ \sim \rho}[V^\star(\mathbf{s}_+)|\mathbf{s}, \mathbf{a}]$$

Optimal policy

$$\boldsymbol{\pi}^\star(\mathbf{s}) = \arg\min_{\mathbf{a}} Q^\star(\mathbf{s}, \mathbf{a})$$

MPC or Sim2Real - It's the same question...

The theory is about model-based decisions, and equivalences between MDPs

World MDP

World MDP definition

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Optimal value functions

$$V^\star(\mathbf{s}) = \min_{\mathbf{a}} Q^\star(\mathbf{s}, \mathbf{a})$$

$$Q^\star(\mathbf{s}, \mathbf{a}) = L(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}_+ \sim \rho}[V^\star(\mathbf{s}_+)|\mathbf{s}, \mathbf{a}]$$

Optimal policy

$$\pi^\star(\mathbf{s}) = \arg\min_{\mathbf{a}} Q^\star(\mathbf{s}, \mathbf{a})$$

Model MDP (simulation environment)

Model MDP definition

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Optimal value functions

$$\hat{V}^\star(\mathbf{s}) = \min_{\mathbf{a}} \hat{Q}^\star(\mathbf{s}, \mathbf{a})$$

$$\hat{Q}^\star(\mathbf{s}, \mathbf{a}) = \hat{L}(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}_+ \sim \hat{\rho}}[V^\star(\hat{\mathbf{s}}_+)|\mathbf{s}, \mathbf{a}]$$

Optimal policy

$$\hat{\pi}^\star(\mathbf{s}) = \arg\min_{\mathbf{a}} \hat{Q}^\star(\mathbf{s}, \mathbf{a})$$

MPC or Sim2Real - It's the same question...

The theory is about model-based decisions, and equivalences between MDPs

World MDP

World MDP definition

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Optimal value functions

$$V^\star(\mathbf{s}) = \min_{\mathbf{a}} Q^\star(\mathbf{s}, \mathbf{a})$$

$$Q^\star(\mathbf{s}, \mathbf{a}) = L(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}_+ \sim \rho}[V^\star(\mathbf{s}_+)|\mathbf{s}, \mathbf{a}]$$

Optimal policy

$$\pi^\star(\mathbf{s}) = \arg\min_{\mathbf{a}} Q^\star(\mathbf{s}, \mathbf{a})$$

Model MDP (simulation environment)

Model MDP definition

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Optimal value functions

$$\hat{V}^\star(\mathbf{s}) = \min_{\mathbf{a}} \hat{Q}^\star(\mathbf{s}, \mathbf{a})$$

$$\hat{Q}^\star(\mathbf{s}, \mathbf{a}) = \hat{L}(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}_+ \sim \hat{\rho}}[V^\star(\hat{\mathbf{s}}_+)|\mathbf{s}, \mathbf{a}]$$

Optimal policy

$$\hat{\pi}^\star(\mathbf{s}) = \arg\min_{\mathbf{a}} \hat{Q}^\star(\mathbf{s}, \mathbf{a})$$

Theory says that - under some technical conditions - there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$

MPC or Sim2Real - It's the same question...

The theory is about model-based decisions, and equivalences between MDPs

World MDP

World MDP definition

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Optimal value functions

$$V^\star(\mathbf{s}) = \min_{\mathbf{a}} Q^\star(\mathbf{s}, \mathbf{a})$$

$$Q^\star(\mathbf{s}, \mathbf{a}) = L(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}_+ \sim \rho}[V^\star(\mathbf{s}_+)|\mathbf{s}, \mathbf{a}]$$

Optimal policy

$$\boldsymbol{\pi}^\star(\mathbf{s}) = \arg\min_{\mathbf{a}} Q^\star(\mathbf{s}, \mathbf{a})$$

Model MDP (simulation environment)

Model MDP definition

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Optimal value functions

$$\hat{V}^\star(\mathbf{s}) = \min_{\mathbf{a}} \hat{Q}^\star(\mathbf{s}, \mathbf{a})$$

$$\hat{Q}^\star(\mathbf{s}, \mathbf{a}) = \hat{L}(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}_+ \sim \hat{\rho}}[V^\star(\hat{\mathbf{s}}_+)|\mathbf{s}, \mathbf{a}]$$

Optimal policy

$$\hat{\boldsymbol{\pi}}^\star(\mathbf{s}) = \arg\min_{\mathbf{a}} \hat{Q}^\star(\mathbf{s}, \mathbf{a})$$

Theory says that - under some technical conditions - there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$

Proof: telescopic sum, some non-trivial assumptions to prevent $\infty - \infty$ cancellations

More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Theory says that

- Under some technical conditions there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$

More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Theory says that

- Under some technical conditions there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$
- For $\hat{L} = L$ there is a (non-unique) "optimal" model $\hat{\rho}$ such that $\hat{Q}^\star = Q^\star$

More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Theory says that

- Under some technical conditions there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$
- For $\hat{L} = L$ there is a (non-unique) "optimal" model $\hat{\rho}$ such that $\hat{Q}^\star = Q^\star$
- Conditions for model $\hat{\rho}$ "optimality" $\neq$ min of classical loss functions (except. LQR)

More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Theory says that

- Under some technical conditions there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$
- For $\hat{L} = L$ there is a (non-unique) "optimal" model $\hat{\rho}$ such that $\hat{Q}^\star = Q^\star$
- Conditions for model $\hat{\rho}$ "optimality" $\neq$ min of classical loss functions (except. LQR)
- World MDP and Model MDP do not need to use the same discount γ

More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Theory says that

- Under some technical conditions there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$
- For $\hat{L} = L$ there is a (non-unique) "optimal" model $\hat{\rho}$ such that $\hat{Q}^\star = Q^\star$
- Conditions for model $\hat{\rho}$ "optimality" $\neq$ min of classical loss functions (except. LQR)
- World MDP and Model MDP do not need to use the same discount γ

In-Sim policy training uses a Model MDP

- Practitioners mostly work on model $\hat{\rho}$, but "unsure" on how to tune it
- In the ML communities, people talk about "value alignment", we are trying to make this "MDP equivalence" understood

More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Theory says that

- Under some technical conditions there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$
- For $\hat{L} = L$ there is a (non-unique) "optimal" model $\hat{\rho}$ such that $\hat{Q}^\star = Q^\star$
- Conditions for model $\hat{\rho}$ "optimality" $\neq$ min of classical loss functions (except. LQR)
- World MDP and Model MDP do not need to use the same discount γ

Remarks

- Non-unique optimal model $\hat{\rho}$ leaves room for aligning it to the real world (classical fitting)

More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Theory says that

- Under some technical conditions there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$
- For $\hat{L} = L$ there is a (non-unique) "optimal" model $\hat{\rho}$ such that $\hat{Q}^\star = Q^\star$
- Conditions for model $\hat{\rho}$ "optimality" $\neq$ min of classical loss functions (except. LQR)
- World MDP and Model MDP do not need to use the same discount γ

Remarks

- Non-unique optimal model $\hat{\rho}$ leaves room for aligning it to the real world (classical fitting)
- If $V^\star$ is continuous and support of ρ is bounded(?) and path-connected, then we can have support $\hat{\rho} \subset$ support of ρ

More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

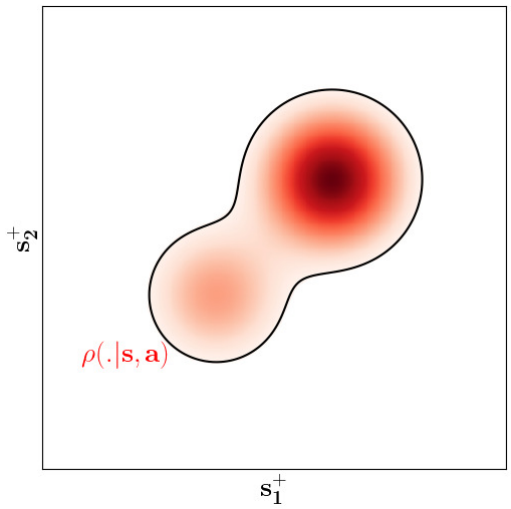
- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Theory says that

- Under some technical conditions there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$
- For $\hat{L} = L$ there is a (non-unique) "optimal" model $\hat{\rho}$ such that $\hat{Q}^\star = Q^\star$
- Conditions for model $\hat{\rho}$ "optimality" $\neq$ min of classical loss functions (except. LQR)
- World MDP and Model MDP do not need to use the same discount γ

Remarks

- Non-unique optimal model $\hat{\rho}$ leaves room for aligning it to the real world (classical fitting)
- If $V^\star$ is continuous and support of ρ is bounded(?) and path-connected, then we can have support $\hat{\rho} \subset$ support of ρ



More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

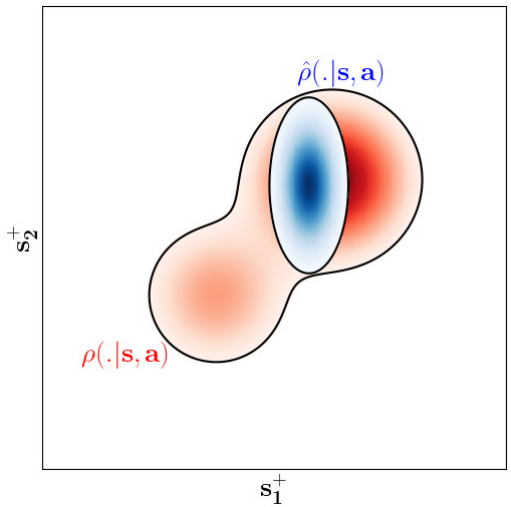
- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Theory says that

- Under some technical conditions there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$
- For $\hat{L} = L$ there is a (non-unique) "optimal" model $\hat{\rho}$ such that $\hat{Q}^\star = Q^\star$
- Conditions for model $\hat{\rho}$ "optimality" $\neq$ min of classical loss functions (except. LQR)
- World MDP and Model MDP do not need to use the same discount γ

Remarks

- Non-unique optimal model $\hat{\rho}$ leaves room for aligning it to the real world (classical fitting)
- If $V^\star$ is continuous and support of ρ is bounded(?) and path-connected, then we can have support $\hat{\rho} \subset$ support of ρ



More on MDP Equivalence

World MDP

- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $L(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \rho[\cdot|\mathbf{s}, \mathbf{a}]$

Model MDP

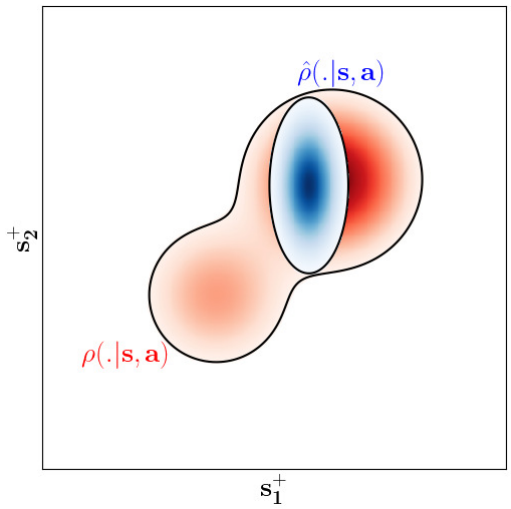
- States $\mathbf{s}$ and actions $\mathbf{a}$
- Cost $\hat{L}(\mathbf{s}, \mathbf{a})$
- Transition $\mathbf{s}_+ \sim \hat{\rho}[\cdot|\mathbf{s}, \mathbf{a}]$

Theory says that

- Under some technical conditions there is a $\hat{L}$ such that $\hat{Q}^\star = Q^\star$
- For $\hat{L} = L$ there is a (non-unique) "optimal" model $\hat{\rho}$ such that $\hat{Q}^\star = Q^\star$
- Conditions for model $\hat{\rho}$ "optimality" $\neq$ min of classical loss functions (except. LQR)
- World MDP and Model MDP do not need to use the same discount γ

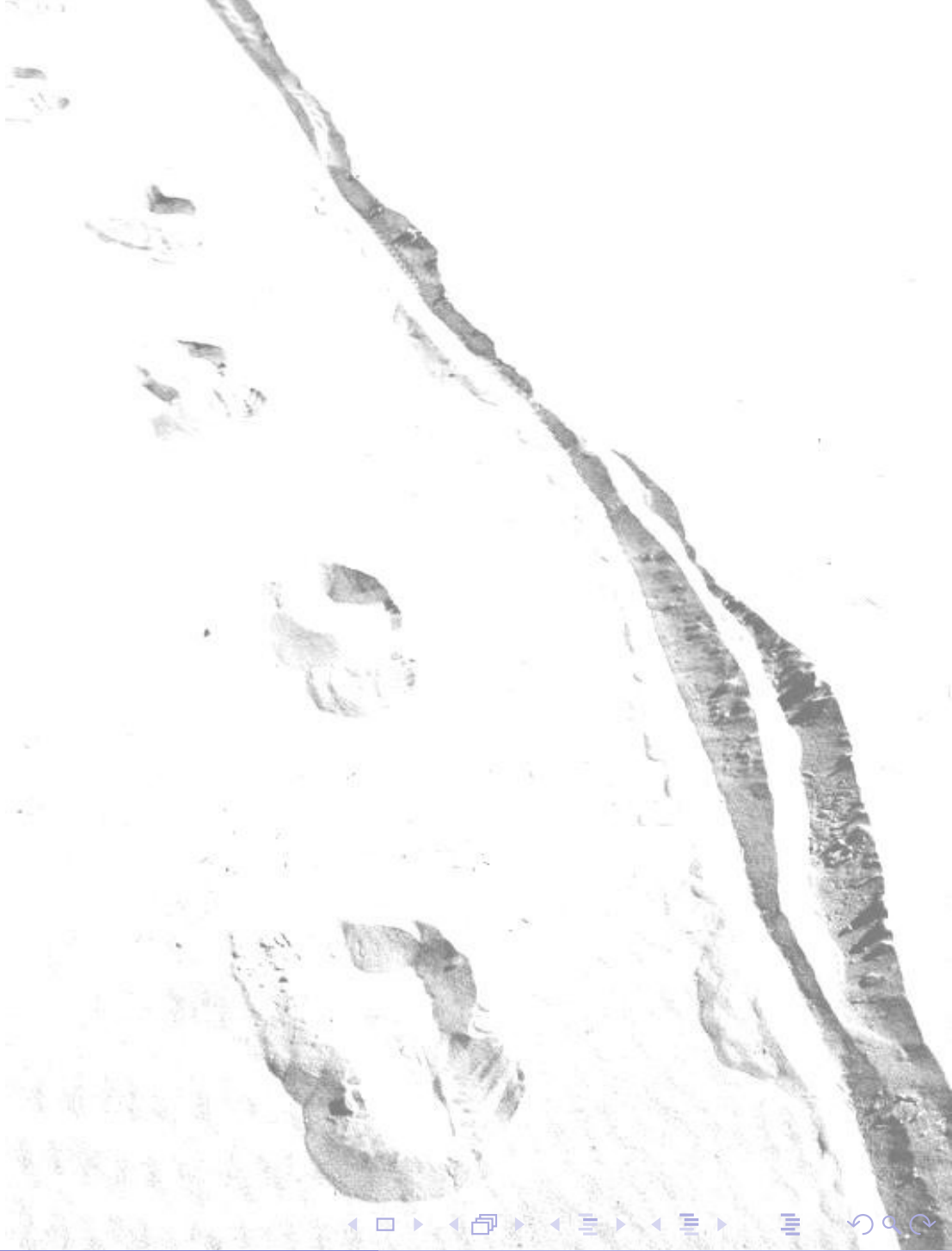
Remarks

- Non-unique optimal model $\hat{\rho}$ leaves room for aligning it to the real world (classical fitting)
- If $V^\star$ is continuous and support of ρ is bounded(?) and path-connected, then we can have support $\hat{\rho} \subset$ support of ρ
- *More simply said*: we can build optimal models $\hat{\rho}$ that make "plausible" predictions about the real world.

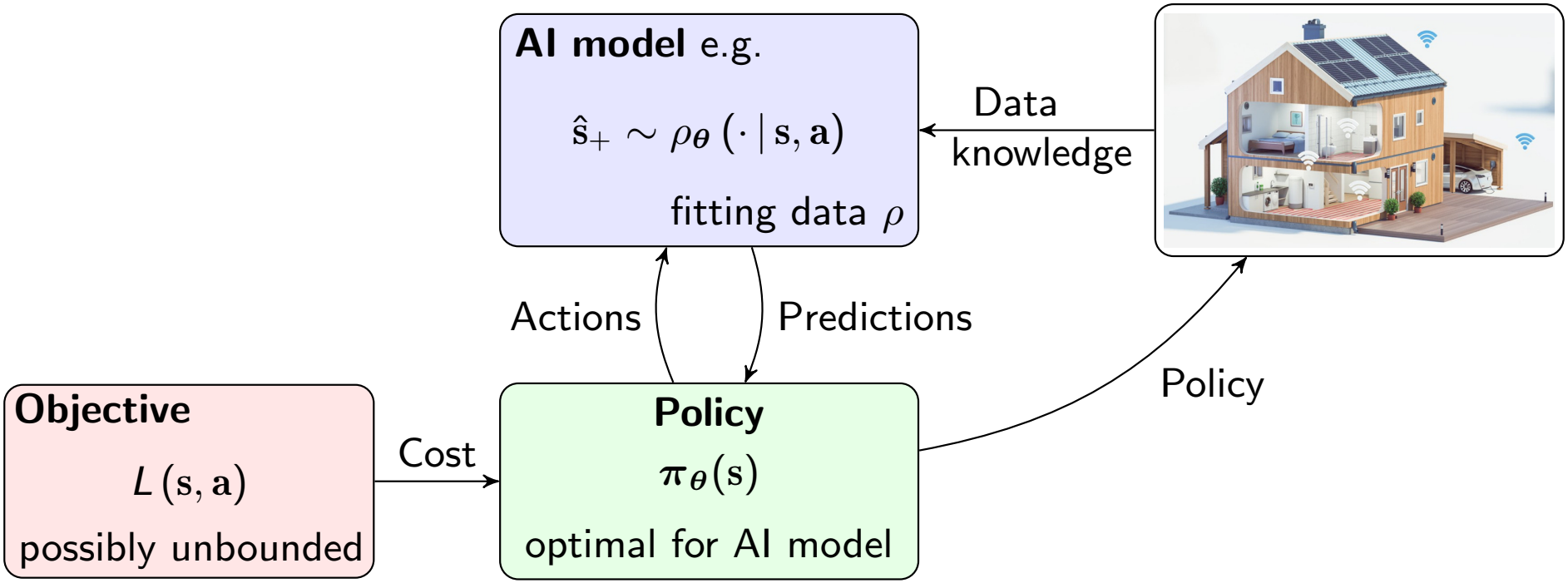


Outline

1. Decisions from data
2. AI models for Decision
3. Differentiating Policy Training
4. Examples



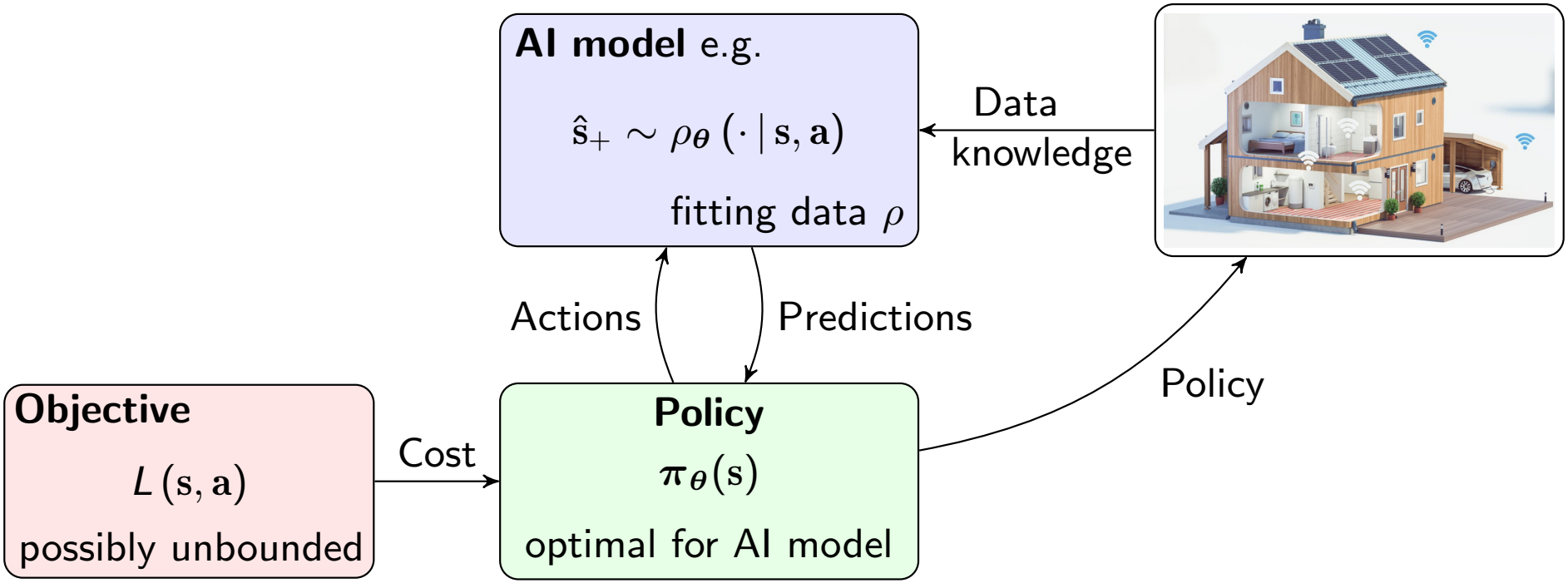
AI models for Decisions?



Learning model ρ_θ should aim at identifying

$$\boldsymbol{\theta}^\star = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\pi}_{\boldsymbol{\theta}}) \tag{1}$$

AI models for Decisions?

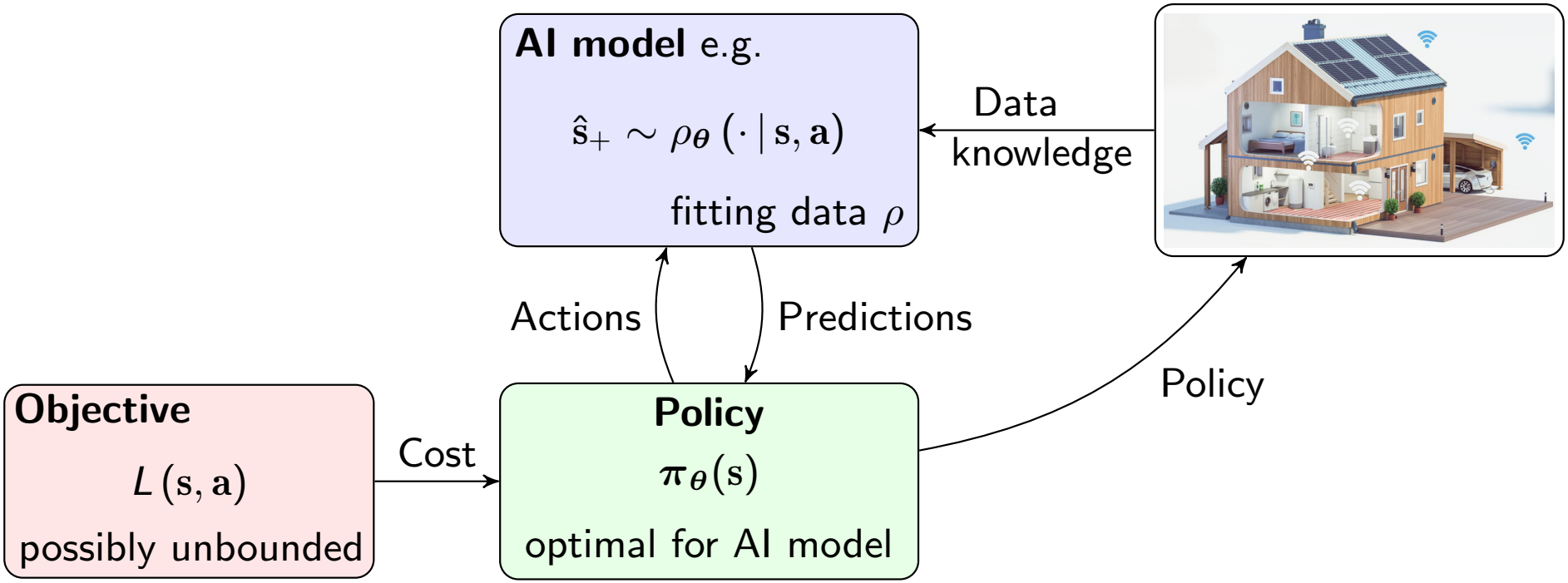


Learning model ρ_θ should aim at identifying

$$\boldsymbol{\theta}^\star = \arg\min_{\boldsymbol{\theta}} J(\pi_{\boldsymbol{\theta}}) \tag{1}$$

- Closed-loop performance J is intricate ($\boldsymbol{\theta} \rightarrow$ simulations $\rightarrow$ policy $\rightarrow$ real system)

AI models for Decisions?

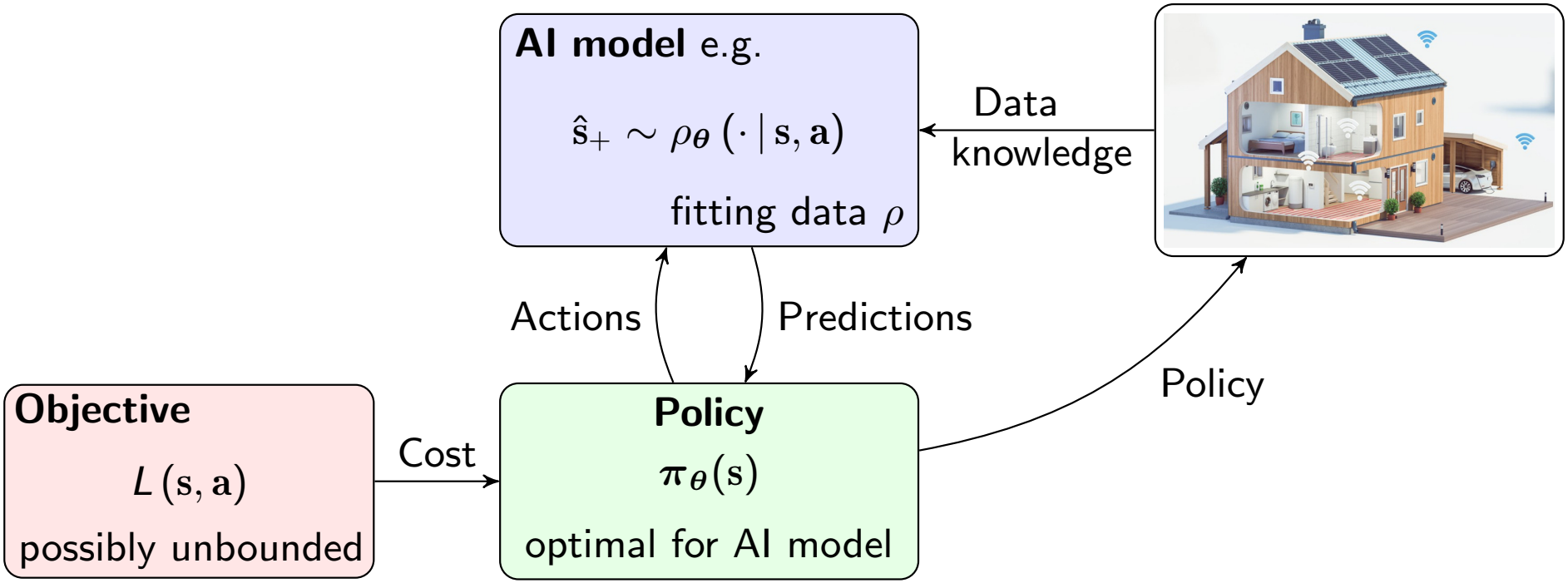


Learning model ρ_θ should aim at identifying

$$\boldsymbol{\theta}^\star = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\pi}_{\boldsymbol{\theta}}) \tag{1}$$

- Closed-loop performance J is intricate ($\boldsymbol{\theta} \rightarrow$ simulations $\rightarrow$ policy $\rightarrow$ real system)
- $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\pi}_{\boldsymbol{\theta}})$ is to be estimated from data, i.e. data hungry, noisy, possible biases

AI models for Decisions?

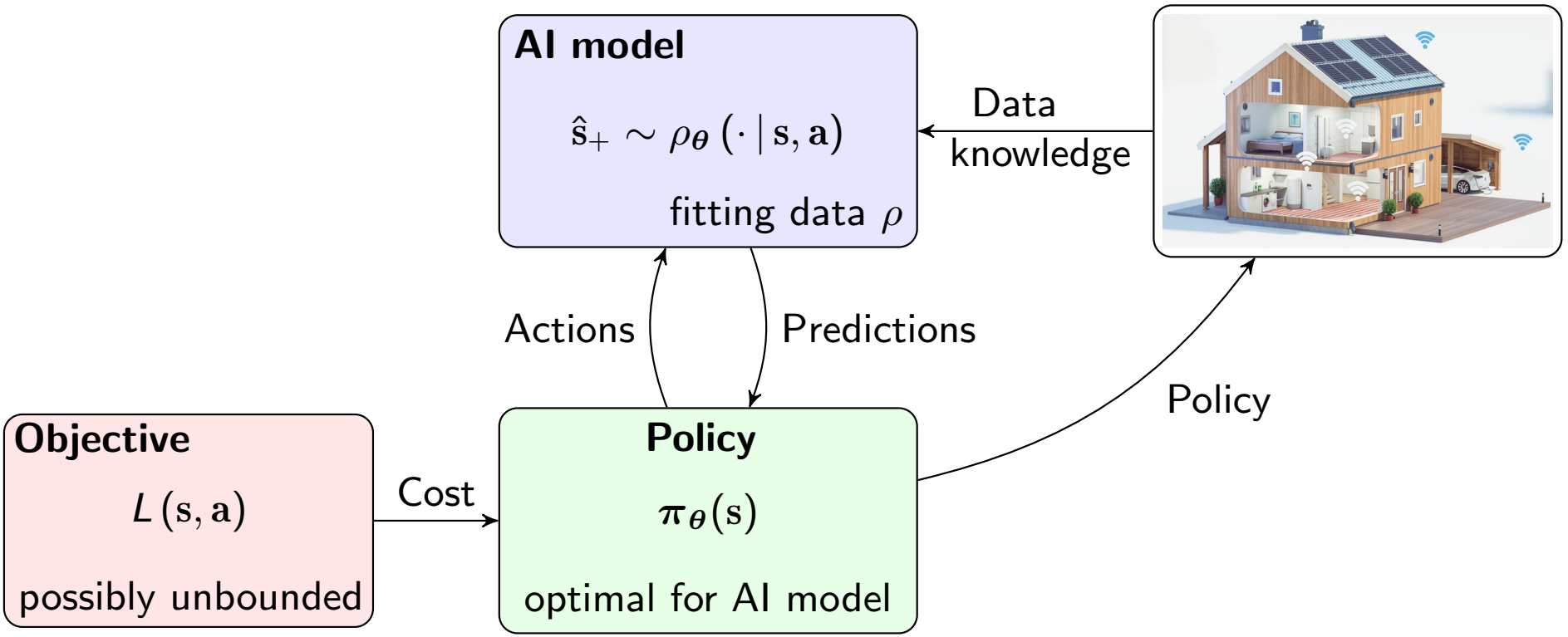


Learning model ρ_θ should aim at identifying

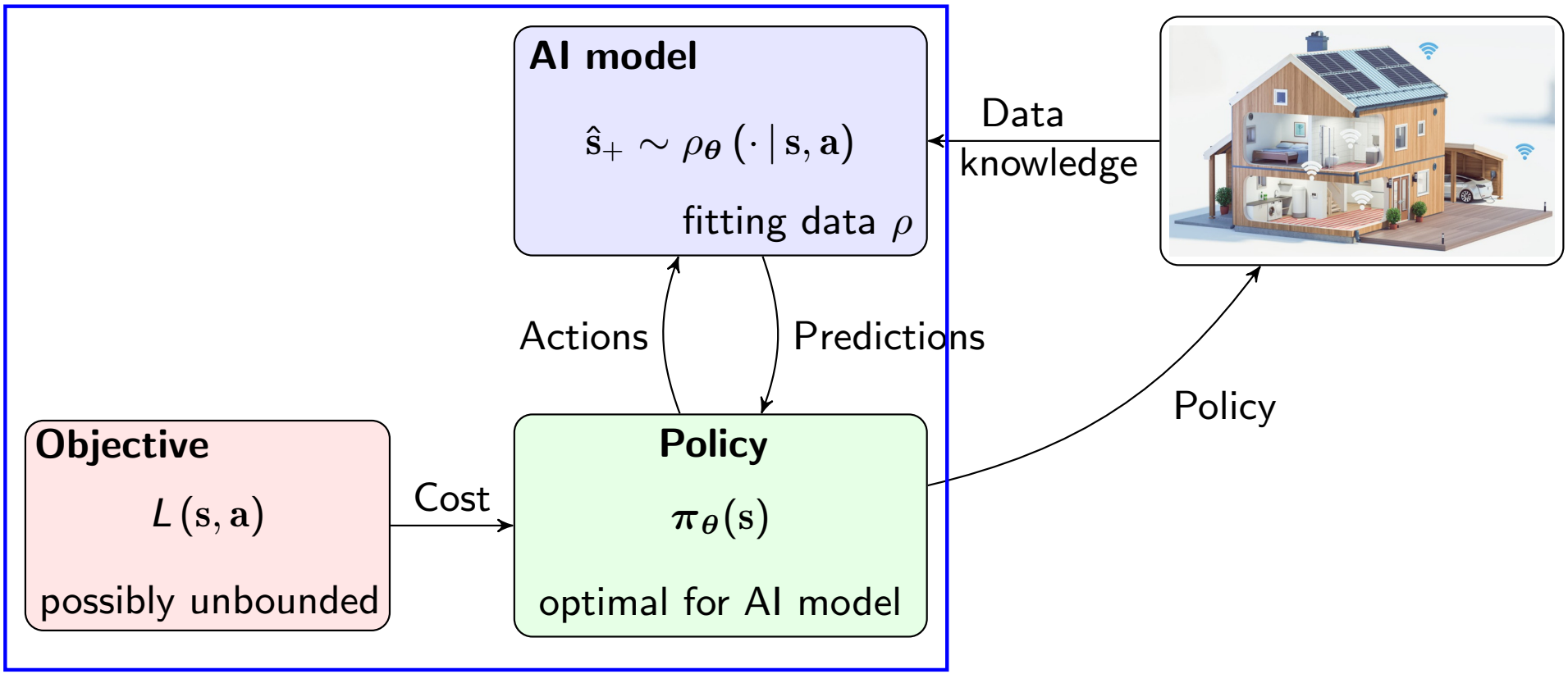
$$\boldsymbol{\theta}^\star = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\pi}_{\boldsymbol{\theta}}) \tag{1}$$

- Closed-loop performance J is intricate ($\boldsymbol{\theta} \rightarrow$ simulations $\rightarrow$ policy $\rightarrow$ real system)
- $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\pi}_{\boldsymbol{\theta}})$ is to be estimated from data, i.e. data hungry, noisy, possible biases
- (1) is in general different than model fitting, i.e. no loss function does (1)

Optimal AI models for Decision - A Paradigm Shift

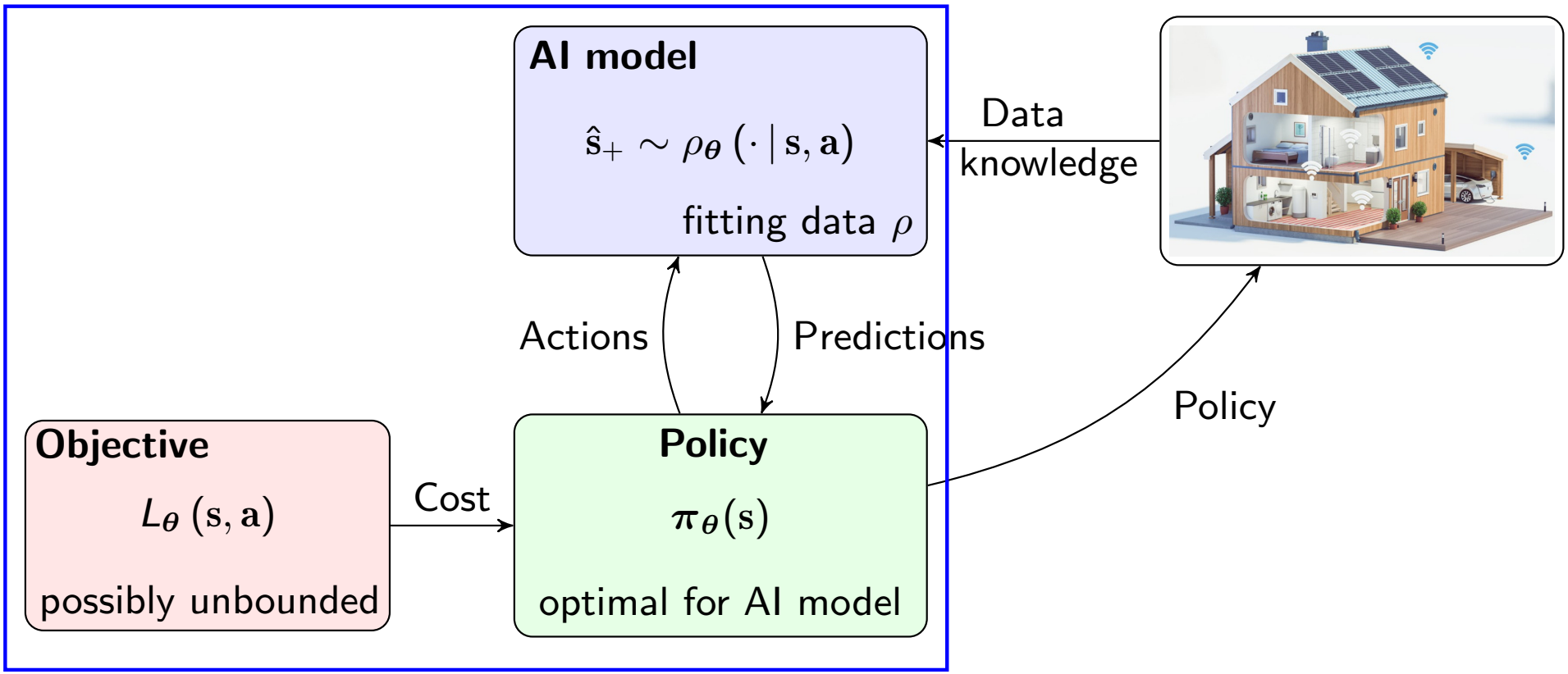


Optimal AI models for Decision - A Paradigm Shift



- The model is not $\rho_{\boldsymbol{\theta}}$, it is the entire "decision-making box"

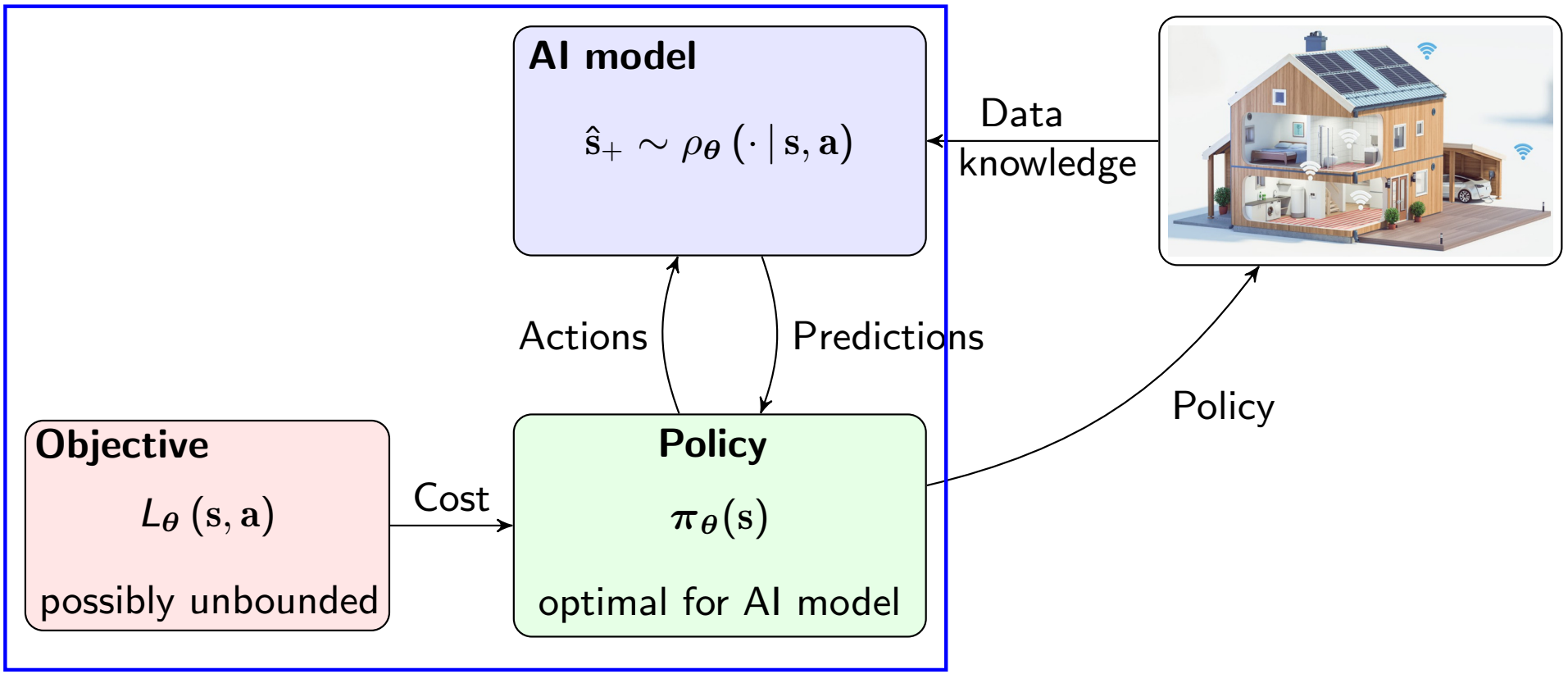
Optimal AI models for Decision - A Paradigm Shift



- The model is not $\rho_{\boldsymbol{\theta}}$, it is the entire "decision-making box"
- The model includes objective $L_{\boldsymbol{\theta}}$ used to build the policy*

* policy performance on real system still assessed via L

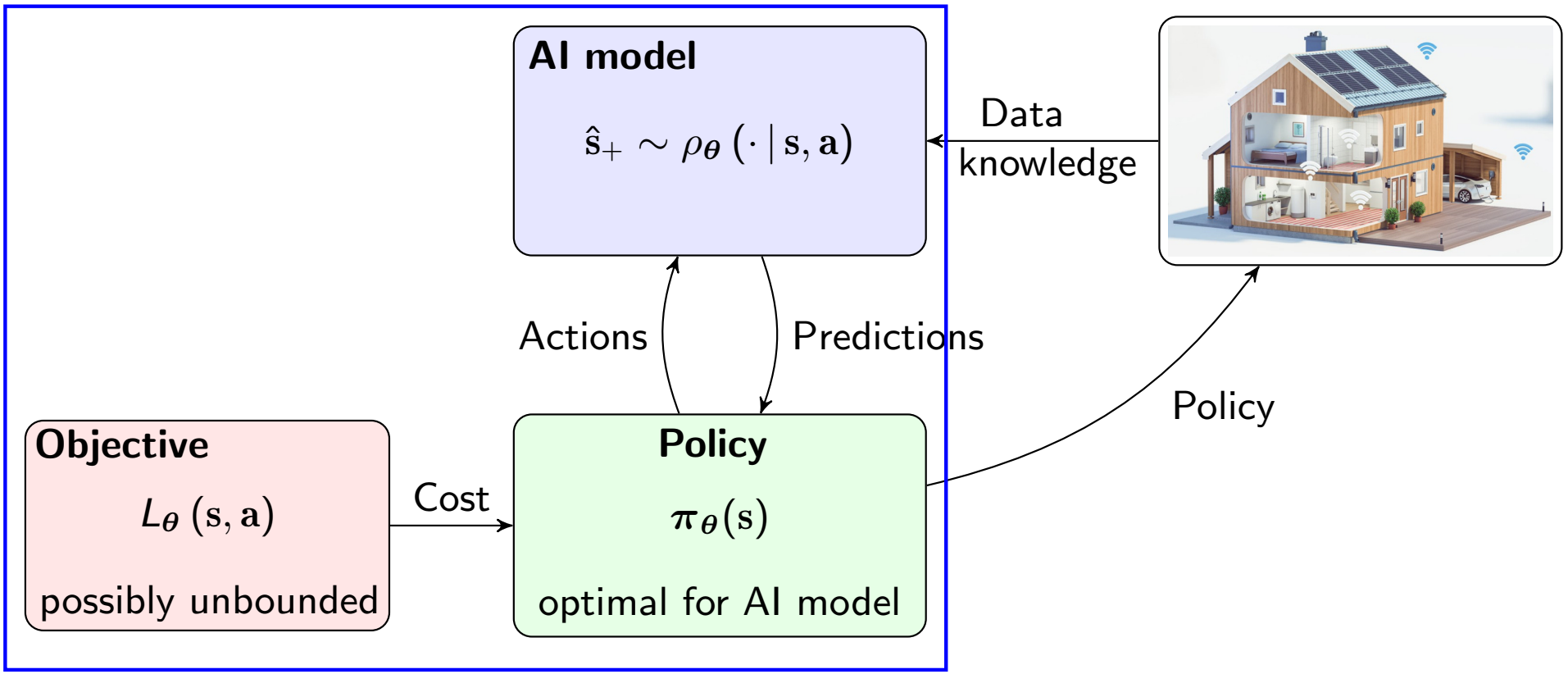
Optimal AI models for Decision - A Paradigm Shift



- The model is not $\rho_{\boldsymbol{\theta}}$, it is the entire "decision-making box"
- The model includes objective $L_{\boldsymbol{\theta}}$ used to build the policy*
- Best model $\rho_{\boldsymbol{\theta}}$ should *not necessarily* represent the data in a "classical sense"

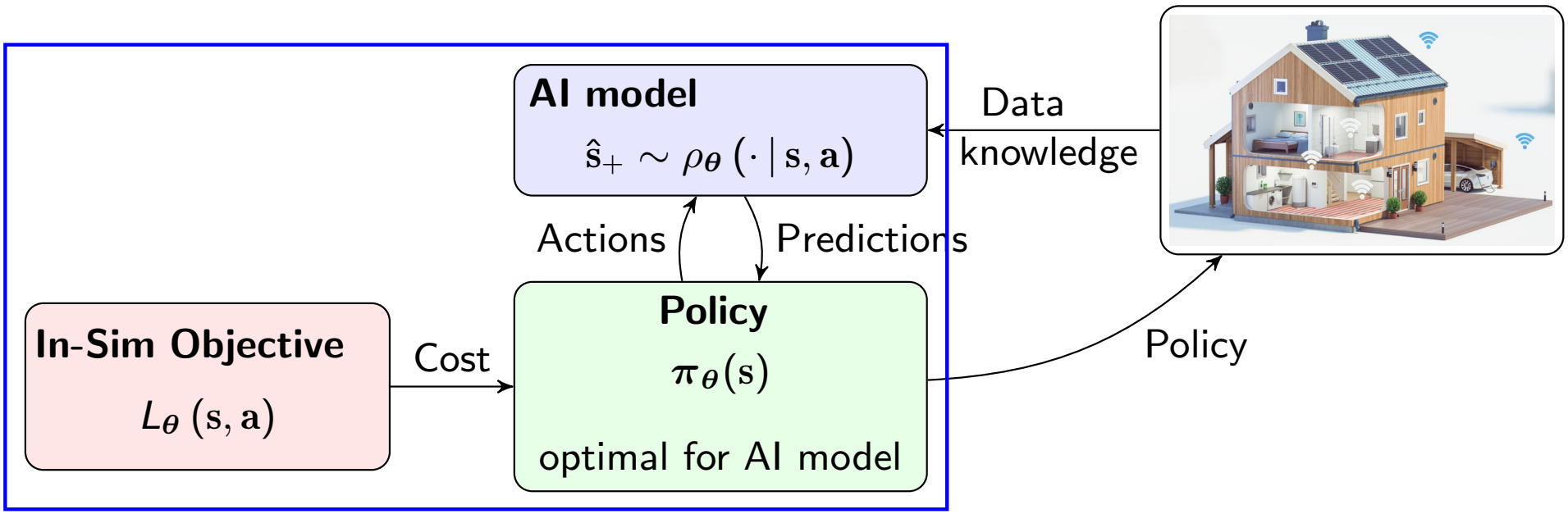
* policy performance on real system still assessed via L

Optimal AI models for Decision - A Paradigm Shift

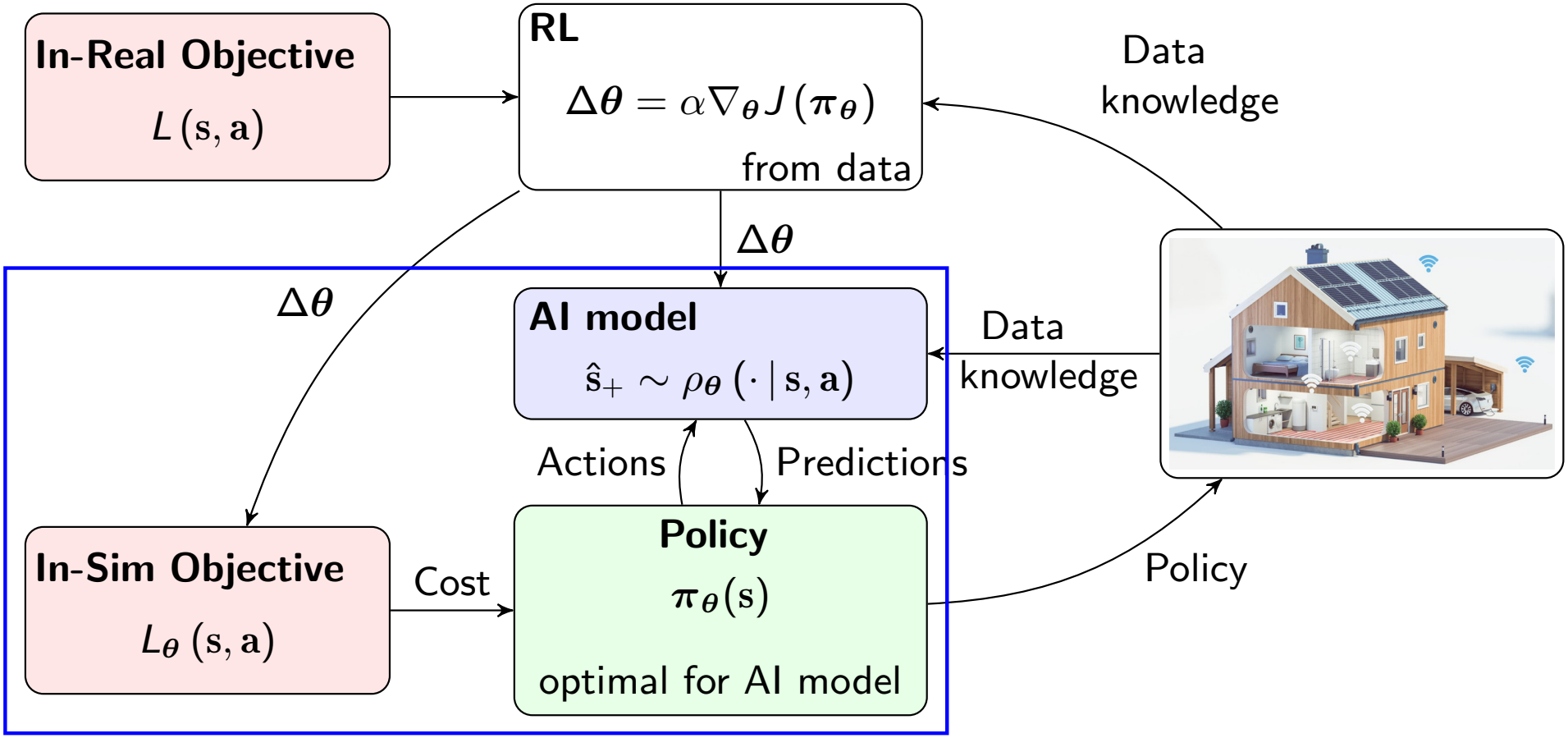


Theorem: under some (technical) assumptions, there is a $L_{\boldsymbol{\theta}}$ such that $\boldsymbol{\pi}_{\boldsymbol{\theta}} = \boldsymbol{\pi}_\star$, even if $\rho_{\boldsymbol{\theta}}$ does not represent real world ρ correctly

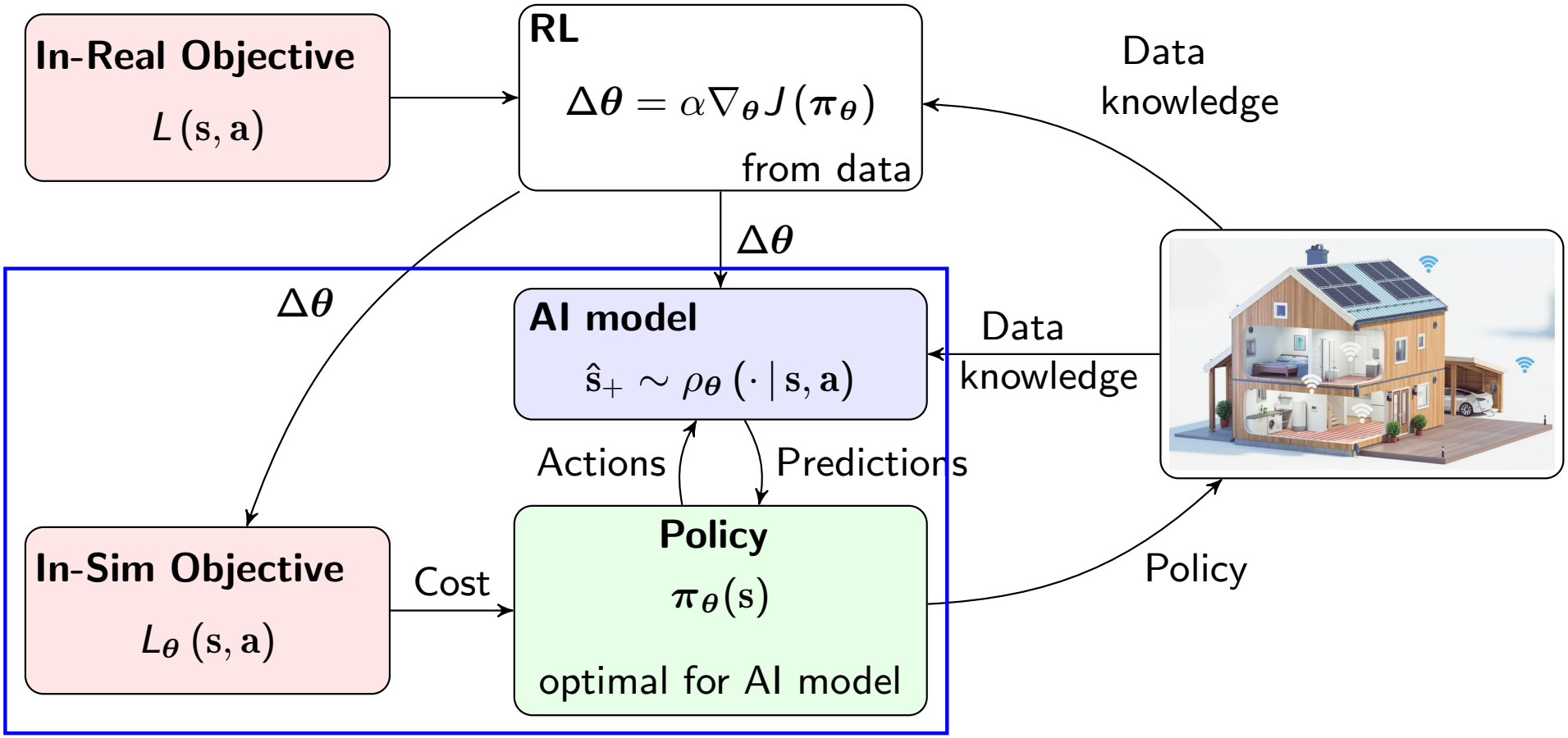
How to use this more generally? RL over Policy Training



How to use this more generally? RL over Policy Training



How to use this more generally? RL over Policy Training

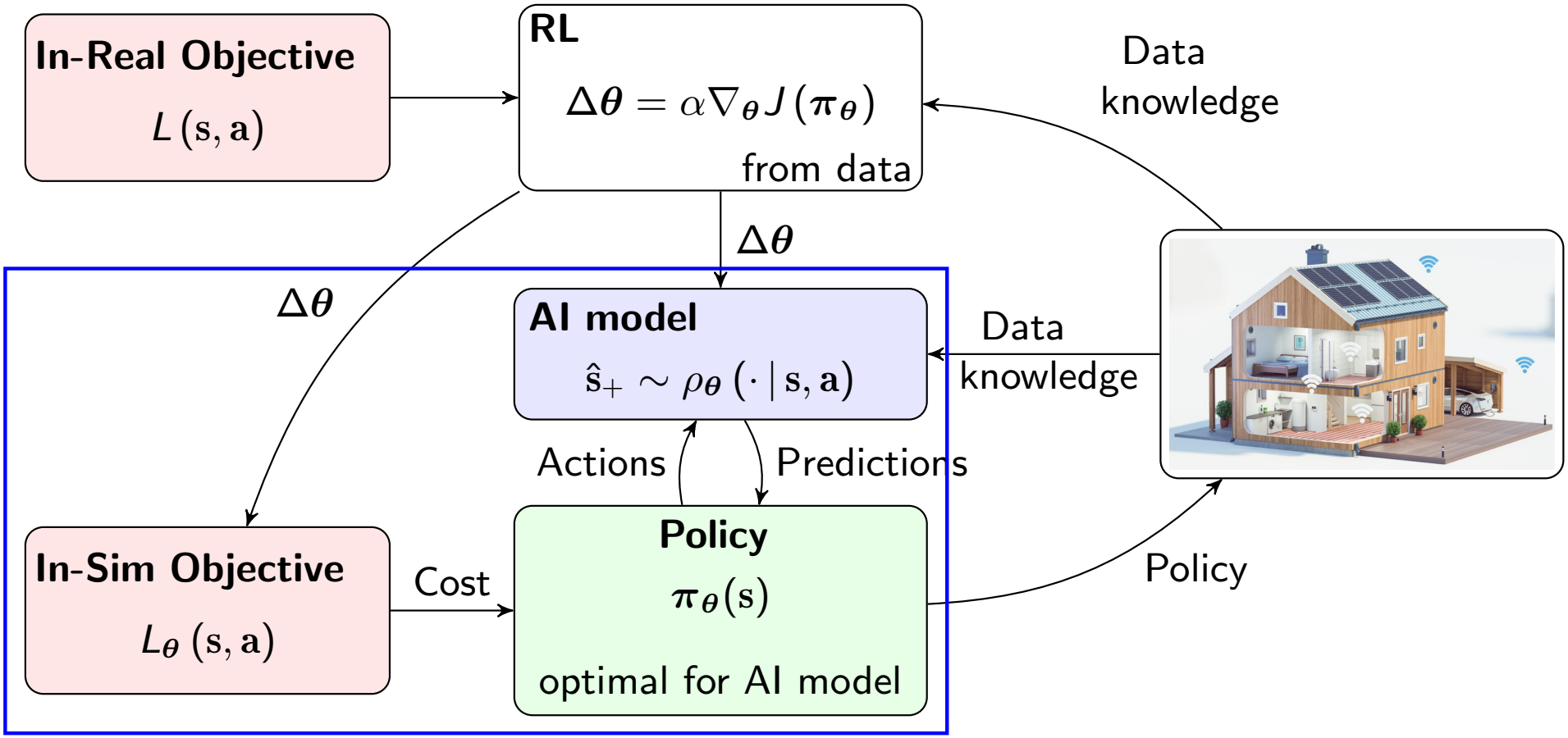


Policy gradient

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\pi}_{\boldsymbol{\theta}}) = \mathbb{E}\left[\nabla_{\boldsymbol{\theta}} \boldsymbol{\pi}_{\boldsymbol{\theta}} \nabla_{\mathbf{a}} Q^{\boldsymbol{\pi}_{\boldsymbol{\theta}}}\right]$$

- $Q^{\boldsymbol{\pi}_{\boldsymbol{\theta}}}$ is the critic, well-established RL tool
- $\nabla_{\boldsymbol{\theta}} \boldsymbol{\pi}_{\boldsymbol{\theta}}$ requires differentiating the closed-loop simulation...

How to use this more generally? RL over Policy Training



Difficulty: computing $\nabla_{\boldsymbol{\theta}} \boldsymbol{\pi}_{\boldsymbol{\theta}}$ requires total differentiation through the simulation-based policy optimization process:

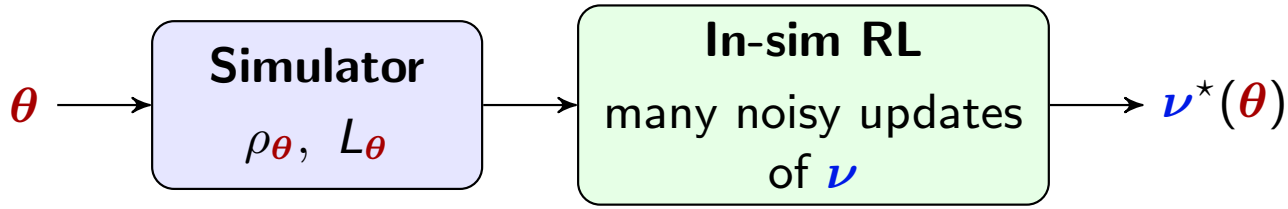
$\boldsymbol{\theta} \rightarrow$ simulations $\rightarrow$ optimal policy for $\rho_{\boldsymbol{\theta}}$

Differentiating through policy optimization can be computationally heavy

Outline

1. Decisions from data
2. AI models for Decision
3. Differentiating Policy Training
4. Examples

What we need: the sensitivity of a trained policy

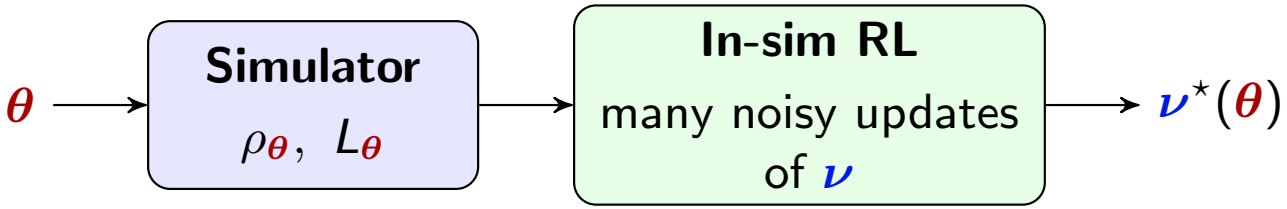


In-sim policy training, for a given $\boldsymbol{\theta}$

$$\boldsymbol{\nu}^\star(\boldsymbol{\theta}) = \arg\min_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu})$$

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) = \mathbb{E}_{\rho_{\boldsymbol{\theta}}, \pi_{\boldsymbol{\nu}}}\left[\sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)\right]$$

What we need: the sensitivity of a trained policy



In-sim policy training, for a given $\boldsymbol{\theta}$

$$\boldsymbol{\nu}^\star(\boldsymbol{\theta}) = \arg\min_{\boldsymbol{\nu}} \ \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu})$$

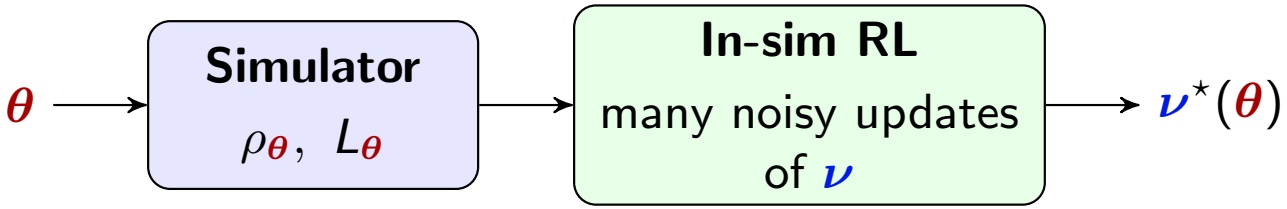
$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) = \mathbb{E}_{\rho_{\boldsymbol{\theta}}, \pi_{\boldsymbol{\nu}}}\left[\sum\nolimits_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)\right]$$

Policy sensitivity

$$\boldsymbol{\kappa} := \frac{\mathrm{d}\boldsymbol{\nu}^\star}{\mathrm{d}\boldsymbol{\theta}} \in \mathbb{R}^{\dim \boldsymbol{\nu} \times \dim \boldsymbol{\theta}}$$

This is the missing piece of the previous slide:
$\nabla_{\boldsymbol{\theta}} \boldsymbol{\pi}_{\boldsymbol{\theta}} = \nabla_{\boldsymbol{\nu}} \pi_{\boldsymbol{\nu}}\big|_{\boldsymbol{\nu}^\star} \boldsymbol{\kappa}$

What we need: the sensitivity of a trained policy



In-sim policy training, for a given $\boldsymbol{\theta}$

$$\boldsymbol{\nu}^\star(\boldsymbol{\theta}) = \arg\min_{\boldsymbol{\nu}} \ \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu})$$

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) = \mathbb{E}_{\rho_{\boldsymbol{\theta}}, \pi_{\boldsymbol{\nu}}}\left[\sum\nolimits_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)\right]$$

Policy sensitivity

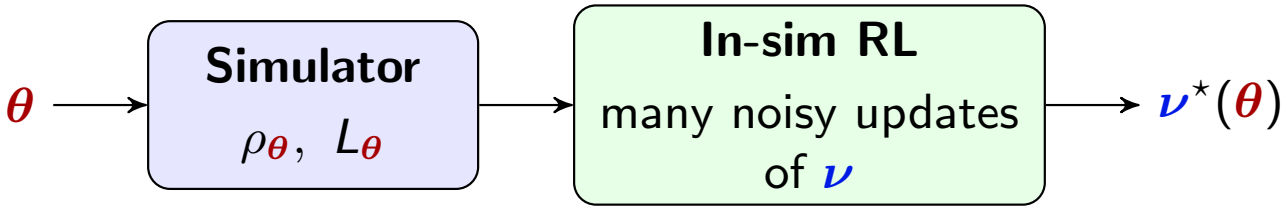
$$\boldsymbol{\kappa} := \frac{\mathrm{d}\boldsymbol{\nu}^\star}{\mathrm{d}\boldsymbol{\theta}} \in \mathbb{R}^{\dim \boldsymbol{\nu} \times \dim \boldsymbol{\theta}}$$

This is the missing piece of the previous slide:
$\nabla_{\boldsymbol{\theta}} \boldsymbol{\pi}_{\boldsymbol{\theta}} = \nabla_{\boldsymbol{\nu}} \pi_{\boldsymbol{\nu}}\big|_{\boldsymbol{\nu}^\star} \boldsymbol{\kappa}$

How to get $\boldsymbol{\kappa}$?

1. **Unroll training**: backpropagate through all updates $\boldsymbol{\nu}_{j+1} = \boldsymbol{\nu}_j - \beta\, \hat{\mathbf{g}}_j$
   ✘ cost $\propto$ number of updates

What we need: the sensitivity of a trained policy



In-sim policy training, for a given $\boldsymbol{\theta}$

$$\boldsymbol{\nu}^\star(\boldsymbol{\theta}) = \arg\min_{\boldsymbol{\nu}} \ \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu})$$

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) = \mathbb{E}_{\rho_{\boldsymbol{\theta}},\, \pi_{\boldsymbol{\nu}}}\left[\sum\nolimits_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)\right]$$

Policy sensitivity

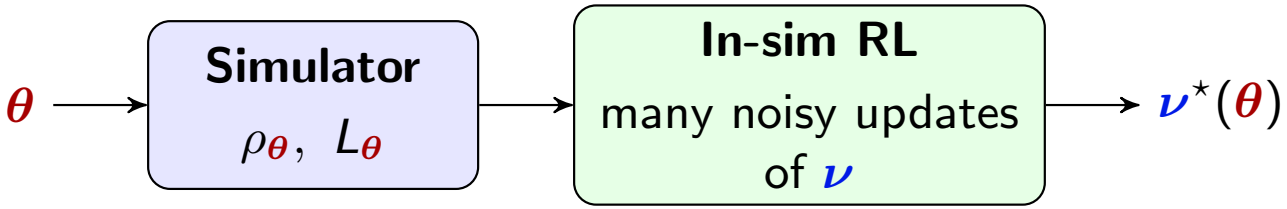
$$\boldsymbol{\kappa} := \frac{\mathrm{d}\boldsymbol{\nu}^\star}{\mathrm{d}\boldsymbol{\theta}} \in \mathbb{R}^{\dim \boldsymbol{\nu} \times \dim \boldsymbol{\theta}}$$

This is the missing piece of the previous slide:
$\nabla_{\boldsymbol{\theta}} \boldsymbol{\pi}_{\boldsymbol{\theta}} = \nabla_{\boldsymbol{\nu}} \pi_{\boldsymbol{\nu}}\big|_{\boldsymbol{\nu}^\star} \boldsymbol{\kappa}$

How to get $\boldsymbol{\kappa}$?

1. **Unroll training**: backpropagate through all updates $\boldsymbol{\nu}_{j+1} = \boldsymbol{\nu}_j - \beta\, \hat{\mathbf{g}}_j$
   ✘ cost $\propto$ number of updates
2. **Differentiate the Bellman optimality equations** (Nikishin et al., 2022)
   ✘ needs $\pi = \mathrm{softmax}(Q^\star)$

What we need: the sensitivity of a trained policy



In-sim policy training, for a given $\boldsymbol{\theta}$

$$\boldsymbol{\nu}^\star(\boldsymbol{\theta}) = \arg\min_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu})$$

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) = \mathbb{E}_{\rho_{\boldsymbol{\theta}}, \pi_{\boldsymbol{\nu}}}\left[\sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)\right]$$

Policy sensitivity

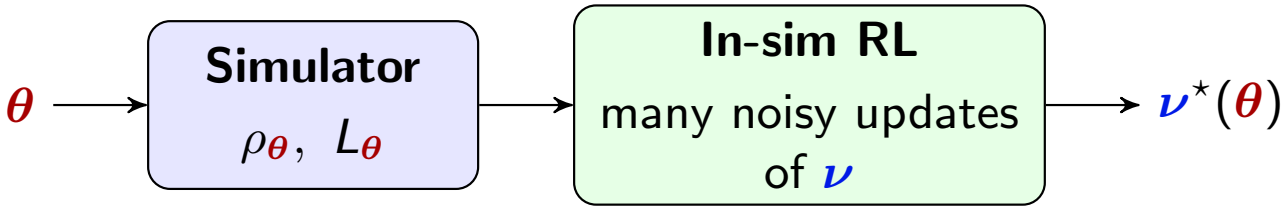
$$\boldsymbol{\kappa} := \frac{\mathrm{d}\boldsymbol{\nu}^\star}{\mathrm{d}\boldsymbol{\theta}} \in \mathbb{R}^{\dim \boldsymbol{\nu} \times \dim \boldsymbol{\theta}}$$

This is the missing piece of the previous slide:
$\nabla_{\boldsymbol{\theta}} \boldsymbol{\pi}_{\boldsymbol{\theta}} = \nabla_{\boldsymbol{\nu}} \pi_{\boldsymbol{\nu}}\big|_{\boldsymbol{\nu}^\star} \boldsymbol{\kappa}$

How to get $\boldsymbol{\kappa}$?

1. **Unroll training**: backpropagate through all updates $\boldsymbol{\nu}_{j+1} = \boldsymbol{\nu}_j - \beta\, \hat{\mathbf{g}}_j$
   ✘ cost $\propto$ number of updates
2. **Differentiate the Bellman optimality equations** (Nikishin et al., 2022)
   ✘ needs $\pi = \mathrm{softmax}(Q^\star)$
3. **Implicitly differentiate the end point of training**
   ✔ any policy class, any policy gradient

What we need: the sensitivity of a trained policy



In-sim policy training, for a given $\boldsymbol{\theta}$

$$\boldsymbol{\nu}^\star(\boldsymbol{\theta}) = \arg\min_{\boldsymbol{\nu}} \ \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu})$$

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) = \mathbb{E}_{\rho_{\boldsymbol{\theta}},\, \pi_{\boldsymbol{\nu}}}\left[\sum\nolimits_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)\right]$$

Policy sensitivity

$$\boldsymbol{\kappa} := \frac{\mathrm{d}\boldsymbol{\nu}^\star}{\mathrm{d}\boldsymbol{\theta}} \in \mathbb{R}^{\dim \boldsymbol{\nu} \times \dim \boldsymbol{\theta}}$$

This is the missing piece of the previous slide:
$\nabla_{\boldsymbol{\theta}} \boldsymbol{\pi}_{\boldsymbol{\theta}} = \nabla_{\boldsymbol{\nu}} \pi_{\boldsymbol{\nu}}\big|_{\boldsymbol{\nu}^\star} \boldsymbol{\kappa}$

How to get $\boldsymbol{\kappa}$?

1. **Unroll training**: backpropagate through all updates $\boldsymbol{\nu}_{j+1} = \boldsymbol{\nu}_j - \beta\, \hat{\mathbf{g}}_j$
   ✘ cost $\propto$ number of updates
2. **Differentiate the Bellman optimality equations** (Nikishin et al., 2022)
   ✘ needs $\pi = \mathrm{softmax}(Q^\star)$
3. **Implicitly differentiate the end point of training**
   ✔ any policy class, any policy gradient

So first: How do we differentiate through an MDP at all?

Two ways to differentiate an expectation

- **Problem**: estimate $\nabla_\psi \mathbb{E}_{x\sim p_\psi}\left[f(x)\right]$ from samples $x_1, \ldots, x_n \sim p_\psi$ (ψ: later $\boldsymbol{\theta}$ or $\boldsymbol{\nu}$)

Two ways to differentiate an expectation

- **Problem**: estimate $\nabla_\psi \mathbb{E}_{x\sim p_\psi}\left[f(x)\right]$ from samples $x_1, \ldots, x_n \sim p_\psi$ (ψ: later $\boldsymbol{\theta}$ or $\boldsymbol{\nu}$)
- **Idea**: a gradient is the change of the sample average when ψ moves a little

$$\nabla_\psi \mathbb{E}\big[f(x)\big] \approx \frac{\mathbb{E}_{\psi+\mathrm{d}\psi}[f] - \mathbb{E}_\psi[f]}{\mathrm{d}\psi}, \qquad \mathbb{E}_\psi[f] \approx \frac{1}{n}\sum_i f(x_i)$$

- **Question**: how to get the *new* average $\mathbb{E}_{\psi+\mathrm{d}\psi}[f]$ from the *same* samples?

Two ways to differentiate an expectation

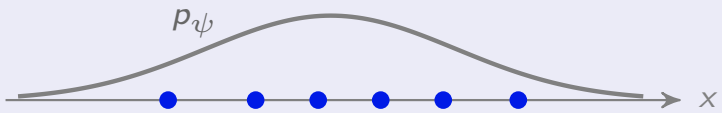
- **Problem**: estimate $\nabla_\psi \mathbb{E}_{x\sim p_\psi}\big[f(x)\big]$ from samples $x_1, \ldots, x_n \sim p_\psi$ (ψ: later $\boldsymbol{\theta}$ or $\boldsymbol{\nu}$)
- **Idea**: a gradient is the change of the sample average when ψ moves a little

$$\nabla_\psi \mathbb{E}\big[f(x)\big] \approx \frac{\mathbb{E}_{\psi+\mathrm{d}\psi}[f] - \mathbb{E}_\psi[f]}{\mathrm{d}\psi}, \qquad \mathbb{E}_\psi[f] \approx \frac{1}{n}\sum_i f(x_i)$$

- **Question**: how to get the *new* average $\mathbb{E}_{\psi+\mathrm{d}\psi}[f]$ from the *same* samples?

LR (likelihood ratio): **re-weight** samples

$$\nabla_\psi \mathbb{E}\big[f(x)\big] = \mathbb{E}\Big[f(x)\,\nabla_\psi \log p_\psi(x)\Big]$$



- samples $x_i \sim p_\psi$ stay where they are

Pictures: $\psi = \mu$, $x = \mu + \varepsilon$, $\varepsilon \sim \mathcal{N}(0,1)$; gray $= p_\psi$, dashed $= p_{\psi+\mathrm{d}\psi}$

Two ways to differentiate an expectation

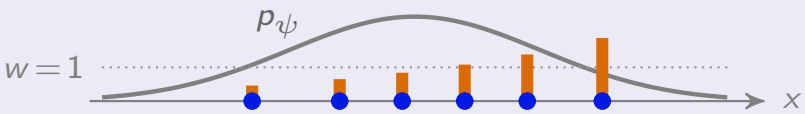
- **Problem**: estimate $\nabla_\psi \mathbb{E}_{x\sim p_\psi}\big[f(x)\big]$ from samples $x_1, \dots, x_n \sim p_\psi$ (ψ: later $\boldsymbol{\theta}$ or $\boldsymbol{\nu}$)
- **Idea**: a gradient is the change of the sample average when ψ moves a little

$$\nabla_\psi \mathbb{E}\big[f(x)\big] \approx \frac{\mathbb{E}_{\psi+\mathrm{d}\psi}[f] - \mathbb{E}_\psi[f]}{\mathrm{d}\psi}, \qquad \mathbb{E}_\psi[f] \approx \frac{1}{n}\sum_i f(x_i)$$

- **Question**: how to get the *new* average $\mathbb{E}_{\psi+\mathrm{d}\psi}[f]$ from the *same* samples?

LR (likelihood ratio): **re-weight** samples

$$\nabla_\psi \mathbb{E}\big[f(x)\big] = \mathbb{E}\Big[f(x)\,\nabla_\psi \log p_\psi(x)\Big]$$



- samples $x_i \sim p_\psi$ stay where they are
- $w_i = \frac{p_{\psi+\mathrm{d}\psi}}{p_\psi}(x_i) \approx 1 + \nabla_\psi \log p_\psi(x_i)\,\mathrm{d}\psi$

Pictures: $\psi = \mu$, $x = \mu + \varepsilon$, $\varepsilon \sim \mathcal{N}(0,1)$; gray $= p_\psi$, dashed $= p_{\psi+\mathrm{d}\psi}$

Two ways to differentiate an expectation

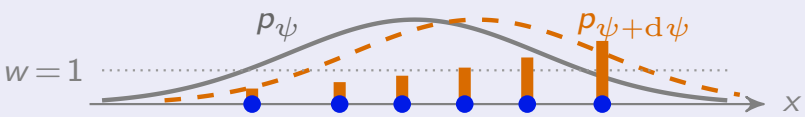
- **Problem**: estimate $\nabla_\psi \mathbb{E}_{x\sim p_\psi}\big[f(x)\big]$ from samples $x_1, \dots, x_n \sim p_\psi$ (ψ: later $\boldsymbol{\theta}$ or $\boldsymbol{\nu}$)
- **Idea**: a gradient is the change of the sample average when ψ moves a little

$$\nabla_\psi \mathbb{E}\big[f(x)\big] \approx \frac{\mathbb{E}_{\psi+\mathrm{d}\psi}[f] - \mathbb{E}_\psi[f]}{\mathrm{d}\psi}, \qquad \mathbb{E}_\psi[f] \approx \frac{1}{n}\sum_i f(x_i)$$

- **Question**: how to get the *new* average $\mathbb{E}_{\psi+\mathrm{d}\psi}[f]$ from the *same* samples?

LR (likelihood ratio): **re-weight** samples

$$\nabla_\psi \mathbb{E}\big[f(x)\big] = \mathbb{E}\Big[f(x)\,\nabla_\psi \log p_\psi(x)\Big]$$



- samples $x_i \sim p_\psi$ stay where they are
- $w_i = \frac{p_{\psi+\mathrm{d}\psi}}{p_\psi}(x_i) \approx 1 + \nabla_\psi \log p_\psi(x_i)\,\mathrm{d}\psi$
- new average $\approx \frac{1}{n}\sum_i w_i\, f(x_i)$

Pictures: $\psi = \mu$, $x = \mu + \varepsilon$, $\varepsilon \sim \mathcal{N}(0,1)$; gray $= p_\psi$, dashed $= p_{\psi+\mathrm{d}\psi}$

Two ways to differentiate an expectation

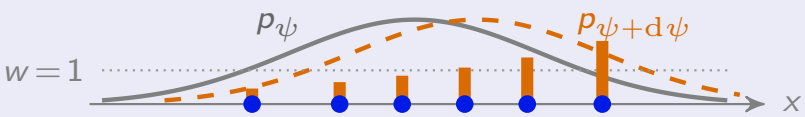
- **Problem**: estimate $\nabla_\psi \mathbb{E}_{x\sim p_\psi}\big[f(x)\big]$ from samples $x_1, \dots, x_n \sim p_\psi$ (ψ: later $\boldsymbol{\theta}$ or $\boldsymbol{\nu}$)
- **Idea**: a gradient is the change of the sample average when ψ moves a little

$$\nabla_\psi \mathbb{E}\big[f(x)\big] \approx \frac{\mathbb{E}_{\psi+\mathrm{d}\psi}[f] - \mathbb{E}_\psi[f]}{\mathrm{d}\psi}, \qquad \mathbb{E}_\psi[f] \approx \frac{1}{n}\sum_i f(x_i)$$

- **Question**: how to get the *new* average $\mathbb{E}_{\psi+\mathrm{d}\psi}[f]$ from the *same* samples?

LR (likelihood ratio): **re-weight** samples

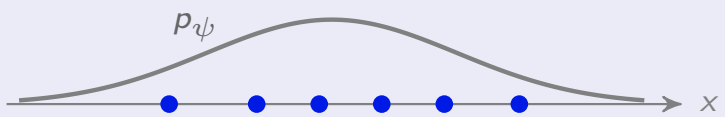
$$\nabla_\psi \mathbb{E}\big[f(x)\big] = \mathbb{E}\Big[f(x)\,\nabla_\psi \log p_\psi(x)\Big]$$



- samples $x_i \sim p_\psi$ stay where they are
- $w_i = \frac{p_{\psi+\mathrm{d}\psi}}{p_\psi}(x_i) \approx 1 + \nabla_\psi \log p_\psi(x_i)\,\mathrm{d}\psi$
- new average $\approx \frac{1}{n}\sum_i w_i\, f(x_i)$

PD (reparameterization): **move** samples

$$\nabla_\psi \mathbb{E}\big[f(x)\big] = \mathbb{E}_\varepsilon\Big[\nabla_x f(x)\,\nabla_\psi g_\psi(\varepsilon)\Big]$$



- samples $x_i = g_\psi(\varepsilon_i)$, noise $\varepsilon_i \sim p(\varepsilon)$

Pictures: $\psi = \mu$, $x = \mu + \varepsilon$, $\varepsilon \sim \mathcal{N}(0,1)$; gray $= p_\psi$, dashed $= p_{\psi+\mathrm{d}\psi}$

Two ways to differentiate an expectation

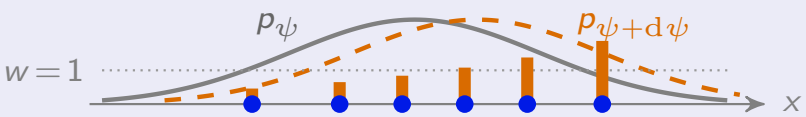
- **Problem**: estimate $\nabla_\psi \mathbb{E}_{x\sim p_\psi}\big[f(x)\big]$ from samples $x_1, \dots, x_n \sim p_\psi$ (ψ: later $\boldsymbol{\theta}$ or $\boldsymbol{\nu}$)
- **Idea**: a gradient is the change of the sample average when ψ moves a little

$$\nabla_\psi \mathbb{E}\big[f(x)\big] \approx \frac{\mathbb{E}_{\psi+\mathrm{d}\psi}[f] - \mathbb{E}_\psi[f]}{\mathrm{d}\psi}, \qquad \mathbb{E}_\psi[f] \approx \frac{1}{n}\sum_i f(x_i)$$

- **Question**: how to get the *new* average $\mathbb{E}_{\psi+\mathrm{d}\psi}[f]$ from the *same* samples?

LR (likelihood ratio): **re-weight** samples

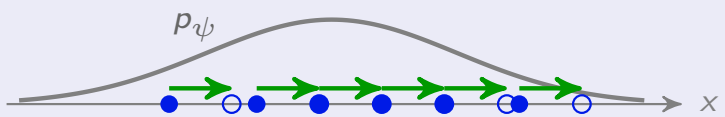
$$\nabla_\psi \mathbb{E}\big[f(x)\big] = \mathbb{E}\Big[f(x)\,\nabla_\psi \log p_\psi(x)\Big]$$



- samples $x_i \sim p_\psi$ stay where they are
- $w_i = \frac{p_{\psi+\mathrm{d}\psi}}{p_\psi}(x_i) \approx 1 + \nabla_\psi \log p_\psi(x_i)\,\mathrm{d}\psi$
- new average $\approx \frac{1}{n}\sum_i w_i\, f(x_i)$

PD (reparameterization): **move** samples

$$\nabla_\psi \mathbb{E}\big[f(x)\big] = \mathbb{E}_\varepsilon\Big[\nabla_x f(x)\,\nabla_\psi g_\psi(\varepsilon)\Big]$$



- samples $x_i = g_\psi(\varepsilon_i)$, noise $\varepsilon_i \sim p(\varepsilon)$
- keep ε_i, move: $x_i' \approx x_i + \nabla_\psi g_\psi(\varepsilon_i)\,\mathrm{d}\psi$

Pictures: $\psi = \mu$, $x = \mu + \varepsilon$, $\varepsilon \sim \mathcal{N}(0,1)$; gray $= p_\psi$, dashed $= p_{\psi+\mathrm{d}\psi}$

Two ways to differentiate an expectation

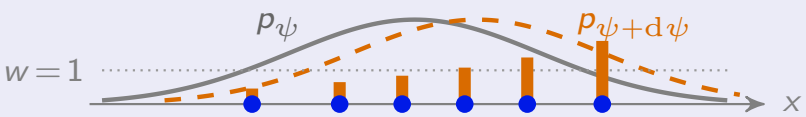
- **Problem**: estimate $\nabla_\psi \mathbb{E}_{x\sim p_\psi}\big[f(x)\big]$ from samples $x_1, \dots, x_n \sim p_\psi$ (ψ: later $\boldsymbol{\theta}$ or $\boldsymbol{\nu}$)
- **Idea**: a gradient is the change of the sample average when ψ moves a little

$$\nabla_\psi \mathbb{E}\big[f(x)\big] \approx \frac{\mathbb{E}_{\psi+\mathrm{d}\psi}[f] - \mathbb{E}_\psi[f]}{\mathrm{d}\psi}, \qquad \mathbb{E}_\psi[f] \approx \frac{1}{n}\sum_i f(x_i)$$

- **Question**: how to get the *new* average $\mathbb{E}_{\psi+\mathrm{d}\psi}[f]$ from the *same* samples?

LR (likelihood ratio): **re-weight** samples

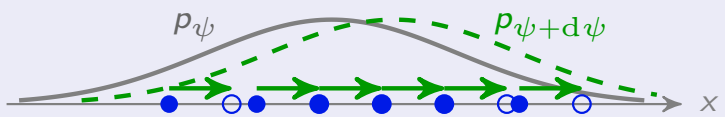
$$\nabla_\psi \mathbb{E}\big[f(x)\big] = \mathbb{E}\Big[f(x)\,\nabla_\psi \log p_\psi(x)\Big]$$



- samples $x_i \sim p_\psi$ stay where they are
- $w_i = \frac{p_{\psi+\mathrm{d}\psi}}{p_\psi}(x_i) \approx 1 + \nabla_\psi \log p_\psi(x_i)\,\mathrm{d}\psi$
- new average $\approx \frac{1}{n}\sum_i w_i\, f(x_i)$

PD (reparameterization): **move** samples

$$\nabla_\psi \mathbb{E}\big[f(x)\big] = \mathbb{E}_\varepsilon\Big[\nabla_x f(x)\,\nabla_\psi g_\psi(\varepsilon)\Big]$$



- samples $x_i = g_\psi(\varepsilon_i)$, noise $\varepsilon_i \sim p(\varepsilon)$
- keep ε_i, move: $x_i' \approx x_i + \nabla_\psi g_\psi(\varepsilon_i)\,\mathrm{d}\psi$
- new average $\approx \frac{1}{n}\sum_i f(x_i')$

Pictures: $\psi = \mu$, $x = \mu + \varepsilon$, $\varepsilon \sim \mathcal{N}(0,1)$; gray $= p_\psi$, dashed $= p_{\psi+\mathrm{d}\psi}$

Two families of policy gradients

RL: $x \rightarrow$ trajectory $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$, $\psi \rightarrow \boldsymbol{\nu}$, $f \rightarrow \sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)$

Two families of policy gradients

RL: $x \to$ trajectory $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$, $\psi \to \boldsymbol{\nu}$, $f \to \sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)$

LR (score function): **re-weight** trajectories

- τ has a **density**:
  $p(\tau) = \rho_0(\mathbf{s}_0) \prod_k \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$

Two families of policy gradients

RL: $x \to$ trajectory $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$, $\psi \to \boldsymbol{\nu}$, $f \to \sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)$

LR (score function): **re-weight** trajectories

- τ has a **density**:
  $p(\tau) = \rho_0(\mathbf{s}_0) \prod_k \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$
- score: $\nabla_{\boldsymbol{\nu}} \log p(\tau) = \sum_k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$

Two families of policy gradients

RL: $x \to$ trajectory $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$, $\psi \to \boldsymbol{\nu}$, $f \to \sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)$

LR (score function): **re-weight** trajectories

- τ has a **density**:
  $p(\tau) = \rho_0(\mathbf{s}_0) \prod_k \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$
- score: $\nabla_{\boldsymbol{\nu}} \log p(\tau) = \sum_k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$
  - the model $\rho_{\boldsymbol{\theta}}$ drops out

Two families of policy gradients

RL: $x \to$ trajectory $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$, $\psi \to \boldsymbol{\nu}$, $f \to \sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)$

LR (score function): **re-weight** trajectories

- τ has a **density**:
  $p(\tau) = \rho_0(\mathbf{s}_0) \prod_k \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$
- score: $\nabla_{\boldsymbol{\nu}} \log p(\tau) = \sum_k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$
  - the model $\rho_{\boldsymbol{\theta}}$ drops out

PD (pathwise): **move** trajectories

- τ is a **function** of fixed noise (ε_k, ξ_k):
  $\mathbf{a}_k = g_{\boldsymbol{\nu}}(\mathbf{s}_k, \varepsilon_k)$, $\mathbf{s}_{k+1} = f_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k, \xi_k)$

Two families of policy gradients

RL: $x \to$ trajectory $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$, $\psi \to \boldsymbol{\nu}$, $f \to \sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)$

LR (score function): **re-weight** trajectories

- τ has a **density**:
  $p(\tau) = \rho_0(\mathbf{s}_0) \prod_k \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$
- score: $\nabla_{\boldsymbol{\nu}} \log p(\tau) = \sum_k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$
  - the model $\rho_{\boldsymbol{\theta}}$ drops out

PD (pathwise): **move** trajectories

- τ is a **function** of fixed noise (ε_k, ξ_k):
  $\mathbf{a}_k = g_{\boldsymbol{\nu}}(\mathbf{s}_k, \varepsilon_k)$, $\mathbf{s}_{k+1} = f_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k, \xi_k)$
- gradients: $\frac{\mathrm{d}\mathbf{s}_{k+1}}{\mathrm{d}\boldsymbol{\nu}} = \nabla_{\mathbf{s}} f_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} f_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}$

Two families of policy gradients

RL: $x \to$ trajectory $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$, $\psi \to \boldsymbol{\nu}$, $f \to \sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)$

LR (score function): **re-weight** trajectories

- τ has a **density**:
  $p(\tau) = \rho_0(\mathbf{s}_0) \prod_k \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$
- score: $\nabla_{\boldsymbol{\nu}} \log p(\tau) = \sum_k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$
  - the model $\rho_{\boldsymbol{\theta}}$ drops out

PD (pathwise): **move** trajectories

- τ is a **function** of fixed noise (ε_k, ξ_k):
  $\mathbf{a}_k = g_{\boldsymbol{\nu}}(\mathbf{s}_k, \varepsilon_k)$, $\mathbf{s}_{k+1} = f_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k, \xi_k)$
- gradients: $\frac{\mathrm{d}\mathbf{s}_{k+1}}{\mathrm{d}\boldsymbol{\nu}} = \nabla_{\mathbf{s}} f_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} f_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}$
  - the model $f_{\boldsymbol{\theta}}$ stays in

Two families of policy gradients

RL: $x \to$ trajectory $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$, $\psi \to \boldsymbol{\nu}$, $f \to \sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)$

LR (score function): **re-weight** trajectories

- τ has a **density**:
  $p(\tau) = \rho_0(\mathbf{s}_0) \prod_k \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$
- score: $\nabla_{\boldsymbol{\nu}} \log p(\tau) = \sum_k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$
  - the model $\rho_{\boldsymbol{\theta}}$ drops out

PD (pathwise): **move** trajectories

- τ is a **function** of fixed noise (ε_k, ξ_k):
  $\mathbf{a}_k = g_{\boldsymbol{\nu}}(\mathbf{s}_k, \varepsilon_k)$, $\mathbf{s}_{k+1} = f_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k, \xi_k)$
- gradients: $\frac{\mathrm{d}\mathbf{s}_{k+1}}{\mathrm{d}\boldsymbol{\nu}} = \nabla_{\mathbf{s}} f_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} f_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}$
  - the model $f_{\boldsymbol{\theta}}$ stays in

The policy gradient in the two views

LR : $\nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}\Big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}(\mathbf{s}_k, \mathbf{a}_k)\Big]$ REINFORCE, A2C, PPO

Two families of policy gradients

RL: $x \to$ trajectory $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$, $\psi \to \boldsymbol{\nu}$, $f \to \sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)$

LR (score function): **re-weight** trajectories

- τ has a **density**: $p(\tau) = \rho_0(\mathbf{s}_0) \prod_k \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$
- score: $\nabla_{\boldsymbol{\nu}} \log p(\tau) = \sum_k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$
  - the model $\rho_{\boldsymbol{\theta}}$ drops out

PD (pathwise): **move** trajectories

- τ is a **function** of fixed noise (ε_k, ξ_k): $\mathbf{a}_k = g_{\boldsymbol{\nu}}(\mathbf{s}_k, \varepsilon_k)$, $\mathbf{s}_{k+1} = f_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k, \xi_k)$
- gradients: $\frac{\mathrm{d}\mathbf{s}_{k+1}}{\mathrm{d}\boldsymbol{\nu}} = \nabla_{\mathbf{s}} f_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} f_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}$
  - the model $f_{\boldsymbol{\theta}}$ stays in

The policy gradient in the two views

LR : $\nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}\Big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}(\mathbf{s}_k, \mathbf{a}_k)\Big]$ REINFORCE, A2C, PPO

PD : $\nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}_\omega\Big[\sum_k \gamma^k \big(\nabla_{\mathbf{s}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}\big)\Big]$ BPTT, SVG, SAC (SVG(0))

Two families of policy gradients

RL: $x \to$ trajectory $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \dots)$, $\psi \to \boldsymbol{\nu}$, $f \to \sum_k \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k)$

LR (score function): **re-weight** trajectories

- τ has a **density**: $p(\tau) = \rho_0(\mathbf{s}_0) \prod_k \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$
- score: $\nabla_{\boldsymbol{\nu}} \log p(\tau) = \sum_k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$
  - the model $\rho_{\boldsymbol{\theta}}$ drops out

PD (pathwise): **move** trajectories

- τ is a **function** of fixed noise (ε_k, ξ_k): $\mathbf{a}_k = g_{\boldsymbol{\nu}}(\mathbf{s}_k, \varepsilon_k)$, $\mathbf{s}_{k+1} = f_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k, \xi_k)$
- gradients: $\frac{\mathrm{d}\mathbf{s}_{k+1}}{\mathrm{d}\boldsymbol{\nu}} = \nabla_{\mathbf{s}} f_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} f_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}$
  - the model $f_{\boldsymbol{\theta}}$ stays in

The policy gradient in the two views

LR : $\nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}\Big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}(\mathbf{s}_k, \mathbf{a}_k)\Big]$ REINFORCE, A2C, PPO

PD : $\nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}_\omega\Big[\sum_k \gamma^k \big(\nabla_{\mathbf{s}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}\big)\Big]$ BPTT, SVG, SAC (SVG(0))

Two estimates of the same gradient, but the simulator $\boldsymbol{\theta}$ sits in different places:
LR: only in the **sampling** ($\rho_{\boldsymbol{\theta}}$) and the critic $Q_{\boldsymbol{\theta}}$; **PD**: inside each path, through the **Jacobians** of $f_{\boldsymbol{\theta}}$.
This decides how we will differentiate training with respect to $\boldsymbol{\theta}$ later.

LR or PD? Requirements, bias and variance

	LR (re-weight)	**PD** (move)
needs	samples and $\nabla_{\nu} \log \pi_{\nu}$	derivatives of f_{θ}, g_{ν}, L_{θ}

LR or PD? Requirements, bias and variance

	LR (re-weight)	**PD** (move)
needs	samples and $\nabla_{\nu} \log \pi_{\nu}$	derivatives of f_{θ}, g_{ν}, L_{θ}
simulator	black box is fine	must be differentiable

LR or PD? Requirements, bias and variance

	LR (re-weight)	**PD** (move)
needs	samples and $\nabla_{\nu} \log \pi_{\nu}$	derivatives of f_{θ}, g_{ν}, L_{θ}
simulator	black box is fine	must be differentiable
bias	none	at jumps (contacts, min/max)

LR or PD? Requirements, bias and variance

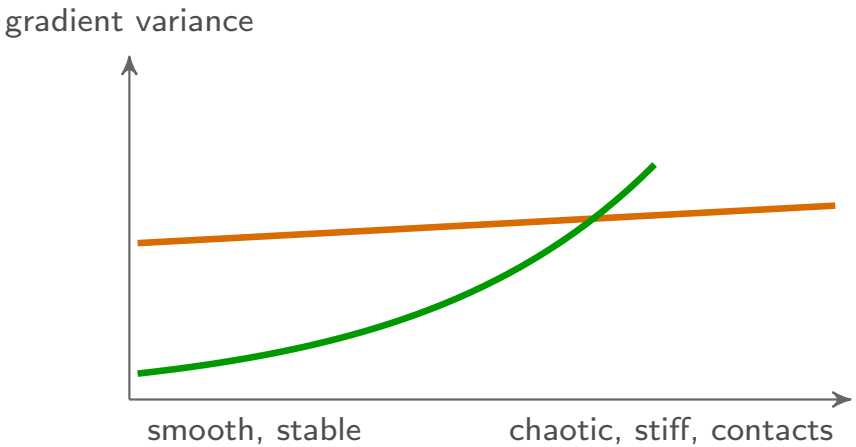
	LR (re-weight)	**PD** (move)
needs	samples and $\nabla_{\nu} \log \pi_{\nu}$	derivatives of f_{θ}, g_{ν}, L_{θ}
simulator	black box is fine	must be differentiable
bias	none	at jumps (contacts, min/max)
variance grows with	size of Q, horizon	products of Jacobians ∇f_{θ}

LR or PD? Requirements, bias and variance

	LR (re-weight)	**PD** (move)
needs	samples and $\nabla_{\nu} \log \pi_{\nu}$	derivatives of f_{θ}, g_{ν}, L_{θ}
simulator	black box is fine	must be differentiable
bias	none	at jumps (contacts, min/max)
variance grows with	size of Q, horizon	products of Jacobians ∇f_{θ}
reduce variance	baseline, critic: $Q - V$	short horizon + critic (SVG, SHAC)

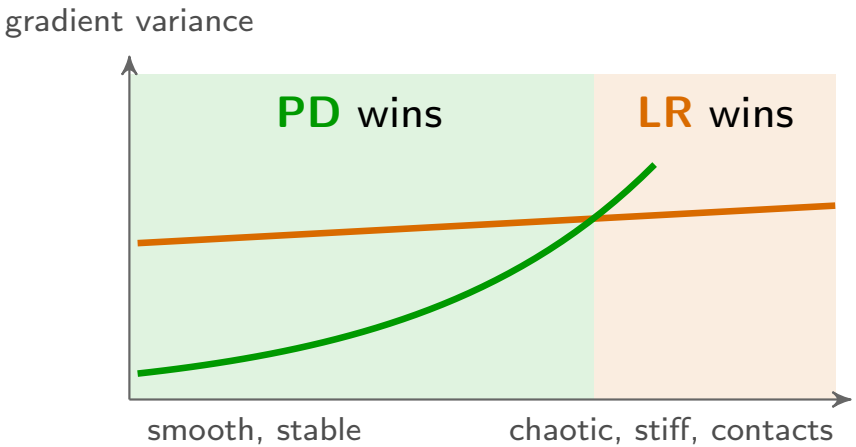
LR or PD? Requirements, bias and variance

	LR (re-weight)	**PD** (move)
needs	samples and $\nabla_{\nu} \log \pi_{\nu}$	derivatives of f_{θ}, g_{ν}, L_{θ}
simulator	black box is fine	must be differentiable
bias	none	at jumps (contacts, min/max)
variance grows with	size of Q, horizon	products of Jacobians ∇f_{θ}
reduce variance	baseline, critic: $Q - V$	short horizon + critic (SVG, SHAC)



LR or PD? Requirements, bias and variance

	LR (re-weight)	**PD** (move)
needs	samples and $\nabla_\nu \log \pi_\nu$	derivatives of f_θ, g_ν, L_θ
simulator	black box is fine	must be differentiable
bias	none	at jumps (contacts, min/max)
variance grows with	size of Q, horizon	products of Jacobians ∇f_θ
reduce variance	baseline, critic: $Q - V$	short horizon + critic (SVG, SHAC)

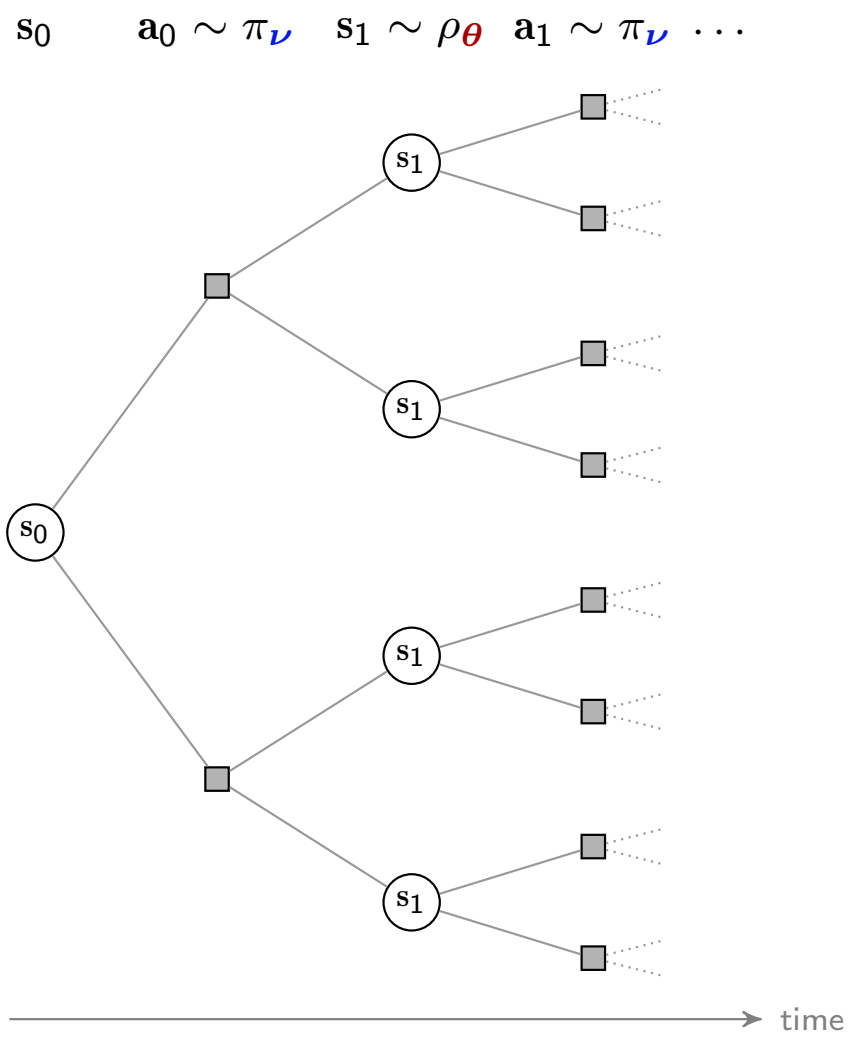


Neither always has lower variance.

PD is far less noisy on smooth, stable systems, but can be far noisier than **LR** on chaotic, stiff or contact-rich ones.

(Parmas et al., 2018; Suh et al., 2022)

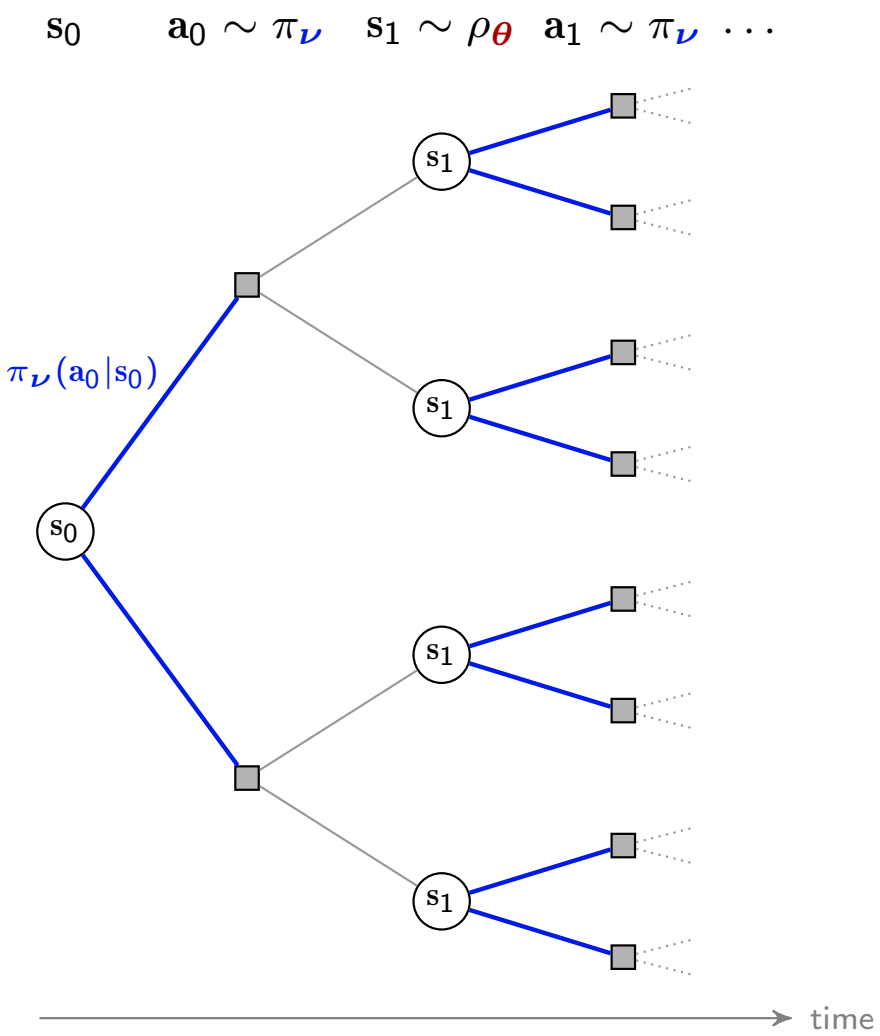
The Bellman computational tree



All possible histories: every root-to-leaf path is one trajectory; its probability is the product of its edge probabilities

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) = \sum_{\text{paths}} P(\text{path}) \sum_{k} \gamma^k L_{\boldsymbol{\theta}}$$

The Bellman computational tree

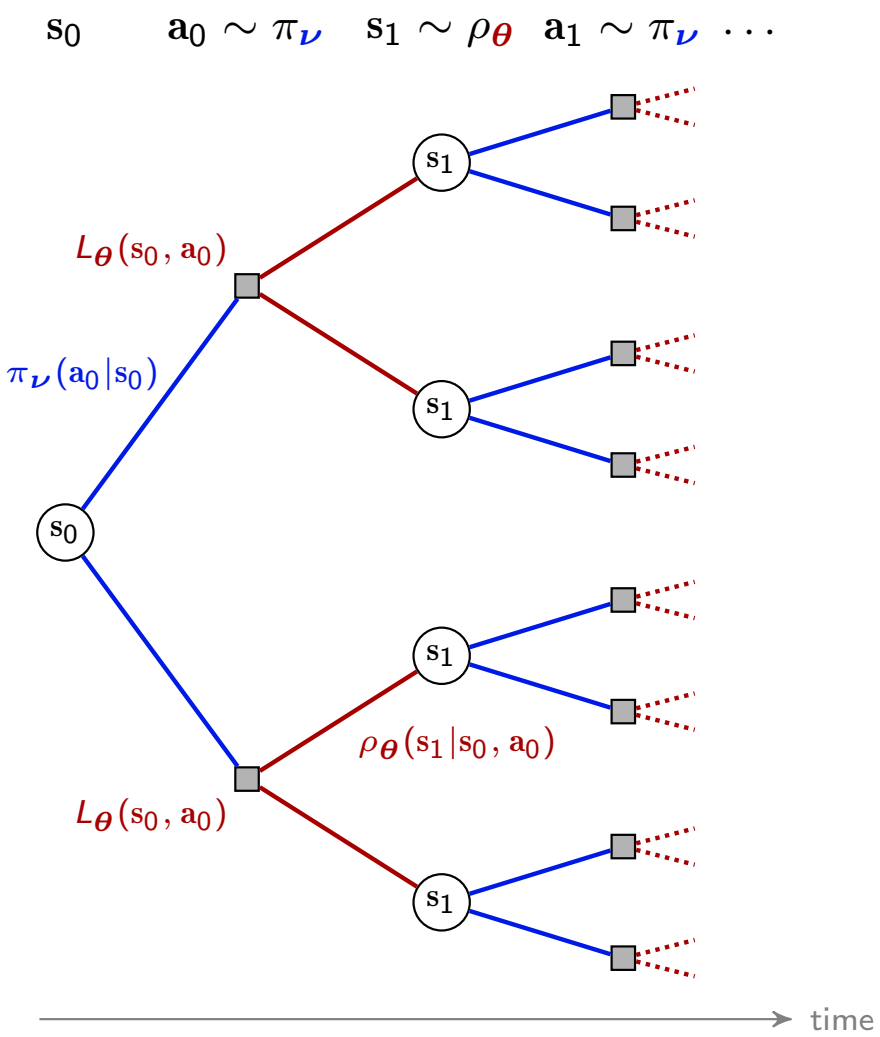


All possible histories: every root-to-leaf path is one trajectory; its probability is the product of its edge probabilities

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) = \sum_{\text{paths}} P(\text{path}) \sum_{k} \gamma^k L_{\boldsymbol{\theta}}$$

- **Policy edges** $\pi_{\boldsymbol{\nu}}(\mathbf{a}|\mathbf{s})$: $\boldsymbol{\nu}$ lives here

The Bellman computational tree

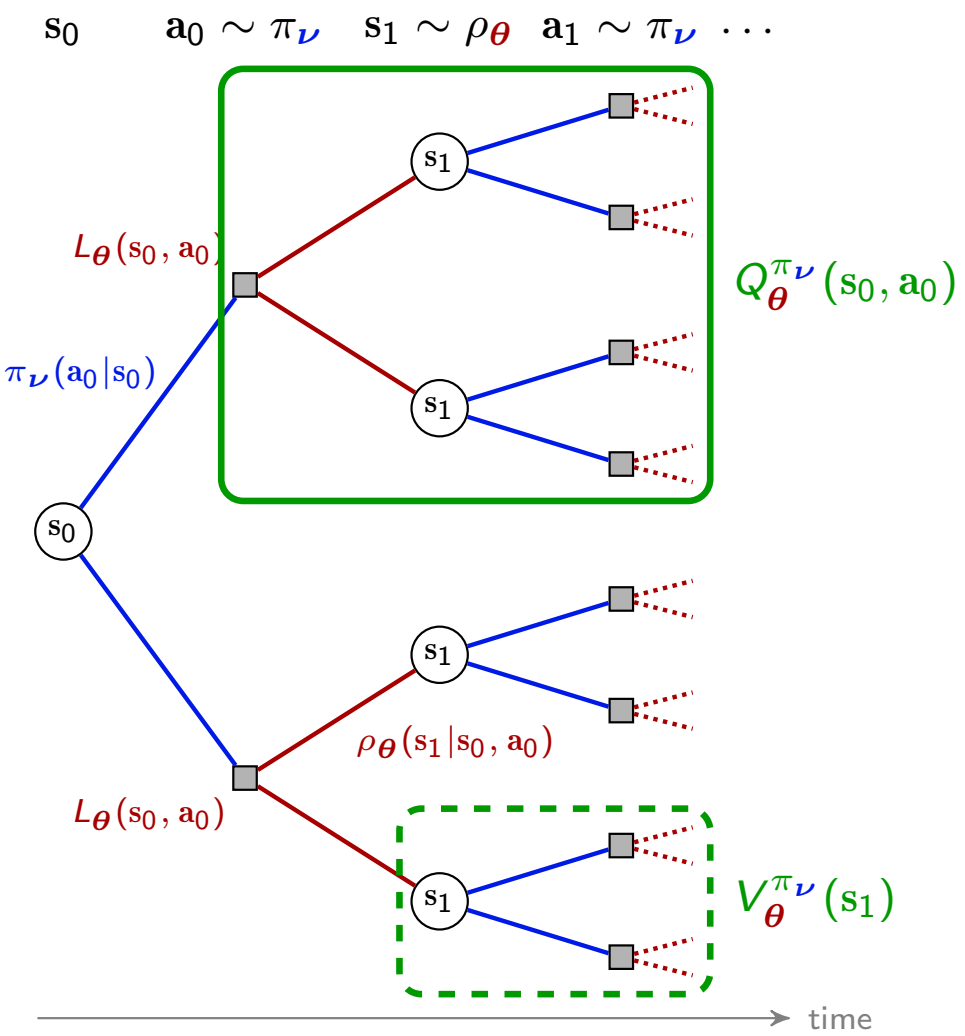


All possible histories: every root-to-leaf path is one trajectory; its probability is the product of its edge probabilities

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) = \sum_{\text{paths}} P(\text{path}) \sum_{k} \gamma^k L_{\boldsymbol{\theta}}$$

- **Policy edges** $\pi_{\boldsymbol{\nu}}(\mathbf{a}|\mathbf{s})$: $\boldsymbol{\nu}$ lives here
- **Model edges** $\rho_{\boldsymbol{\theta}}(\mathbf{s}'|\mathbf{s}, \mathbf{a})$ and **costs** $L_{\boldsymbol{\theta}}(\mathbf{s}, \mathbf{a})$ at the action nodes: $\boldsymbol{\theta}$ lives here

The Bellman computational tree



All possible histories: every root-to-leaf path is one trajectory; its probability is the product of its edge probabilities

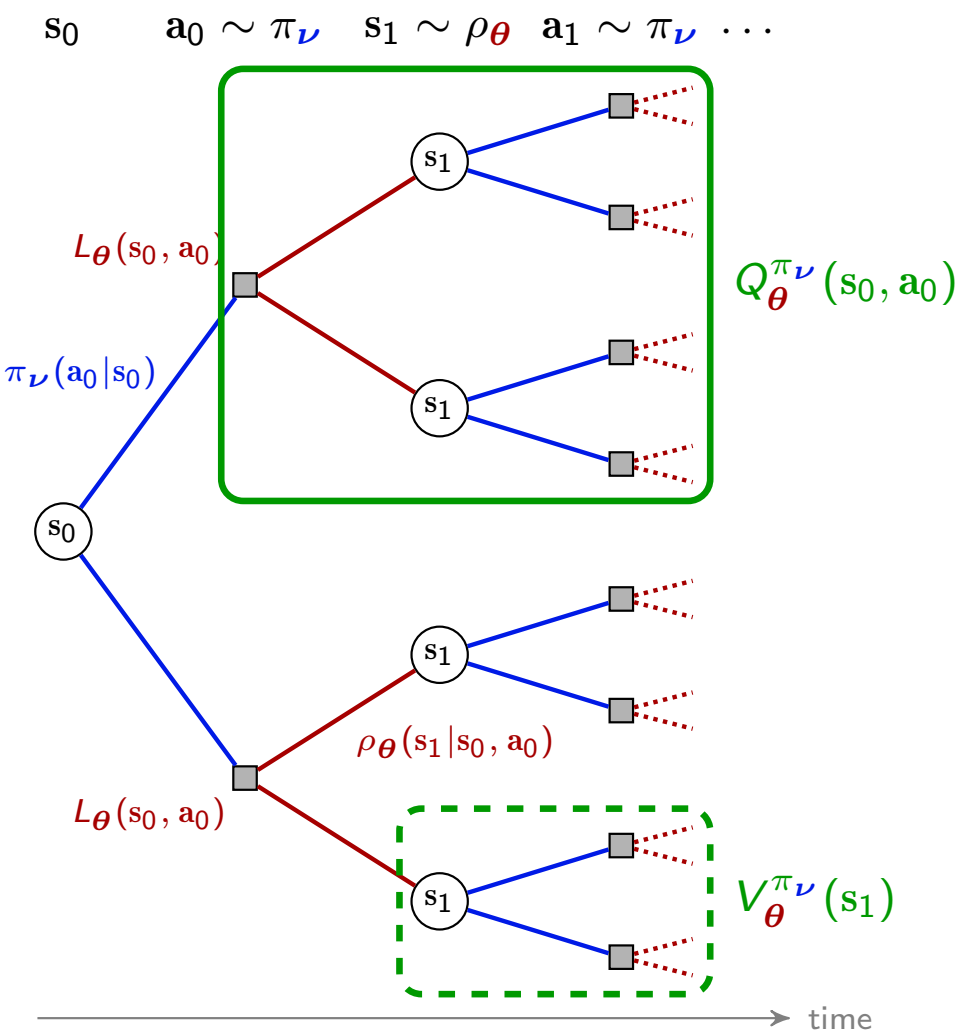
$$\hat{J}_\theta(\nu) = \sum_{\text{paths}} P(\text{path}) \sum_k \gamma^k L_\theta$$

- **Policy edges** $\pi_\nu(\mathbf{a}|\mathbf{s})$: ν lives here
- **Model edges** $\rho_\theta(\mathbf{s}'|\mathbf{s}, \mathbf{a})$ and **costs** $L_\theta(\mathbf{s}, \mathbf{a})$ at the action nodes: θ lives here

Value of a subtree (Bellman)

$$Q(\mathbf{s}, \mathbf{a}) = L_\theta(\mathbf{s}, \mathbf{a}) + \gamma \, \mathbb{E}_{\mathbf{s}' \sim \rho_\theta}\left[V(\mathbf{s}')\right]$$
$$V(\mathbf{s}) = \mathbb{E}_{\mathbf{a} \sim \pi_\nu}\left[Q(\mathbf{s}, \mathbf{a})\right]$$

The Bellman computational tree



All possible histories: every root-to-leaf path is one trajectory; its probability is the product of its edge probabilities

$$\hat{J}_\theta(\nu) = \sum_{\text{paths}} P(\text{path}) \sum_k \gamma^k L_\theta$$

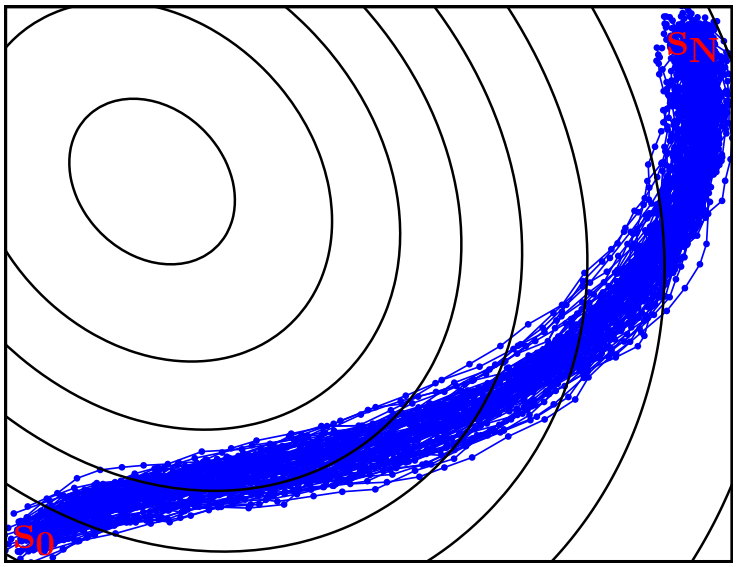
- **Policy edges** $\pi_\nu(a|s)$: ν lives here
- **Model edges** $\rho_\theta(s'|s,a)$ and **costs** $L_\theta(s,a)$ at the action nodes: θ lives here

Value of a subtree (Bellman)

$$Q(s,a) = L_\theta(s,a) + \gamma\, \mathbb{E}_{s' \sim \rho_\theta}\left[V(s')\right]$$
$$V(s) = \mathbb{E}_{a \sim \pi_\nu}\left[Q(s,a)\right]$$

In the **LR** policy gradient $\mathbb{E}\left[\sum_k \gamma^k \nabla_\nu \log \pi_\nu\, Q\right]$, θ acts in two places: the **edges above** a node (which histories are sampled) and the **subtree below** it (Q). **Differentiating training with respect to θ means tracking both.**

Stochastic trajectories: sampling the tree

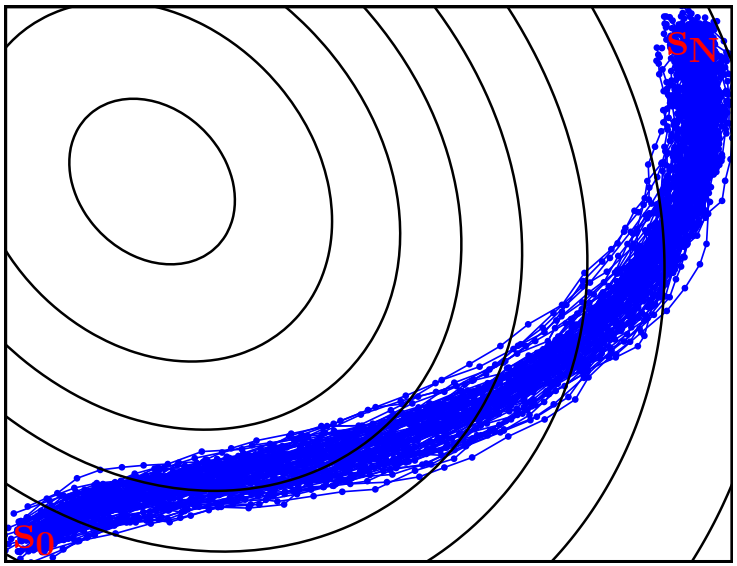


The tree has $(|\mathcal{A}||\mathcal{S}|)^N$ paths: we never enumerate it, we **sample** it.

n simulated closed-loop trajectories from s_0 to s_N = n sampled root-to-leaf paths of the tree

$$\hat{J}_{\theta}(\nu) \approx \frac{1}{n} \sum_{i=1}^{n} \sum_{k=0}^{N} \gamma^k L_{\theta}(\mathbf{s}_k^i, \mathbf{a}_k^i)$$

Stochastic trajectories: sampling the tree



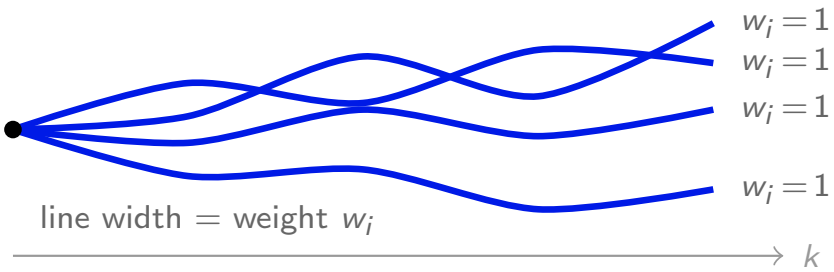
The tree has $(|\mathcal{A}||\mathcal{S}|)^N$ paths: we never enumerate it, we **sample** it.

n simulated closed-loop trajectories from s_0 to s_N = n sampled root-to-leaf paths of the tree

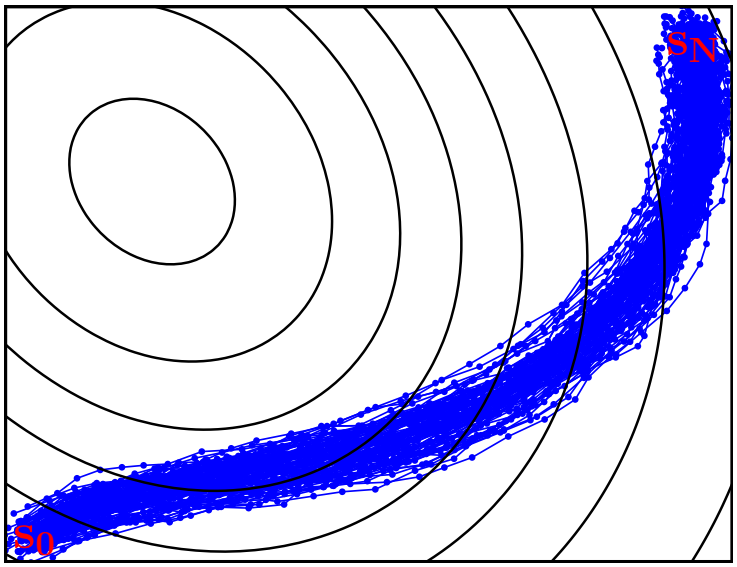
$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) \approx \frac{1}{n} \sum_{i=1}^{n} \sum_{k=0}^{N} \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k^i, \mathbf{a}_k^i)$$

Perturb $\boldsymbol{\theta} \to \boldsymbol{\theta} + \mathrm{d}\boldsymbol{\theta}$: what happens to the samples?

LR: same paths, **new weights**



Stochastic trajectories: sampling the tree



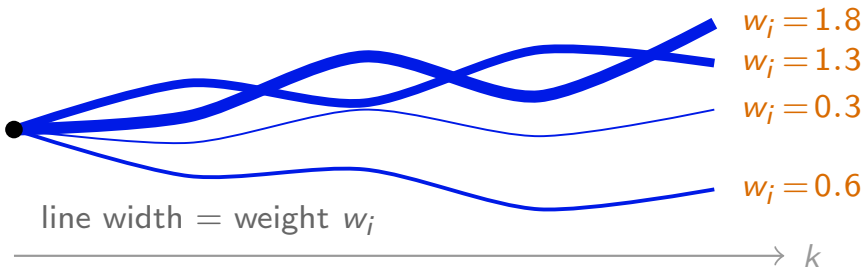
The tree has $(|\mathcal{A}||\mathcal{S}|)^N$ paths: we never enumerate it, we **sample** it.

n simulated closed-loop trajectories from s_0 to s_N = n sampled root-to-leaf paths of the tree

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) \approx \frac{1}{n} \sum_{i=1}^{n} \sum_{k=0}^{N} \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k^i, \mathbf{a}_k^i)$$

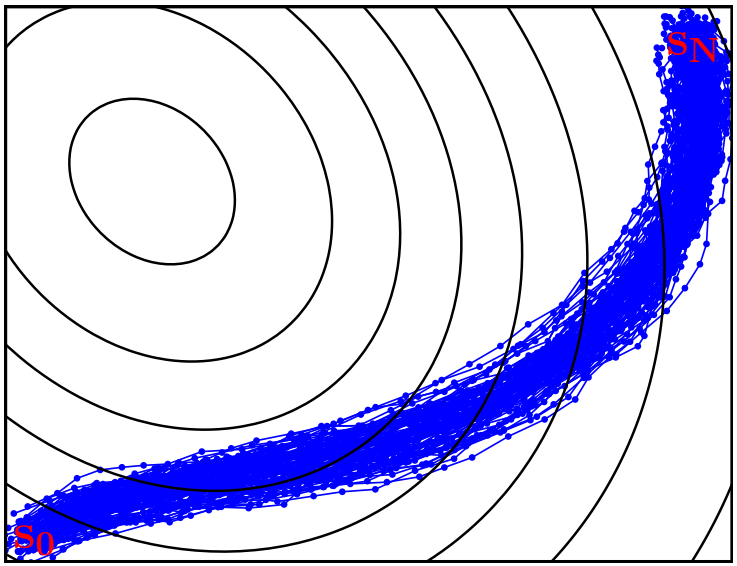
Perturb $\boldsymbol{\theta} \to \boldsymbol{\theta} + \mathrm{d}\boldsymbol{\theta}$: what happens to the samples?

LR: same paths, **new weights**



$$w_i = \frac{p_{\boldsymbol{\theta}+\mathrm{d}\boldsymbol{\theta}}(\tau_i)}{p_{\boldsymbol{\theta}}(\tau_i)} \approx 1 + \sum_k \nabla_{\boldsymbol{\theta}} \log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}^i | \mathbf{s}_k^i, \mathbf{a}_k^i)\, \mathrm{d}\boldsymbol{\theta}$$

Stochastic trajectories: sampling the tree



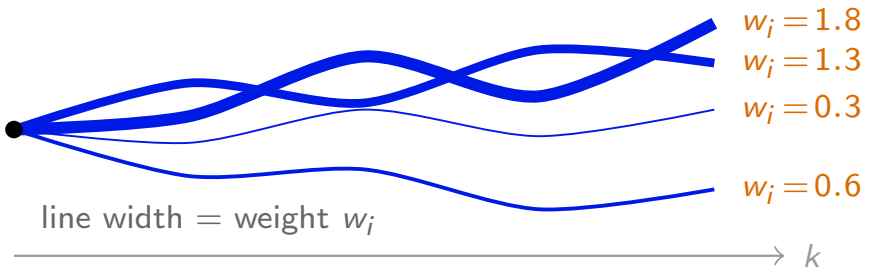
The tree has $(|\mathcal{A}||\mathcal{S}|)^N$ paths: we never enumerate it, we **sample** it.

n simulated closed-loop trajectories from s_0 to s_N = n sampled root-to-leaf paths of the tree

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) \approx \frac{1}{n} \sum_{i=1}^{n} \sum_{k=0}^{N} \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k^i, \mathbf{a}_k^i)$$

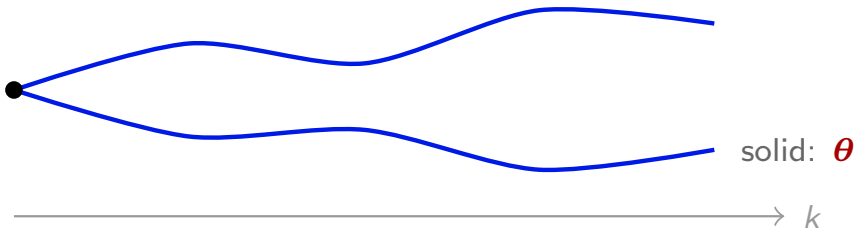
Perturb $\boldsymbol{\theta} \to \boldsymbol{\theta} + \mathrm{d}\boldsymbol{\theta}$: what happens to the samples?

LR: same paths, **new weights**

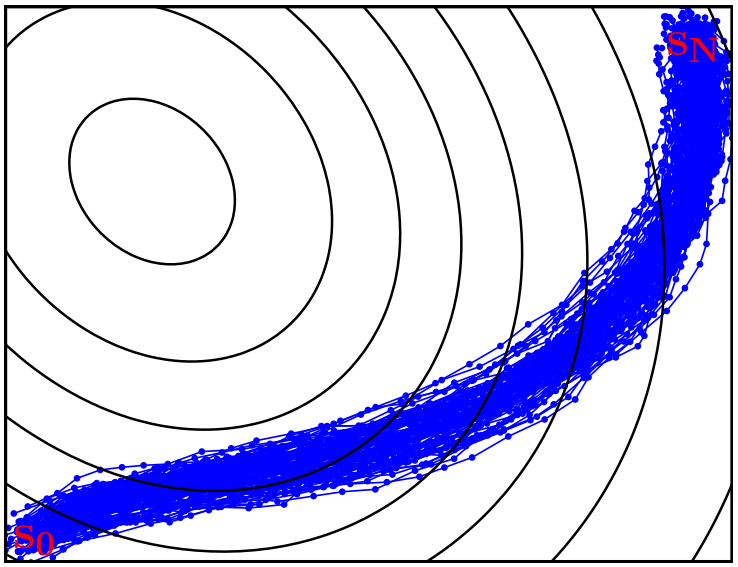


$$w_i = \frac{p_{\boldsymbol{\theta}+\mathrm{d}\boldsymbol{\theta}}(\tau_i)}{p_{\boldsymbol{\theta}}(\tau_i)} \approx 1 + \sum_k \nabla_{\boldsymbol{\theta}} \log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}^i | \mathbf{s}_k^i, \mathbf{a}_k^i)\, \mathrm{d}\boldsymbol{\theta}$$

PD: same noise ω_i, **moved paths**



Stochastic trajectories: sampling the tree



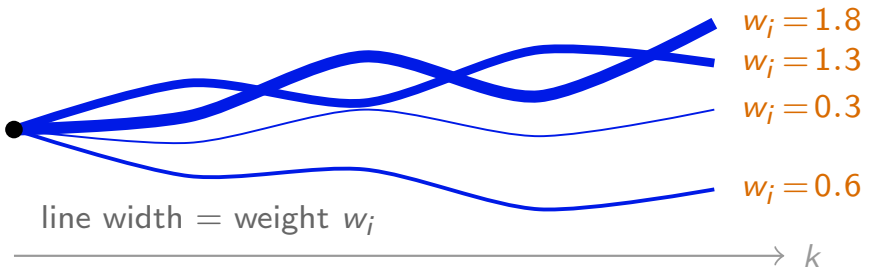
The tree has $(|\mathcal{A}||\mathcal{S}|)^N$ paths: we never enumerate it, we **sample** it.

n simulated closed-loop trajectories from s_0 to s_N = n sampled root-to-leaf paths of the tree

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) \approx \frac{1}{n} \sum_{i=1}^{n} \sum_{k=0}^{N} \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k^i, \mathbf{a}_k^i)$$

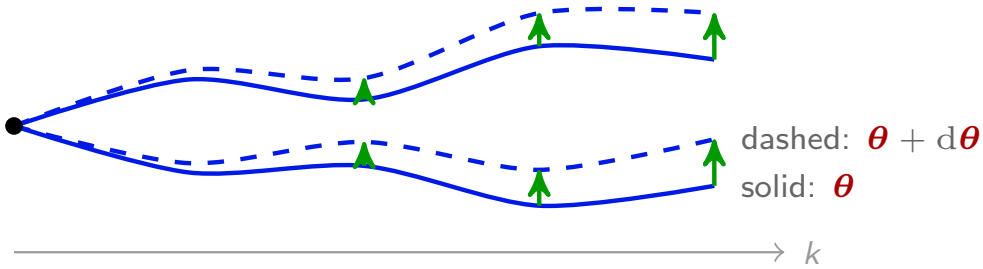
Perturb $\boldsymbol{\theta} \to \boldsymbol{\theta} + \mathrm{d}\boldsymbol{\theta}$: what happens to the samples?

LR: same paths, **new weights**



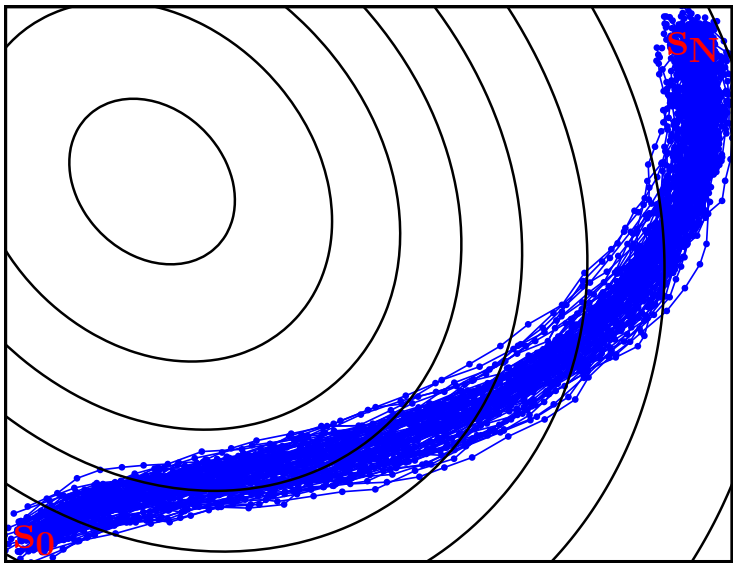
$$w_i = \frac{p_{\boldsymbol{\theta}+\mathrm{d}\boldsymbol{\theta}}(\tau_i)}{p_{\boldsymbol{\theta}}(\tau_i)} \approx 1 + \sum_k \nabla_{\boldsymbol{\theta}} \log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}^i | \mathbf{s}_k^i, \mathbf{a}_k^i)\, \mathrm{d}\boldsymbol{\theta}$$

PD: same noise ω_i, **moved paths**



$\mathbf{s}_k^i(\boldsymbol{\theta} + \mathrm{d}\boldsymbol{\theta}) \approx \mathbf{s}_k^i(\boldsymbol{\theta}) + S_k^i\, \mathrm{d}\boldsymbol{\theta}$, with $S_k = \mathrm{d}\mathbf{s}_k / \mathrm{d}\boldsymbol{\theta}$ (chain rule along the path)

Stochastic trajectories: sampling the tree



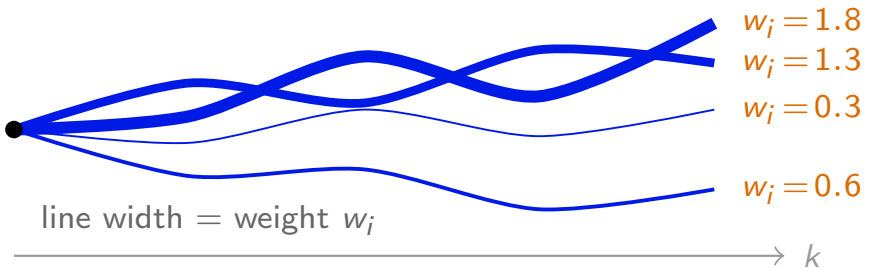
The tree has $(|\mathcal{A}||\mathcal{S}|)^N$ paths: we never enumerate it, we **sample** it.

n simulated closed-loop trajectories from s_0 to s_N = n sampled root-to-leaf paths of the tree

$$\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) \approx \frac{1}{n} \sum_{i=1}^{n} \sum_{k=0}^{N} \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k^i, \mathbf{a}_k^i)$$

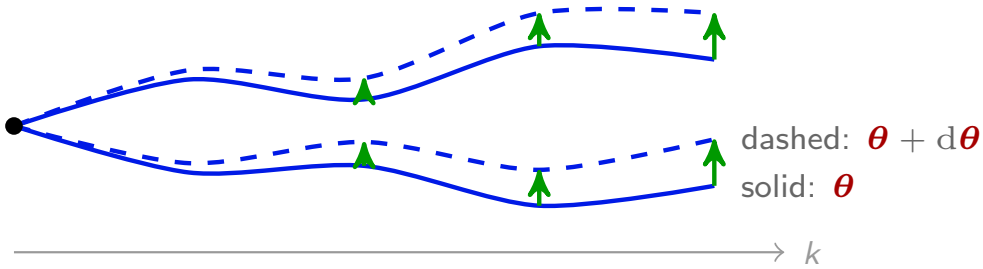
Perturb $\boldsymbol{\theta} \to \boldsymbol{\theta} + \mathrm{d}\boldsymbol{\theta}$: what happens to the samples?

LR: same paths, **new weights**



$$w_i = \frac{p_{\boldsymbol{\theta}+\mathrm{d}\boldsymbol{\theta}}(\tau_i)}{p_{\boldsymbol{\theta}}(\tau_i)} \approx 1 + \sum_k \nabla_{\boldsymbol{\theta}} \log \rho_{\boldsymbol{\theta}}(\mathbf{s}_{k+1}^i | \mathbf{s}_k^i, \mathbf{a}_k^i)\, \mathrm{d}\boldsymbol{\theta}$$

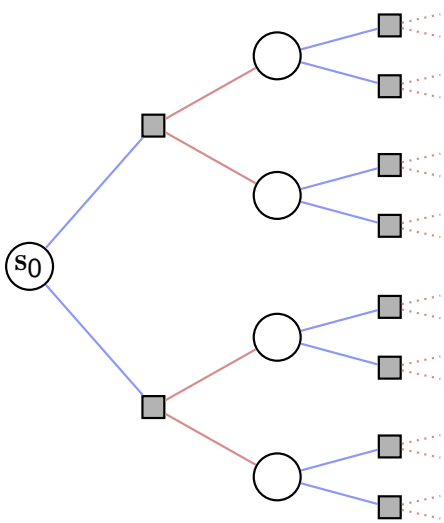
PD: same noise ω_i, **moved paths**



$\mathbf{s}_k^i(\boldsymbol{\theta} + \mathrm{d}\boldsymbol{\theta}) \approx \mathbf{s}_k^i(\boldsymbol{\theta}) + S_k^i\, \mathrm{d}\boldsymbol{\theta}$, with $S_k = \mathrm{d}\mathbf{s}_k/\mathrm{d}\boldsymbol{\theta}$ (chain rule along the path)

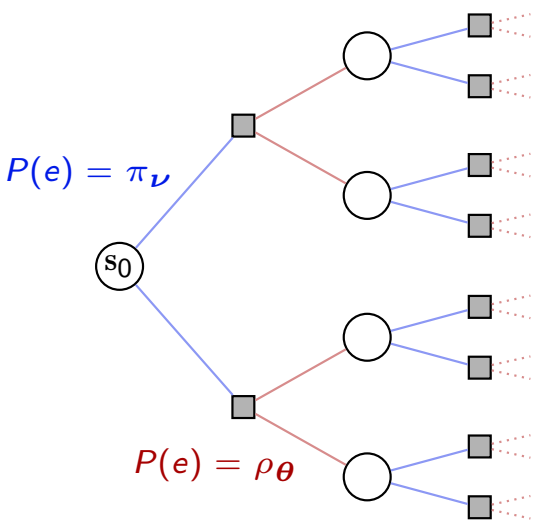
LR changes **how likely** each history is; **PD** changes **where** it goes

Differentiating the tree, LR style: edges pass their score down



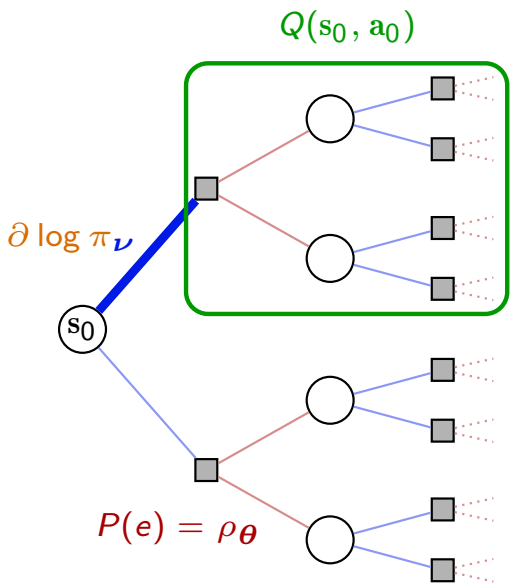
One rule: $\partial P(\tau) = P(\tau)\,\partial \log P(\tau)$,
$\log P(\tau) = \sum_{e \in \tau} \log P(e) \;\Rightarrow$ each edge e contributes
$\partial \log P(e) \;\times$ (discounted cost of the subtree below e)

Differentiating the tree, LR style: edges pass their score down



One rule: $\partial P(\tau) = P(\tau)\, \partial \log P(\tau)$,
$\log P(\tau) = \sum_{e \in \tau} \log P(e) \;\Rightarrow$ each edge e contributes
$\partial \log P(e) \;\times$ (discounted cost of the subtree below e)
$\partial \log P(e)$: relative change of $P(e) = \pi_{\nu}$ or ρ_{θ}

Differentiating the tree, LR style: edges pass their score down



One rule: $\partial P(\tau) = P(\tau)\, \partial \log P(\tau)$,
$\log P(\tau) = \sum_{e \in \tau} \log P(e) \;\Rightarrow$ each edge e contributes

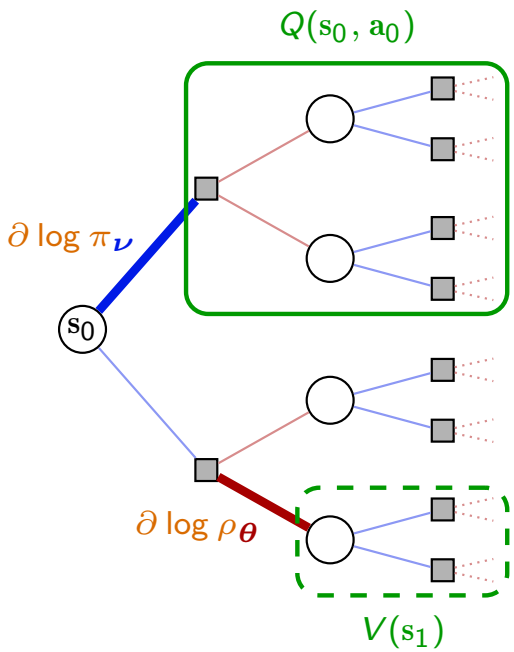
$\partial \log P(e) \;\times$ (discounted cost of the subtree below e)

$\partial \log P(e)$: relative change of $P(e) = \pi_{\boldsymbol{\nu}}$ or $\rho_{\boldsymbol{\theta}}$

Policy edges, $\partial = \nabla_{\boldsymbol{\nu}}$: below $(\mathbf{s}_k, \mathbf{a}_k)$ hangs Q
(*recall*: the **LR** policy gradient)

$$\nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}\Big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)\, Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}(\mathbf{s}_k, \mathbf{a}_k)\Big]$$

Differentiating the tree, LR style: edges pass their score down



One rule: $\partial P(\tau) = P(\tau)\, \partial \log P(\tau)$,
$\log P(\tau) = \sum_{e \in \tau} \log P(e) \;\Rightarrow$ each edge e contributes
$\partial \log P(e) \;\times$ (discounted cost of the subtree below e)

$\partial \log P(e)$: relative change of $P(e) = \pi_\nu$ or ρ_θ

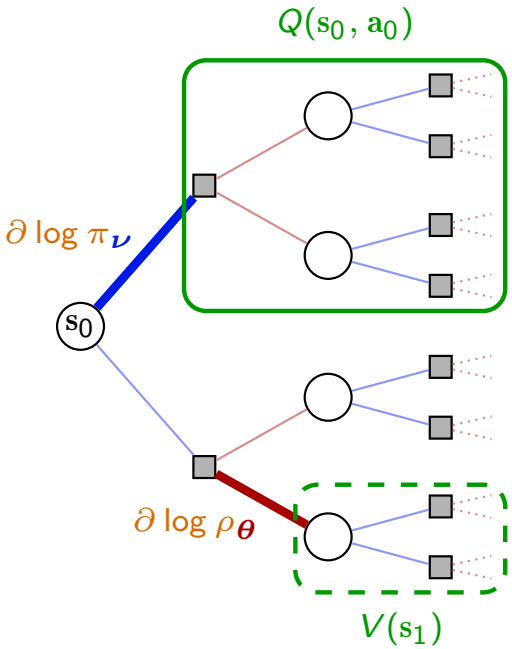
Policy edges, $\partial = \nabla_\nu$: below (s_k, a_k) hangs Q
(*recall*: the **LR** policy gradient)

$$\nabla_\nu \hat{J}_\theta = \mathbb{E}\Big[\sum_k \gamma^k \nabla_\nu \log \pi_\nu(a_k|s_k)\, Q_\theta^{\pi_\nu}(s_k, a_k)\Big]$$

Model edges and costs, $\partial = \nabla_\theta$: below s_{k+1} hangs V (**new**)

$$\nabla_\theta \hat{J}_\theta = \mathbb{E}\Big[\sum_k \gamma^k \Big(\nabla_\theta L_\theta(s_k, a_k) + \gamma \nabla_\theta \log \rho_\theta(s_{k+1}|s_k, a_k)\, V_\theta^{\pi_\nu}(s_{k+1})\Big)\Big]$$

Differentiating the tree, LR style: edges pass their score down



One rule: $\partial P(\tau) = P(\tau)\, \partial \log P(\tau)$,
$\log P(\tau) = \sum_{e \in \tau} \log P(e) \;\Rightarrow$ each edge e contributes
$\partial \log P(e) \;\times$ (discounted cost of the subtree below e)

$\partial \log P(e)$: relative change of $P(e) = \pi_\nu$ or ρ_θ

Policy edges, $\partial = \nabla_\nu$: below (s_k, a_k) hangs Q
(*recall*: the **LR** policy gradient)

$$\nabla_\nu \hat{J}_\theta = \mathbb{E}\Big[\sum_k \gamma^k \nabla_\nu \log \pi_\nu(a_k|s_k)\, Q_\theta^{\pi_\nu}(s_k, a_k)\Big]$$

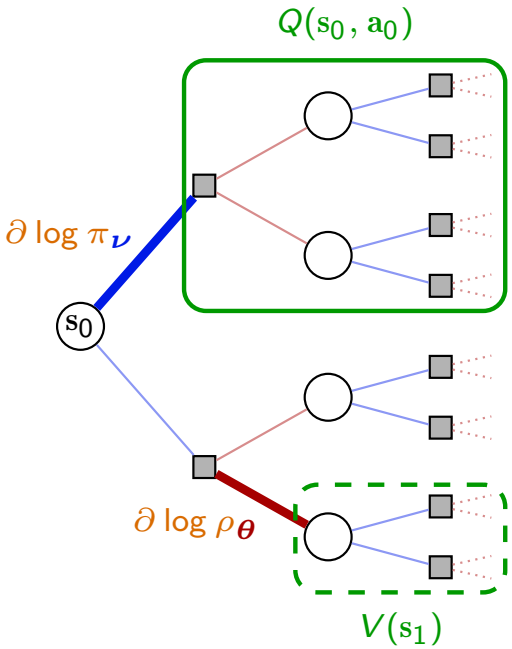
Model edges and costs, $\partial = \nabla_\theta$: below s_{k+1} hangs V (**new**)

$$\nabla_\theta \hat{J}_\theta = \mathbb{E}\Big[\sum_k \gamma^k \Big(\nabla_\theta L_\theta(s_k, a_k) + \gamma \nabla_\theta \log \rho_\theta(s_{k+1}|s_k, a_k)\, V_\theta^{\pi_\nu}(s_{k+1})\Big)\Big]$$

Two tools we reuse later, to differentiate the policy gradient itself:

- **Critic sensitivity**: $Q(s, a)$ is a subtree too, so the same rule gives $\nabla_\theta Q$, $\nabla_\nu Q$

Differentiating the tree, LR style: edges pass their score down



One rule: $\partial P(\tau) = P(\tau)\, \partial \log P(\tau)$,
$\log P(\tau) = \sum_{e \in \tau} \log P(e) \;\Rightarrow$ each edge e contributes
$\partial \log P(e) \;\times$ (discounted cost of the subtree below e)

$\partial \log P(e)$: relative change of $P(e) = \pi_\nu$ or ρ_θ

Policy edges, $\partial = \nabla_\nu$: below (s_k, a_k) hangs Q
(*recall*: the **LR** policy gradient)

$$\nabla_\nu \hat{J}_\theta = \mathbb{E}\Big[\sum_k \gamma^k \nabla_\nu \log \pi_\nu(a_k|s_k)\, Q_\theta^{\pi_\nu}(s_k, a_k)\Big]$$

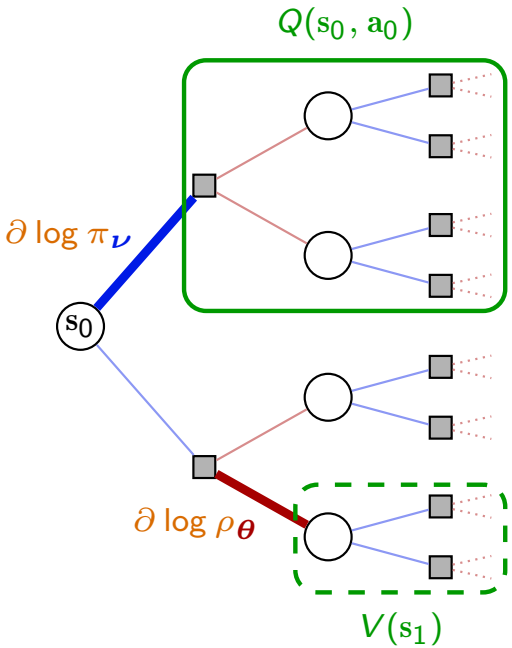
Model edges and costs, $\partial = \nabla_\theta$: below s_{k+1} hangs V (**new**)

$$\nabla_\theta \hat{J}_\theta = \mathbb{E}\Big[\sum_k \gamma^k \Big(\nabla_\theta L_\theta(s_k, a_k) + \gamma \nabla_\theta \log \rho_\theta(s_{k+1}|s_k, a_k)\, V_\theta^{\pi_\nu}(s_{k+1})\Big)\Big]$$

Two tools we reuse later, to differentiate the policy gradient itself:

- **Critic sensitivity**: $Q(s, a)$ is a subtree too, so the same rule gives $\nabla_\theta Q$, $\nabla_\nu Q$
- **Markov-chain sensitivity**: $\partial\, \mathbb{E}[\eta(s_k, a_k)] = \mathbb{E}[\eta\, W_k]$, $W_k = \sum_{e \text{ above } k} \partial \log P(e)$

Differentiating the tree, LR style: edges pass their score down



One rule: $\partial P(\tau) = P(\tau)\, \partial \log P(\tau)$,
$\log P(\tau) = \sum_{e \in \tau} \log P(e) \;\Rightarrow$ each edge e contributes
$\partial \log P(e) \;\times$ (discounted cost of the subtree below e)

$\partial \log P(e)$: relative change of $P(e) = \pi_\nu$ or ρ_θ

Policy edges, $\partial = \nabla_\nu$: below (s_k, a_k) hangs Q
(*recall*: the **LR** policy gradient)

$$\nabla_\nu \hat{J}_\theta = \mathbb{E}\Big[\sum_k \gamma^k \nabla_\nu \log \pi_\nu(a_k|s_k)\, Q_\theta^{\pi_\nu}(s_k, a_k)\Big]$$

Model edges and costs, $\partial = \nabla_\theta$: below s_{k+1} hangs V (**new**)

$$\nabla_\theta \hat{J}_\theta = \mathbb{E}\Big[\sum_k \gamma^k \Big(\nabla_\theta L_\theta(s_k, a_k) + \gamma \nabla_\theta \log \rho_\theta(s_{k+1}|s_k, a_k)\, V_\theta^{\pi_\nu}(s_{k+1})\Big)\Big]$$

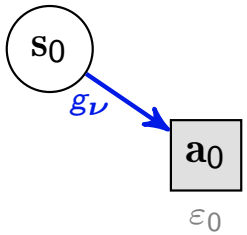
Two tools we reuse later, to differentiate the policy gradient itself:

- **Critic sensitivity**: $Q(s, a)$ is a subtree too, so the same rule gives $\nabla_\theta Q$, $\nabla_\nu Q$
- **Markov-chain sensitivity**: $\partial\, \mathbb{E}[\eta(s_k, a_k)] = \mathbb{E}[\eta\, W_k]$, $W_k = \sum_{e \text{ above } k} \partial \log P(e)$

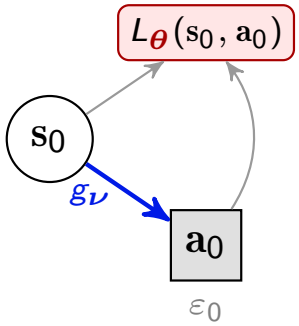
  ✓ no derivatives of L_θ in (s, a): costs may be non-smooth, states and actions discrete

  ✘ ∇_θ needs a simulator *density* ρ_θ ✘ variance $\propto |V|$

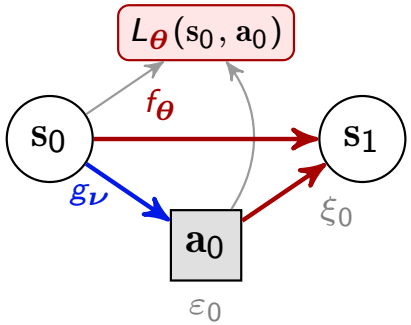
Differentiating PD style: no tree needed, follow each sampled path



Differentiating PD style: no tree needed, follow each sampled path

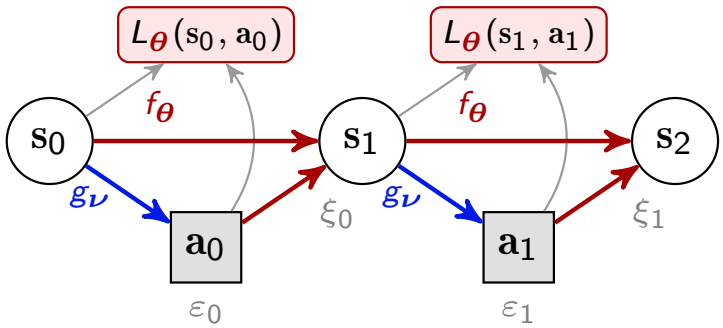


Differentiating PD style: no tree needed, follow each sampled path



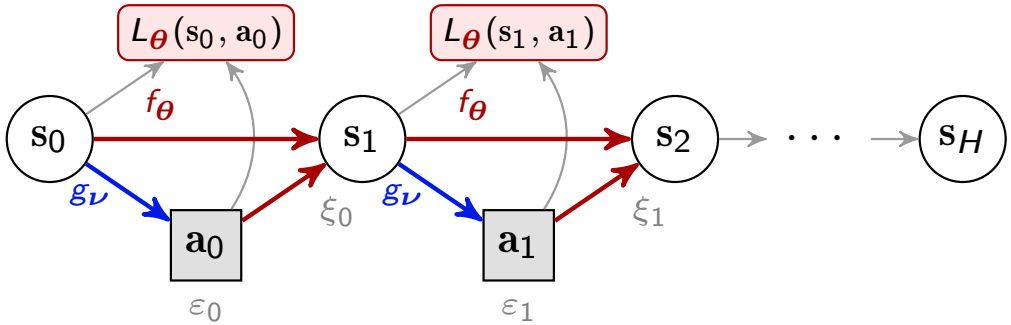
noise ε_k, ξ_k drawn once, then kept fixed: the path is a deterministic chain

Differentiating PD style: no tree needed, follow each sampled path



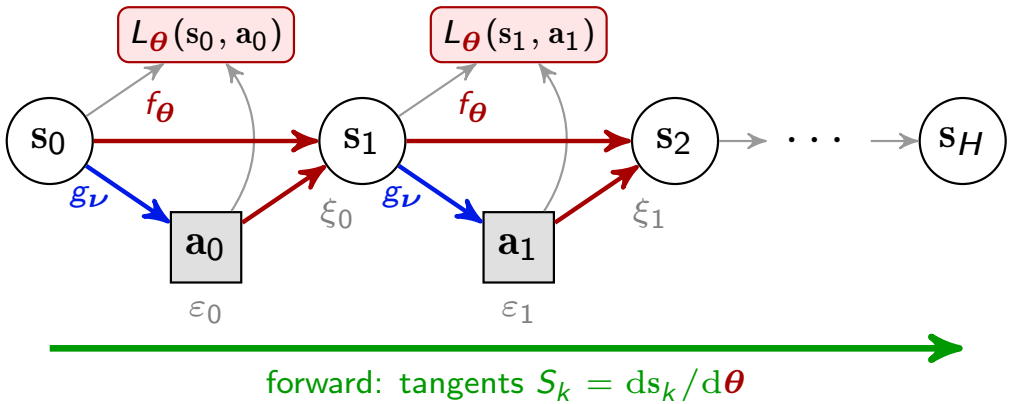
noise ε_k, ξ_k drawn once, then kept fixed: the path is a deterministic chain

Differentiating PD style: no tree needed, follow each sampled path



noise ε_k, ξ_k drawn once, then kept fixed: the path is a deterministic chain

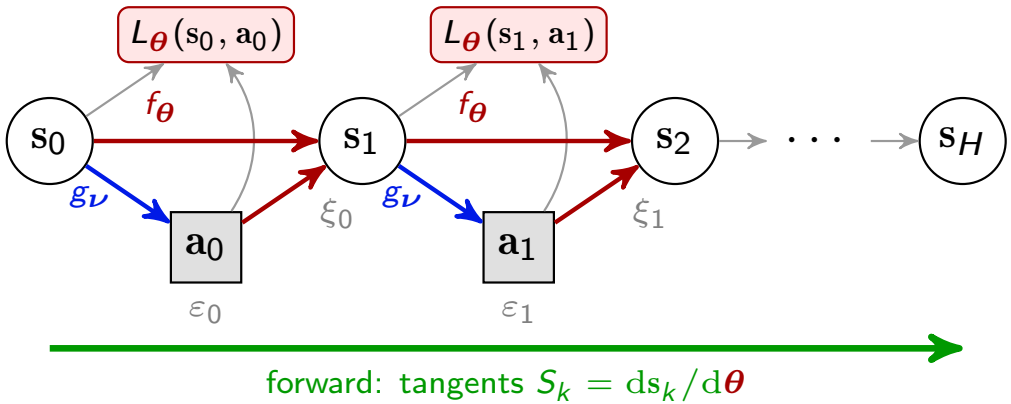
Differentiating PD style: no tree needed, follow each sampled path



Forward sensitivities (one path, noise ω fixed)

$$A_k = \nabla_s g_\nu \, S_k, \qquad S_0 = 0$$

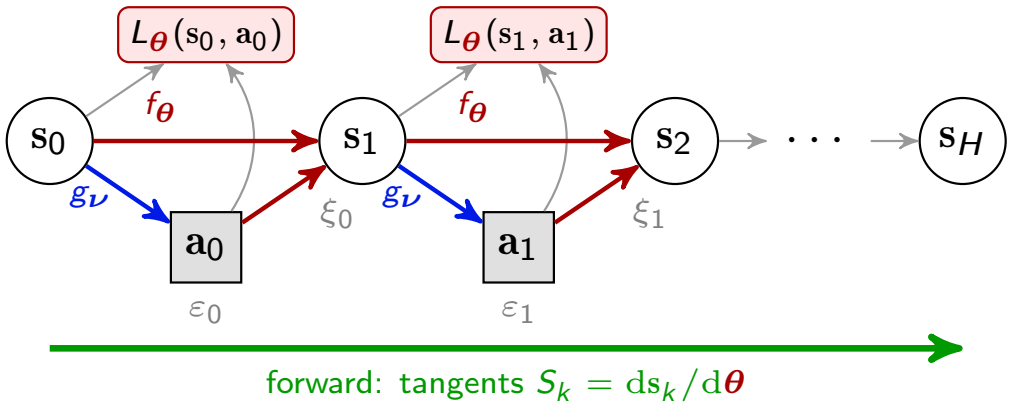
Differentiating PD style: no tree needed, follow each sampled path



Forward sensitivities (one path, noise ω fixed)

$$A_k = \nabla_s g_\nu \, S_k, \qquad S_0 = 0$$
$$S_{k+1} = \partial_\theta f_\theta + \nabla_s f_\theta \, S_k + \nabla_a f_\theta \, A_k$$

Differentiating PD style: no tree needed, follow each sampled path



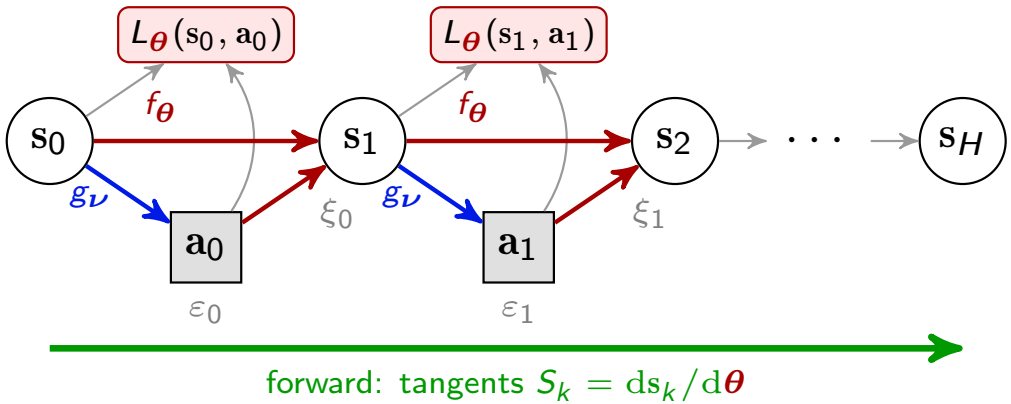
Forward sensitivities (one path, noise ω fixed)

$$A_k = \nabla_s g_\nu \, S_k, \qquad S_0 = 0$$

$$S_{k+1} = \partial_\theta f_\theta + \nabla_s f_\theta \, S_k + \nabla_a f_\theta \, A_k$$

$$\nabla_\theta \hat{J}_\theta = \mathbb{E}_\omega \Big[\sum_k \gamma^k \big(\partial_\theta L_\theta + \nabla_s L_\theta \, S_k + \nabla_a L_\theta \, A_k \big) \Big]$$

Differentiating PD style: no tree needed, follow each sampled path



Forward sensitivities (one path, noise ω fixed)

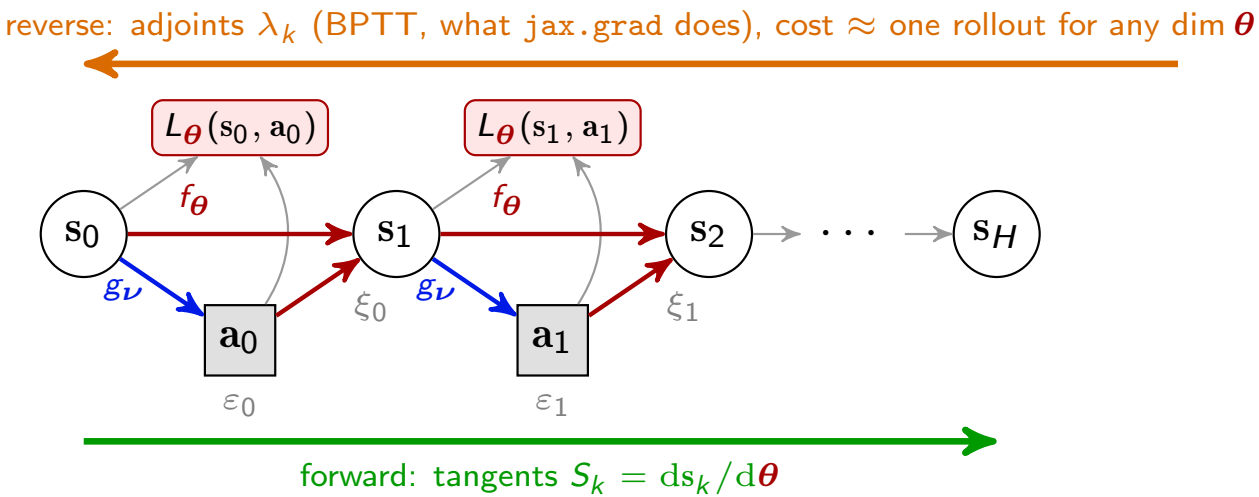
$$A_k = \nabla_s g_\nu \, S_k, \qquad S_0 = 0$$

$$S_{k+1} = \partial_\theta f_\theta + \nabla_s f_\theta \, S_k + \nabla_a f_\theta \, A_k$$

$$\nabla_\theta \hat{J}_\theta = \mathbb{E}_\omega \Big[\sum_k \gamma^k \big(\partial_\theta L_\theta + \nabla_s L_\theta \, S_k + \nabla_a L_\theta \, A_k \big) \Big]$$

For ∇_ν: same recursion with
$A_k = \nabla_s g_\nu S_k + \nabla_\nu g_\nu$, no ∂f term.

Differentiating PD style: no tree needed, follow each sampled path



Forward sensitivities (one path, noise ω fixed)

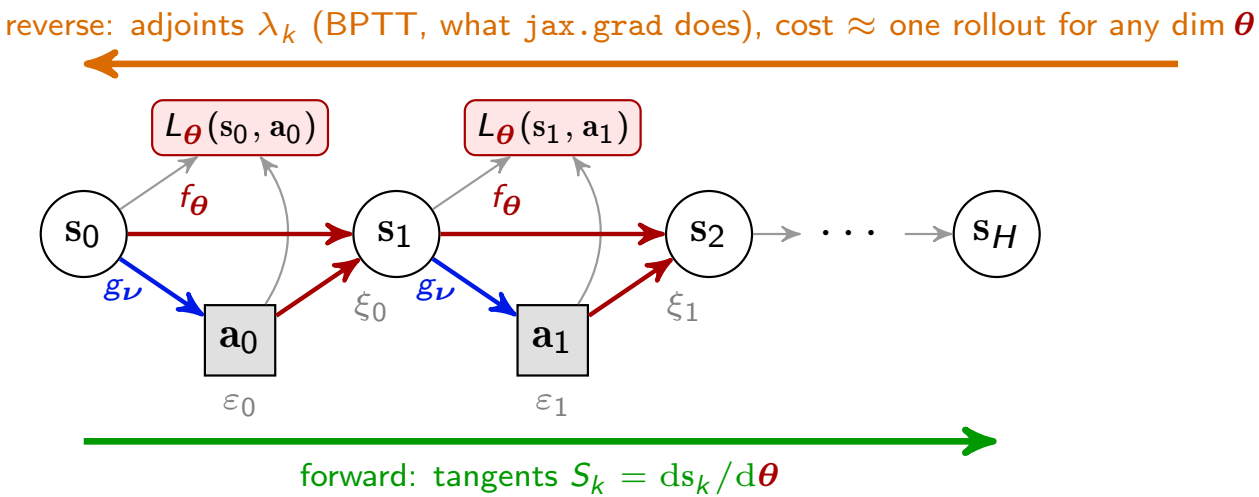
$$A_k = \nabla_{\mathbf{s}} g_{\boldsymbol{\nu}}\, S_k, \qquad S_0 = 0$$

$$S_{k+1} = \partial_{\boldsymbol{\theta}} f_{\boldsymbol{\theta}} + \nabla_{\mathbf{s}} f_{\boldsymbol{\theta}}\, S_k + \nabla_{\mathbf{a}} f_{\boldsymbol{\theta}}\, A_k$$

$$\nabla_{\boldsymbol{\theta}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}_{\omega}\Big[\sum_k \gamma^k \big(\partial_{\boldsymbol{\theta}} L_{\boldsymbol{\theta}} + \nabla_{\mathbf{s}} L_{\boldsymbol{\theta}}\, S_k + \nabla_{\mathbf{a}} L_{\boldsymbol{\theta}}\, A_k\big)\Big]$$

For $\nabla_{\boldsymbol{\nu}}$: same recursion with
$A_k = \nabla_{\mathbf{s}} g_{\boldsymbol{\nu}} S_k + \nabla_{\boldsymbol{\nu}} g_{\boldsymbol{\nu}}$, no ∂f term.

Differentiating PD style: no tree needed, follow each sampled path



Forward sensitivities (one path, noise ω fixed)

$$A_k = \nabla_s g_\nu \, S_k, \qquad S_0 = 0$$

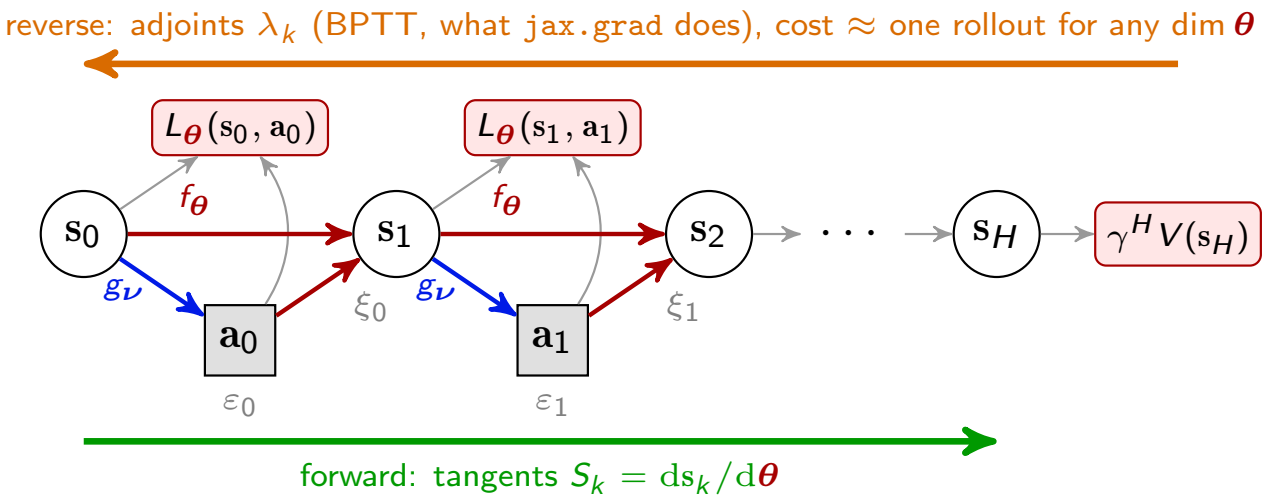
$$S_{k+1} = \partial_\theta f_\theta + \nabla_s f_\theta \, S_k + \nabla_a f_\theta \, A_k$$

$$\nabla_\theta \hat{J}_\theta = \mathbb{E}_\omega \Big[\sum_k \gamma^k \big(\partial_\theta L_\theta + \nabla_s L_\theta \, S_k + \nabla_a L_\theta \, A_k \big) \Big]$$

For ∇_ν: same recursion with
$A_k = \nabla_s g_\nu S_k + \nabla_\nu g_\nu$, no ∂f term.

Catch: S_k contains $\prod_i \nabla_s f_\theta$, which explodes or vanishes for long horizons, contacts, chaos

Differentiating PD style: no tree needed, follow each sampled path



Forward sensitivities (one path, noise ω fixed)

$$A_k = \nabla_{\mathbf{s}} g_{\boldsymbol{\nu}}\, S_k, \qquad S_0 = 0$$

$$S_{k+1} = \partial_{\boldsymbol{\theta}} f_{\boldsymbol{\theta}} + \nabla_{\mathbf{s}} f_{\boldsymbol{\theta}}\, S_k + \nabla_{\mathbf{a}} f_{\boldsymbol{\theta}}\, A_k$$

$$\nabla_{\boldsymbol{\theta}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}_{\omega}\Big[\sum_k \gamma^k \big(\partial_{\boldsymbol{\theta}} L_{\boldsymbol{\theta}} + \nabla_{\mathbf{s}} L_{\boldsymbol{\theta}}\, S_k + \nabla_{\mathbf{a}} L_{\boldsymbol{\theta}}\, A_k\big)\Big]$$

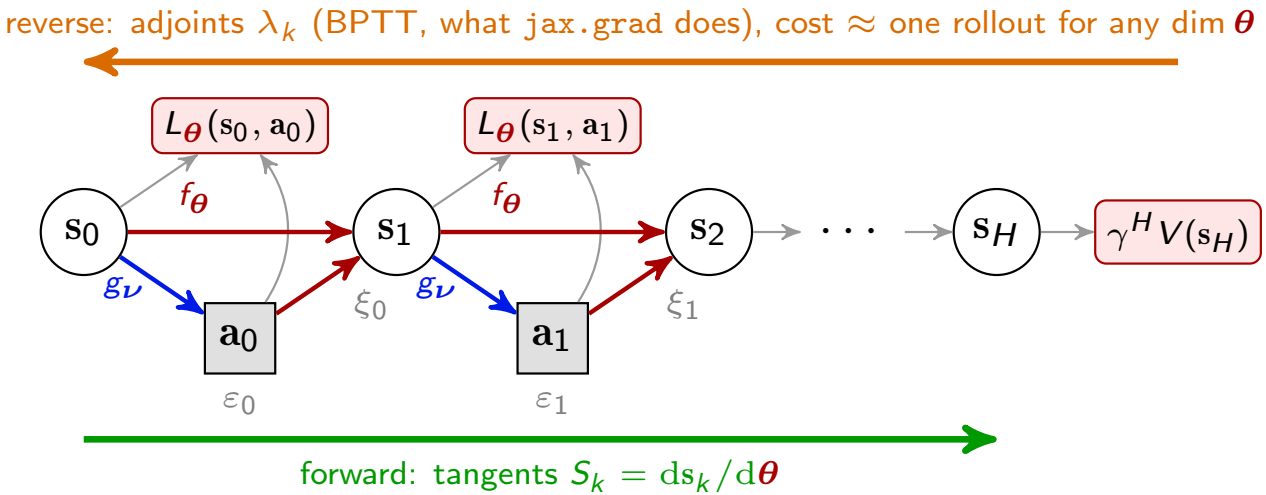
For $\nabla_{\boldsymbol{\nu}}$: same recursion with $A_k = \nabla_{\mathbf{s}} g_{\boldsymbol{\nu}} S_k + \nabla_{\boldsymbol{\nu}} g_{\boldsymbol{\nu}}$, no ∂f term.

Catch: S_k contains $\prod_i \nabla_{\mathbf{s}} f_{\boldsymbol{\theta}}$, which explodes or vanishes for long horizons, contacts, chaos

Fix (SVG, SHAC): differentiate only H steps and bootstrap the tail with a critic

$$\hat{J}_H = \mathbb{E}_{\omega}\big[\textstyle\sum_{k=0}^{H-1} \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k) + \gamma^H V(\mathbf{s}_H)\big]$$

Differentiating PD style: no tree needed, follow each sampled path



Forward sensitivities (one path, noise ω fixed)

$$A_k = \nabla_{\mathbf{s}} g_{\boldsymbol{\nu}}\, S_k, \qquad S_0 = 0$$

$$S_{k+1} = \partial_{\boldsymbol{\theta}} f_{\boldsymbol{\theta}} + \nabla_{\mathbf{s}} f_{\boldsymbol{\theta}}\, S_k + \nabla_{\mathbf{a}} f_{\boldsymbol{\theta}}\, A_k$$

$$\nabla_{\boldsymbol{\theta}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}_\omega \Big[\sum_k \gamma^k \big(\partial_{\boldsymbol{\theta}} L_{\boldsymbol{\theta}} + \nabla_{\mathbf{s}} L_{\boldsymbol{\theta}}\, S_k + \nabla_{\mathbf{a}} L_{\boldsymbol{\theta}}\, A_k \big) \Big]$$

For $\nabla_{\boldsymbol{\nu}}$: same recursion with $A_k = \nabla_{\mathbf{s}} g_{\boldsymbol{\nu}} S_k + \nabla_{\boldsymbol{\nu}} g_{\boldsymbol{\nu}}$, no ∂f term.

Catch: S_k contains $\prod_i \nabla_{\mathbf{s}} f_{\boldsymbol{\theta}}$, which explodes or vanishes for long horizons, contacts, chaos

Fix (SVG, SHAC): differentiate only H steps and bootstrap the tail with a critic

$$\hat{J}_H = \mathbb{E}_\omega \big[\textstyle\sum_{k=0}^{H-1} \gamma^k L_{\boldsymbol{\theta}}(\mathbf{s}_k, \mathbf{a}_k) + \gamma^H V(\mathbf{s}_H) \big]$$

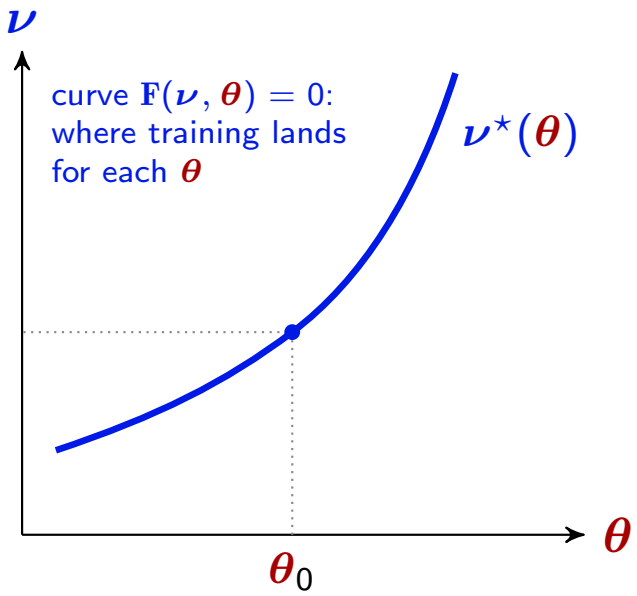
✓ exact per-path gradients; low variance if the dynamics are smooth

✘ needs differentiable $f_{\boldsymbol{\theta}}, g_{\boldsymbol{\nu}}, L_{\boldsymbol{\theta}}$ (smooth surrogates for contact, min, max) and continuous $\mathbf{a}$, $\boldsymbol{\theta}$

Implicit differentiation of policy training

A trained policy is a stationary point of the in-sim objective

$$\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) := \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$



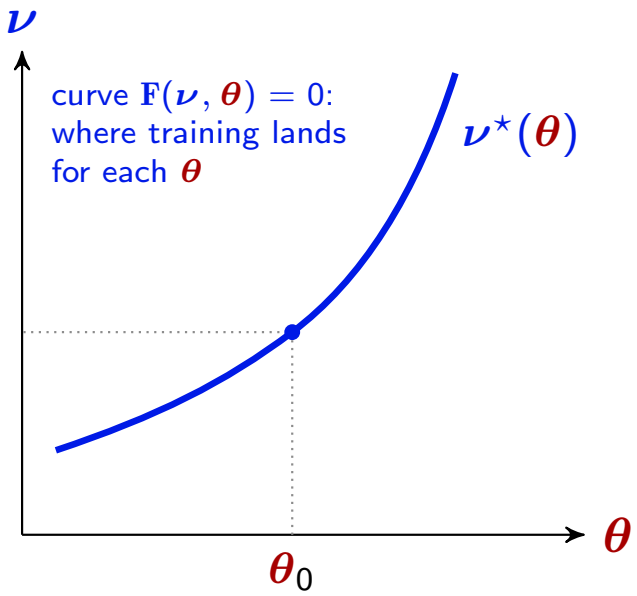
Implicit differentiation of policy training

A trained policy is a stationary point of the in-sim objective

$$\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) := \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

LR: $\mathbb{E}\big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}\, Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}\big] = 0$ (PPO)

PD: $\mathbb{E}_\omega\big[\sum_k \gamma^k (\nabla_{\mathbf{s}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}})\big] = 0$ (SVG)



Implicit differentiation of policy training

A trained policy is a stationary point of the in-sim objective

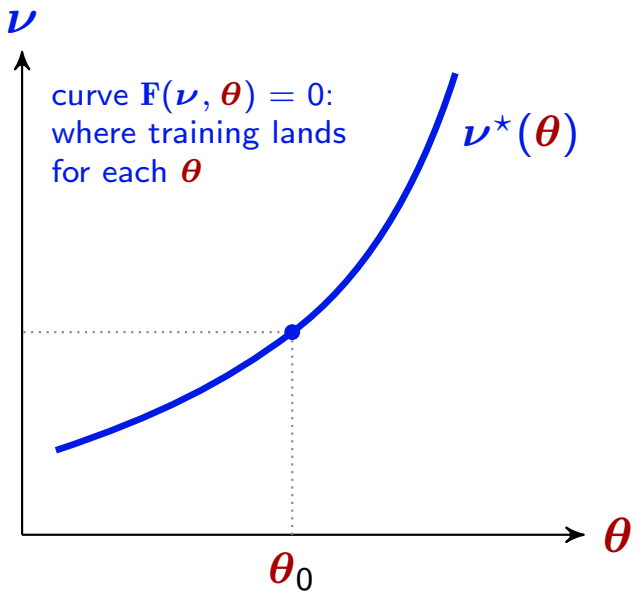
$$\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) := \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

LR: $\mathbb{E}\big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}\, Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}\big] = 0$ (PPO)

PD: $\mathbb{E}_\omega\big[\sum_k \gamma^k (\nabla_{\mathbf{s}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}})\big] = 0$ (SVG)

Assumptions

1. **Smooth**: $\pi_{\boldsymbol{\nu}}, \rho_{\boldsymbol{\theta}}$ (**LR**) or $g_{\boldsymbol{\nu}}, f_{\boldsymbol{\theta}}$ (**PD**), and $L_{\boldsymbol{\theta}}$ are C^2



Implicit differentiation of policy training

A trained policy is a stationary point of the in-sim objective

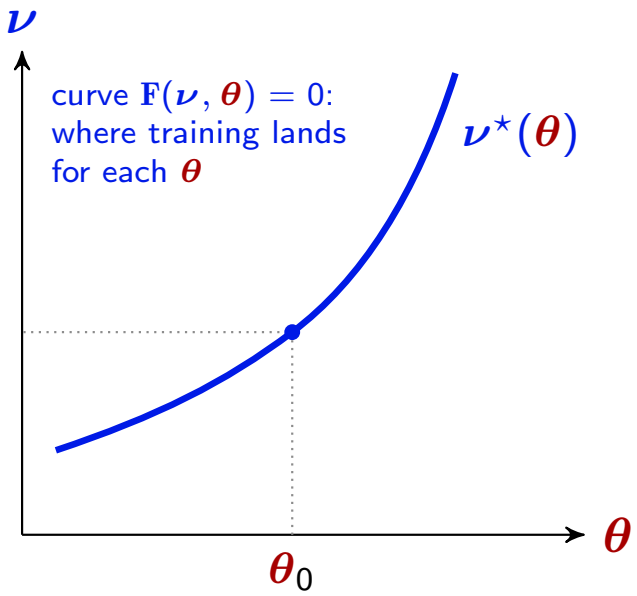
$$\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) := \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

LR: $\mathbb{E}\big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}\, Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}\big] = 0$ (PPO)

PD: $\mathbb{E}_\omega\big[\sum_k \gamma^k (\nabla_{\mathbf{s}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}})\big] = 0$ (SVG)

Assumptions

1. **Smooth**: $\pi_{\boldsymbol{\nu}}, \rho_{\boldsymbol{\theta}}$ (**LR**) or $g_{\boldsymbol{\nu}}, f_{\boldsymbol{\theta}}$ (**PD**), and $L_{\boldsymbol{\theta}}$ are C^2
2. ∇ **and** $\mathbb{E}$ **commute** (**LR**: densities > 0; **PD**: integrable derivatives)



Implicit differentiation of policy training

A trained policy is a stationary point of the in-sim objective

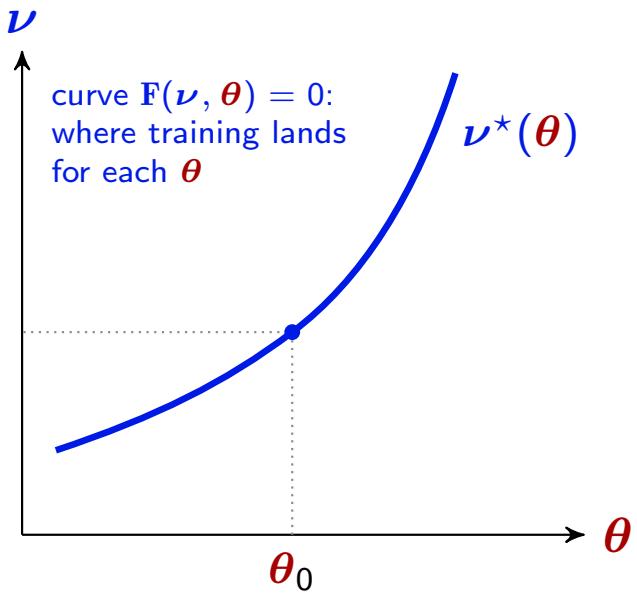
$$\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) := \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

LR: $\mathbb{E}\big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}\, Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}\big] = 0$ (PPO)

PD: $\mathbb{E}_\omega\big[\sum_k \gamma^k (\nabla_{\mathbf{s}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}})\big] = 0$ (SVG)

Assumptions

1. **Smooth**: $\pi_{\boldsymbol{\nu}}, \rho_{\boldsymbol{\theta}}$ (**LR**) or $g_{\boldsymbol{\nu}}, f_{\boldsymbol{\theta}}$ (**PD**), and $L_{\boldsymbol{\theta}}$ are C^2
2. ∇ **and** $\mathbb{E}$ **commute** (**LR**: densities > 0; **PD**: integrable derivatives)
3. **Converged**: $\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) = 0$; a *local* optimum is enough



Implicit differentiation of policy training

A trained policy is a stationary point of the in-sim objective

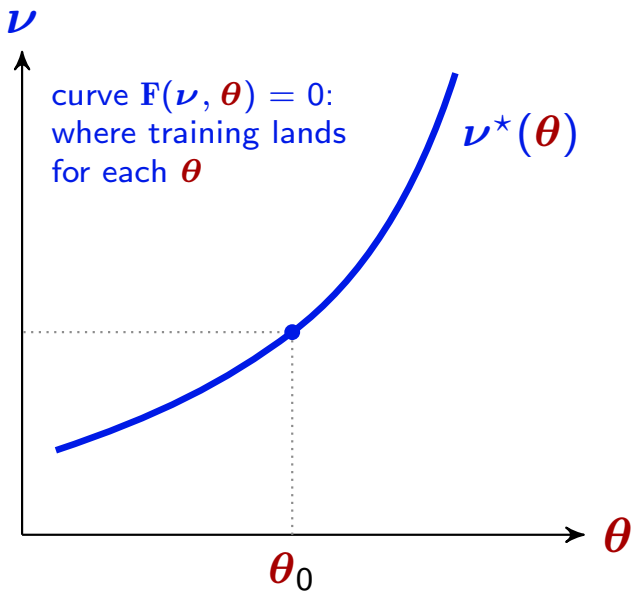
$$\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) := \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

LR: $\mathbb{E}\big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}\, Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}\big] = 0$ (PPO)

PD: $\mathbb{E}_\omega\big[\sum_k \gamma^k \big(\nabla_{\mathbf{s}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}\big)\big] = 0$ (SVG)

Assumptions

1. **Smooth**: $\pi_{\boldsymbol{\nu}}, \rho_{\boldsymbol{\theta}}$ (**LR**) or $g_{\boldsymbol{\nu}}, f_{\boldsymbol{\theta}}$ (**PD**), and $L_{\boldsymbol{\theta}}$ are C^2
2. ∇ **and** $\mathbb{E}$ **commute** (**LR**: densities > 0; **PD**: integrable derivatives)
3. **Converged**: $\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) = 0$; a *local* optimum is enough
4. **Regular**: $\boldsymbol{\nu}^\star$ isolated, $A := \nabla_{\boldsymbol{\nu}} \mathbf{F}$ nonsingular (e.g. $A \succ 0$)



Implicit differentiation of policy training

A trained policy is a stationary point of the in-sim objective

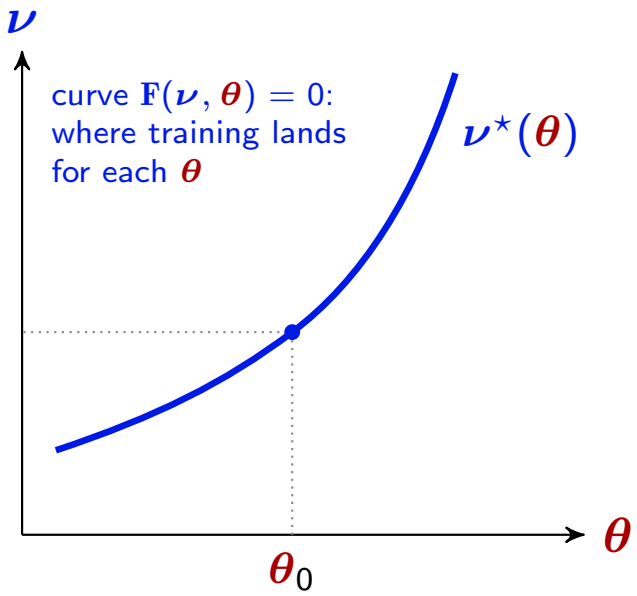
$$\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) := \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

LR: $\mathbb{E}\big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}\, Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}\big] = 0$ (PPO)

PD: $\mathbb{E}_\omega\big[\sum_k \gamma^k \big(\nabla_{\mathbf{s}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}\big)\big] = 0$ (SVG)

Assumptions

1. **Smooth**: $\pi_{\boldsymbol{\nu}}, \rho_{\boldsymbol{\theta}}$ (**LR**) or $g_{\boldsymbol{\nu}}, f_{\boldsymbol{\theta}}$ (**PD**), and $L_{\boldsymbol{\theta}}$ are C^2
2. ∇ **and** $\mathbb{E}$ **commute** (**LR**: densities > 0; **PD**: integrable derivatives)
3. **Converged**: $\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) = 0$; a *local* optimum is enough
4. **Regular**: $\boldsymbol{\nu}^\star$ isolated, $A := \nabla_{\boldsymbol{\nu}} \mathbf{F}$ nonsingular (e.g. $A \succ 0$)
5. **Same spaces**: $\boldsymbol{\theta}$ changes neither $\mathcal{S}$ nor $\mathcal{A}$



Implicit differentiation of policy training

A trained policy is a stationary point of the in-sim objective

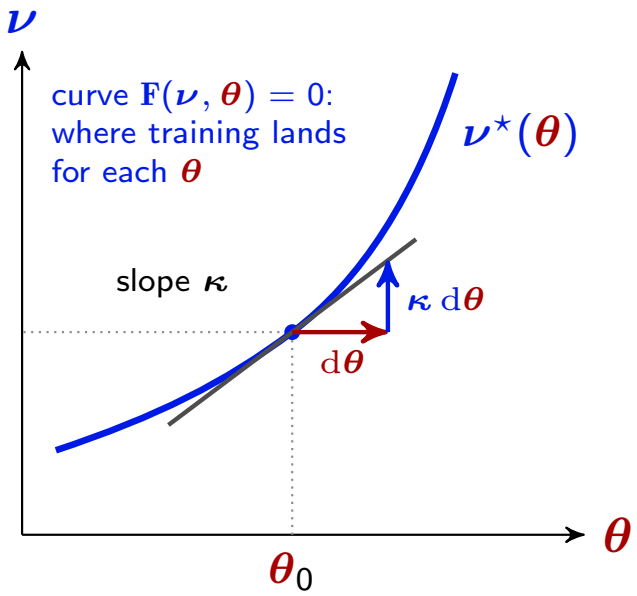
$$\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) := \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

LR: $\mathbb{E}\big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}\, Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}\big] = 0$ (PPO)

PD: $\mathbb{E}_\omega\big[\sum_k \gamma^k \big(\nabla_{\mathbf{s}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}\big)\big] = 0$ (SVG)

Assumptions

1. **Smooth**: $\pi_{\boldsymbol{\nu}}, \rho_{\boldsymbol{\theta}}$ (**LR**) or $g_{\boldsymbol{\nu}}, f_{\boldsymbol{\theta}}$ (**PD**), and $L_{\boldsymbol{\theta}}$ are C^2
2. ∇ **and** $\mathbb{E}$ **commute** (**LR**: densities > 0; **PD**: integrable derivatives)
3. **Converged**: $\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) = 0$; a *local* optimum is enough
4. **Regular**: $\boldsymbol{\nu}^\star$ isolated, $A := \nabla_{\boldsymbol{\nu}} \mathbf{F}$ nonsingular (e.g. $A \succ 0$)
5. **Same spaces**: $\boldsymbol{\theta}$ changes neither $\mathcal{S}$ nor $\mathcal{A}$



IFT: differentiate $\mathbf{F}\big(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\big) \equiv 0 \;\Rightarrow\; A\boldsymbol{\kappa} + B = 0 \;\Rightarrow\; \boxed{\boldsymbol{\kappa} = -A^{-1}B}, \quad B := \nabla_{\boldsymbol{\theta}} \mathbf{F}$

Only the **end point** of training matters: no unrolling, any policy class, any policy-gradient method.

Implicit differentiation of policy training

A trained policy is a stationary point of the in-sim objective

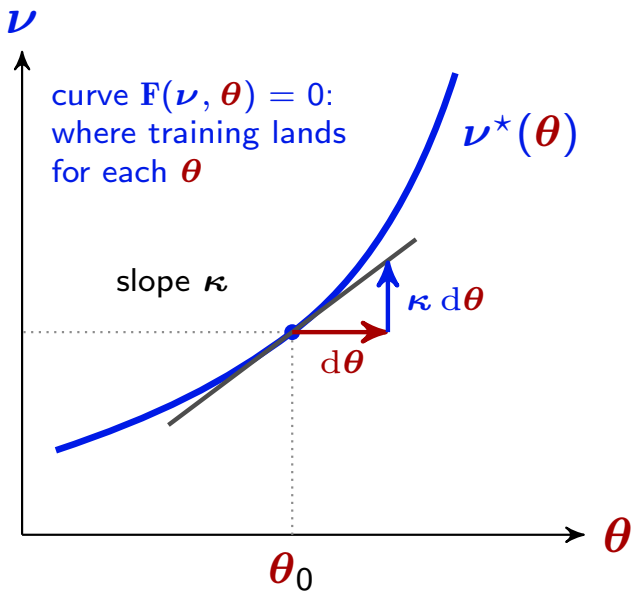
$$\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) := \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

LR: $\mathbb{E}\big[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_{\boldsymbol{\nu}}\, Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}\big] = 0$ (PPO)

PD: $\mathbb{E}_\omega\big[\sum_k \gamma^k \big(\nabla_{\mathbf{s}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{s}_k}{\mathrm{d}\boldsymbol{\nu}} + \nabla_{\mathbf{a}} L_{\boldsymbol{\theta}} \frac{\mathrm{d}\mathbf{a}_k}{\mathrm{d}\boldsymbol{\nu}}\big)\big] = 0$ (SVG)

Assumptions

1. **Smooth**: $\pi_{\boldsymbol{\nu}}, \rho_{\boldsymbol{\theta}}$ (**LR**) or $g_{\boldsymbol{\nu}}, f_{\boldsymbol{\theta}}$ (**PD**), and $L_{\boldsymbol{\theta}}$ are C^2
2. ∇ **and** $\mathbb{E}$ **commute** (**LR**: densities > 0; **PD**: integrable derivatives)
3. **Converged**: $\mathbf{F}(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) = 0$; a *local* optimum is enough
4. **Regular**: $\boldsymbol{\nu}^\star$ isolated, $A := \nabla_{\boldsymbol{\nu}} \mathbf{F}$ nonsingular (e.g. $A \succ 0$)
5. **Same spaces**: $\boldsymbol{\theta}$ changes neither $\mathcal{S}$ nor $\mathcal{A}$



Toy: $\hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}) = \frac{a}{2}\boldsymbol{\nu}^2 - \boldsymbol{\theta}\boldsymbol{\nu}$, so $\boldsymbol{\nu}^\star = \boldsymbol{\theta}/a$
$\mathbf{F} = a\boldsymbol{\nu} - \boldsymbol{\theta}$, $A = a$, $B = -1$
$\boldsymbol{\kappa} = -A^{-1}B = 1/a$ ✓

IFT: differentiate $\mathbf{F}\big(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\big) \equiv 0 \;\Rightarrow\; A\boldsymbol{\kappa} + B = 0 \;\Rightarrow\; \boxed{\boldsymbol{\kappa} = -A^{-1}B}, \quad B := \nabla_{\boldsymbol{\theta}} \mathbf{F}$

Only the **end point** of training matters: no unrolling, any policy class, any policy-gradient method.

Computing A and B for **LR** policies (PPO, A2C)

Recall: $A = \nabla_{\boldsymbol{\nu}} \mathbf{F}$, $B = \nabla_{\boldsymbol{\theta}} \mathbf{F}$, $\mathbf{F} = \mathbb{E}_{\tau \sim p_{\boldsymbol{\nu},\boldsymbol{\theta}}} \left[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_k \, Q_k \right]$: both the integrand ($Q_k$) and the **Markov chain** $p_{\boldsymbol{\nu},\boldsymbol{\theta}}$ depend on $(\boldsymbol{\nu}, \boldsymbol{\theta})$

The parameters act from three places (Anand et al., 2026, Theorem 1)

$\pi_k = \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$, $Q_k = Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}(\mathbf{s}_k, \mathbf{a}_k)$:

$$A = \mathbb{E}\Big[\sum_k \gamma^k \Big(\underbrace{\nabla^2_{\boldsymbol{\nu}\boldsymbol{\nu}} \log \pi_k \, Q_k}_{\text{the node itself}} \Big)\Big]$$

A. S. Anand et al., *Bi-Level Reinforcement Learning Pathway for Sim-to-Real Optimality*, RLC 2026.

Computing A and B for **LR** policies (PPO, A2C)

Recall: $A = \nabla_{\boldsymbol{\nu}} \mathbf{F}$, $B = \nabla_{\boldsymbol{\theta}} \mathbf{F}$, $\mathbf{F} = \mathbb{E}_{\tau \sim p_{\boldsymbol{\nu},\boldsymbol{\theta}}} \left[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_k \, Q_k \right]$: both the integrand ($Q_k$) and the **Markov chain** $p_{\boldsymbol{\nu},\boldsymbol{\theta}}$ depend on $(\boldsymbol{\nu}, \boldsymbol{\theta})$

The parameters act from three places (Anand et al., 2026, Theorem 1)

$\pi_k = \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$, $Q_k = Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}(\mathbf{s}_k, \mathbf{a}_k)$:

$$A = \mathbb{E}\Big[\sum_k \gamma^k \Big(\underbrace{\nabla^2_{\boldsymbol{\nu}\boldsymbol{\nu}} \log \pi_k \, Q_k}_{\text{the node itself}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, (\nabla_{\boldsymbol{\nu}} Q_k)^\top}_{\text{edges below (critic)}} \Big) \Big]$$

A. S. Anand et al., *Bi-Level Reinforcement Learning Pathway for Sim-to-Real Optimality*, RLC 2026.

Computing A and B for **LR** policies (PPO, A2C)

Recall: $A = \nabla_{\boldsymbol{\nu}} \mathbf{F}$, $B = \nabla_{\boldsymbol{\theta}} \mathbf{F}$, $\mathbf{F} = \mathbb{E}_{\tau \sim p_{\boldsymbol{\nu},\boldsymbol{\theta}}} \left[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_k \, Q_k \right]$: both the integrand ($Q_k$) and the **Markov chain** $p_{\boldsymbol{\nu},\boldsymbol{\theta}}$ depend on $(\boldsymbol{\nu}, \boldsymbol{\theta})$

The parameters act from three places (Anand et al., 2026, Theorem 1)

$\pi_k = \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$, $Q_k = Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}(\mathbf{s}_k, \mathbf{a}_k)$:

$$A = \mathbb{E}\Big[\sum_k \gamma^k \Big(\underbrace{\nabla^2_{\boldsymbol{\nu}\boldsymbol{\nu}} \log \pi_k \, Q_k}_{\text{the node itself}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, (\nabla_{\boldsymbol{\nu}} Q_k)^\top}_{\text{edges below (critic)}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, Q_k \, (W_k^{\boldsymbol{\nu}})^\top}_{\text{edges above: the Markov chain}} \Big) \Big]$$

A. S. Anand et al., *Bi-Level Reinforcement Learning Pathway for Sim-to-Real Optimality*, RLC 2026.

Computing A and B for **LR** policies (PPO, A2C)

Recall: $A = \nabla_{\boldsymbol{\nu}} \mathbf{F}$, $B = \nabla_{\boldsymbol{\theta}} \mathbf{F}$, $\mathbf{F} = \mathbb{E}_{\tau \sim p_{\boldsymbol{\nu},\boldsymbol{\theta}}} \left[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_k \, Q_k \right]$: both the integrand ($Q_k$) and the **Markov chain** $p_{\boldsymbol{\nu},\boldsymbol{\theta}}$ depend on $(\boldsymbol{\nu}, \boldsymbol{\theta})$

The parameters act from three places (Anand et al., 2026, Theorem 1)

$\pi_k = \pi_{\boldsymbol{\nu}}(\mathbf{a}_k | \mathbf{s}_k)$, $Q_k = Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}(\mathbf{s}_k, \mathbf{a}_k)$:

$$A = \mathbb{E}\Big[\sum_k \gamma^k \Big(\underbrace{\nabla^2_{\boldsymbol{\nu}\boldsymbol{\nu}} \log \pi_k \, Q_k}_{\text{the node itself}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, (\nabla_{\boldsymbol{\nu}} Q_k)^\top}_{\text{edges below (critic)}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, Q_k \, (W_k^{\boldsymbol{\nu}})^\top}_{\text{edges above: the Markov chain}} \Big) \Big]$$

$$B = \mathbb{E}\Big[\sum_k \gamma^k \Big(\underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, (\nabla_{\boldsymbol{\theta}} Q_k)^\top}_{\text{edges below (critic)}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, Q_k \, (W_k^{\boldsymbol{\theta}})^\top}_{\text{edges above: the Markov chain}} \Big) \Big]$$

A. S. Anand et al., *Bi-Level Reinforcement Learning Pathway for Sim-to-Real Optimality*, RLC 2026.

Computing A and B for **LR** policies (PPO, A2C)

Recall: $A = \nabla_{\boldsymbol{\nu}} \mathbf{F}$, $B = \nabla_{\boldsymbol{\theta}} \mathbf{F}$, $\mathbf{F} = \mathbb{E}_{\tau \sim p_{\boldsymbol{\nu},\boldsymbol{\theta}}} \left[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_k \, Q_k \right]$: both the integrand ($Q_k$) and the **Markov chain** $p_{\boldsymbol{\nu},\boldsymbol{\theta}}$ depend on $(\boldsymbol{\nu}, \boldsymbol{\theta})$

The parameters act from three places (Anand et al., 2026, Theorem 1)

$\pi_k = \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$, $Q_k = Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}(\mathbf{s}_k, \mathbf{a}_k)$:

$$A = \mathbb{E}\Big[\sum_k \gamma^k \Big(\underbrace{\nabla^2_{\boldsymbol{\nu}\boldsymbol{\nu}} \log \pi_k \, Q_k}_{\text{the node itself}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, (\nabla_{\boldsymbol{\nu}} Q_k)^\top}_{\text{edges below (critic)}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, Q_k \, (W_k^{\boldsymbol{\nu}})^\top}_{\text{edges above: the Markov chain}} \Big) \Big]$$

$$B = \mathbb{E}\Big[\sum_k \gamma^k \Big(\underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, (\nabla_{\boldsymbol{\theta}} Q_k)^\top}_{\text{edges below (critic)}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, Q_k \, (W_k^{\boldsymbol{\theta}})^\top}_{\text{edges above: the Markov chain}} \Big) \Big]$$

- **Markov-chain sensitivity** (edges above): $W_k^{\boldsymbol{\nu}} = \sum_{i \le k} \nabla_{\boldsymbol{\nu}} \log \pi_i$, $W_k^{\boldsymbol{\theta}} = \sum_{i \le k} \nabla_{\boldsymbol{\theta}} \log \rho_{\boldsymbol{\theta},i}$

A. S. Anand et al., *Bi-Level Reinforcement Learning Pathway for Sim-to-Real Optimality*, RLC 2026.

Computing A and B for **LR** policies (PPO, A2C)

Recall: $A = \nabla_{\boldsymbol{\nu}} \mathbf{F}$, $B = \nabla_{\boldsymbol{\theta}} \mathbf{F}$, $\mathbf{F} = \mathbb{E}_{\tau \sim p_{\boldsymbol{\nu},\boldsymbol{\theta}}} \left[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_k \, Q_k \right]$: both the integrand ($Q_k$) and the **Markov chain** $p_{\boldsymbol{\nu},\boldsymbol{\theta}}$ depend on $(\boldsymbol{\nu}, \boldsymbol{\theta})$

The parameters act from three places (Anand et al., 2026, Theorem 1)

$\pi_k = \pi_{\boldsymbol{\nu}}(\mathbf{a}_k|\mathbf{s}_k)$, $Q_k = Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}(\mathbf{s}_k, \mathbf{a}_k)$:

$$A = \mathbb{E}\Big[\sum_k \gamma^k \Big(\underbrace{\nabla^2_{\boldsymbol{\nu}\boldsymbol{\nu}} \log \pi_k \, Q_k}_{\text{the node itself}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, (\nabla_{\boldsymbol{\nu}} Q_k)^\top}_{\text{edges below (critic)}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, Q_k \, (W_k^{\boldsymbol{\nu}})^\top}_{\text{edges above: the Markov chain}} \Big) \Big]$$

$$B = \mathbb{E}\Big[\sum_k \gamma^k \Big(\underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, (\nabla_{\boldsymbol{\theta}} Q_k)^\top}_{\text{edges below (critic)}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, Q_k \, (W_k^{\boldsymbol{\theta}})^\top}_{\text{edges above: the Markov chain}} \Big) \Big]$$

- **Markov-chain sensitivity** (edges above): $W_k^{\boldsymbol{\nu}} = \sum_{i \le k} \nabla_{\boldsymbol{\nu}} \log \pi_i$, $W_k^{\boldsymbol{\theta}} = \sum_{i \le k} \nabla_{\boldsymbol{\theta}} \log \rho_{\boldsymbol{\theta},i}$
- **Critic sensitivities** (edges below): $\nabla_{\boldsymbol{\nu}} Q_k$, $\nabla_{\boldsymbol{\theta}} Q_k$, learned like a critic

A. S. Anand et al., *Bi-Level Reinforcement Learning Pathway for Sim-to-Real Optimality*, RLC 2026.

Computing A and B for **LR** policies (PPO, A2C)

Recall: $A = \nabla_{\boldsymbol{\nu}} \mathbf{F}$, $B = \nabla_{\boldsymbol{\theta}} \mathbf{F}$, $\mathbf{F} = \mathbb{E}_{\tau \sim p_{\boldsymbol{\nu},\boldsymbol{\theta}}} \left[\sum_k \gamma^k \nabla_{\boldsymbol{\nu}} \log \pi_k \, Q_k \right]$: both the integrand ($Q_k$) and the **Markov chain** $p_{\boldsymbol{\nu},\boldsymbol{\theta}}$ depend on $(\boldsymbol{\nu}, \boldsymbol{\theta})$

The parameters act from three places (Anand et al., 2026, Theorem 1)

$\pi_k = \pi_{\boldsymbol{\nu}}(\mathbf{a}_k | \mathbf{s}_k)$, $Q_k = Q_{\boldsymbol{\theta}}^{\pi_{\boldsymbol{\nu}}}(\mathbf{s}_k, \mathbf{a}_k)$:

$$A = \mathbb{E}\Big[\sum_k \gamma^k \Big(\underbrace{\nabla^2_{\boldsymbol{\nu}\boldsymbol{\nu}} \log \pi_k \, Q_k}_{\text{the node itself}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, (\nabla_{\boldsymbol{\nu}} Q_k)^\top}_{\text{edges below (critic)}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, Q_k \, (W_k^{\boldsymbol{\nu}})^\top}_{\text{edges above: the Markov chain}} \Big) \Big]$$

$$B = \mathbb{E}\Big[\sum_k \gamma^k \Big(\underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, (\nabla_{\boldsymbol{\theta}} Q_k)^\top}_{\text{edges below (critic)}} + \underbrace{\nabla_{\boldsymbol{\nu}} \log \pi_k \, Q_k \, (W_k^{\boldsymbol{\theta}})^\top}_{\text{edges above: the Markov chain}} \Big) \Big]$$

- **Markov-chain sensitivity** (edges above): $W_k^{\boldsymbol{\nu}} = \sum_{i \le k} \nabla_{\boldsymbol{\nu}} \log \pi_i$, $W_k^{\boldsymbol{\theta}} = \sum_{i \le k} \nabla_{\boldsymbol{\theta}} \log \rho_{\boldsymbol{\theta},i}$
- **Critic sensitivities** (edges below): $\nabla_{\boldsymbol{\nu}} Q_k$, $\nabla_{\boldsymbol{\theta}} Q_k$, learned like a critic
- **Only** B differentiates in $\boldsymbol{\theta}$: through the model edges $\rho_{\boldsymbol{\theta}}$ and the costs $L_{\boldsymbol{\theta}}$ (PPO: $Q \to$ advantage)

A. S. Anand et al., *Bi-Level Reinforcement Learning Pathway for Sim-to-Real Optimality*, RLC 2026.

Computing A and B for **PD** policies (SVG)

Recall: the noise ω has a **fixed** distribution, so the Markov chain changes only by **moving the path** $z_k = (\mathrm{s}_k, \mathbf{a}_k)$, smoothly in $(\boldsymbol{\nu}, \boldsymbol{\theta})$:

$$\mathbf{F} = \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}_{\omega}\Big[\sum_k \gamma^k \nabla_z L_{\boldsymbol{\theta}}(z_k)\, Z_k^{\boldsymbol{\nu}}\Big], \qquad Z_k^{\boldsymbol{\nu}} = \frac{\mathrm{d}z_k}{\mathrm{d}\boldsymbol{\nu}}, \quad Z_k^{\boldsymbol{\theta}} = \frac{\mathrm{d}z_k}{\mathrm{d}\boldsymbol{\theta}} \quad \text{(forward sensitivities)}$$

A. S. Anand et al., *Gradient-Based Co-Design of Reinforcement Learning Policies with Differentiable Simulators*, TMLR 2026

Computing A and B for **PD** policies (SVG)

Recall: the noise ω has a **fixed** distribution, so the Markov chain changes only by **moving the path** $z_k = (\mathrm{s}_k, \mathbf{a}_k)$, smoothly in $(\boldsymbol{\nu}, \boldsymbol{\theta})$:

$$\mathbf{F} = \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}_{\omega}\Big[\sum_k \gamma^k \nabla_z L_{\boldsymbol{\theta}}(z_k)\, Z_k^{\boldsymbol{\nu}}\Big], \qquad Z_k^{\boldsymbol{\nu}} = \frac{\mathrm{d}z_k}{\mathrm{d}\boldsymbol{\nu}}, \quad Z_k^{\boldsymbol{\theta}} = \frac{\mathrm{d}z_k}{\mathrm{d}\boldsymbol{\theta}} \quad \text{(forward sensitivities)}$$

Differentiate once more along the same path (A.S. Anand, 2026)

$$A = \mathbb{E}_{\omega}\Big[\sum_k \gamma^k \Big(\underbrace{(Z_k^{\boldsymbol{\nu}})^\top \nabla_{zz}^2 L_{\boldsymbol{\theta}}\, Z_k^{\boldsymbol{\nu}}}_{\text{curvature of the cost}} + \underbrace{\nabla_z L_{\boldsymbol{\theta}}\, Z_k^{\boldsymbol{\nu}\boldsymbol{\nu}}}_{\text{curvature of the path}} \Big)\Big]$$

A. S. Anand et al., *Gradient-Based Co-Design of Reinforcement Learning Policies with Differentiable Simulators*, TMLR 2026

Computing A and B for **PD** policies (SVG)

Recall: the noise ω has a **fixed** distribution, so the Markov chain changes only by **moving the path** $z_k = (\mathrm{s}_k, \mathrm{a}_k)$, smoothly in $(\boldsymbol{\nu}, \boldsymbol{\theta})$:

$$\mathbf{F} = \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}_\omega \Big[\sum_k \gamma^k \nabla_z L_{\boldsymbol{\theta}}(z_k)\, Z_k^{\boldsymbol{\nu}} \Big], \qquad Z_k^{\boldsymbol{\nu}} = \frac{\mathrm{d}z_k}{\mathrm{d}\boldsymbol{\nu}}, \quad Z_k^{\boldsymbol{\theta}} = \frac{\mathrm{d}z_k}{\mathrm{d}\boldsymbol{\theta}} \quad \text{(forward sensitivities)}$$

Differentiate once more along the same path (A.S. Anand, 2026)

$$A = \mathbb{E}_\omega \Big[\sum_k \gamma^k \Big(\underbrace{(Z_k^{\boldsymbol{\nu}})^\top \nabla_{zz}^2 L_{\boldsymbol{\theta}}\, Z_k^{\boldsymbol{\nu}}}_{\text{curvature of the cost}} + \underbrace{\nabla_z L_{\boldsymbol{\theta}}\, Z_k^{\boldsymbol{\nu}\boldsymbol{\nu}}}_{\text{curvature of the path}} \Big) \Big]$$

$$B = \mathbb{E}_\omega \Big[\sum_k \gamma^k \Big(\underbrace{(Z_k^{\boldsymbol{\nu}})^\top \partial_{\boldsymbol{\theta}} \nabla_z L_{\boldsymbol{\theta}}}_{\boldsymbol{\theta} \text{ in the cost}} \Big) \Big]$$

A. S. Anand et al., *Gradient-Based Co-Design of Reinforcement Learning Policies with Differentiable Simulators*, TMLR 2026

Computing A and B for **PD** policies (SVG)

Recall: the noise ω has a **fixed** distribution, so the Markov chain changes only by **moving the path** $z_k = (\mathrm{s}_k, \mathrm{a}_k)$, smoothly in $(\boldsymbol{\nu}, \boldsymbol{\theta})$:

$$\mathbf{F} = \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}_{\omega}\Big[\sum_k \gamma^k \nabla_z L_{\boldsymbol{\theta}}(z_k)\, Z_k^{\boldsymbol{\nu}}\Big], \qquad Z_k^{\boldsymbol{\nu}} = \frac{\mathrm{d}z_k}{\mathrm{d}\boldsymbol{\nu}}, \quad Z_k^{\boldsymbol{\theta}} = \frac{\mathrm{d}z_k}{\mathrm{d}\boldsymbol{\theta}} \quad \text{(forward sensitivities)}$$

Differentiate once more along the same path (A.S. Anand, 2026)

$$A = \mathbb{E}_{\omega}\Big[\sum_k \gamma^k \Big(\underbrace{(Z_k^{\boldsymbol{\nu}})^\top \nabla_{zz}^2 L_{\boldsymbol{\theta}}\, Z_k^{\boldsymbol{\nu}}}_{\text{curvature of the cost}} + \underbrace{\nabla_z L_{\boldsymbol{\theta}}\, Z_k^{\boldsymbol{\nu}\boldsymbol{\nu}}}_{\text{curvature of the path}} \Big)\Big]$$

$$B = \mathbb{E}_{\omega}\Big[\sum_k \gamma^k \Big(\underbrace{(Z_k^{\boldsymbol{\nu}})^\top \partial_{\boldsymbol{\theta}} \nabla_z L_{\boldsymbol{\theta}}}_{\boldsymbol{\theta} \text{ in the cost}} + \underbrace{(Z_k^{\boldsymbol{\nu}})^\top \nabla_{zz}^2 L_{\boldsymbol{\theta}}\, Z_k^{\boldsymbol{\theta}}}_{\text{the path moves with } \boldsymbol{\theta}} \Big)\Big]$$

A. S. Anand et al., *Gradient-Based Co-Design of Reinforcement Learning Policies with Differentiable Simulators*, TMLR 2026

Computing A and B for **PD** policies (SVG)

Recall: the noise ω has a **fixed** distribution, so the Markov chain changes only by **moving the path** $z_k = (\mathrm{s}_k, \mathrm{a}_k)$, smoothly in $(\boldsymbol{\nu}, \boldsymbol{\theta})$:

$$\mathbf{F} = \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}_\omega \Big[\sum_k \gamma^k \nabla_z L_{\boldsymbol{\theta}}(z_k)\, Z_k^{\boldsymbol{\nu}} \Big], \qquad Z_k^{\boldsymbol{\nu}} = \frac{\mathrm{d}z_k}{\mathrm{d}\boldsymbol{\nu}}, \quad Z_k^{\boldsymbol{\theta}} = \frac{\mathrm{d}z_k}{\mathrm{d}\boldsymbol{\theta}} \quad \text{(forward sensitivities)}$$

Differentiate once more along the same path (A.S. Anand, 2026)

$$A = \mathbb{E}_\omega \Big[\sum_k \gamma^k \Big(\underbrace{(Z_k^{\boldsymbol{\nu}})^\top \nabla_{zz}^2 L_{\boldsymbol{\theta}}\, Z_k^{\boldsymbol{\nu}}}_{\text{curvature of the cost}} + \underbrace{\nabla_z L_{\boldsymbol{\theta}}\, Z_k^{\boldsymbol{\nu}\boldsymbol{\nu}}}_{\text{curvature of the path}} \Big) \Big]$$

$$B = \mathbb{E}_\omega \Big[\sum_k \gamma^k \Big(\underbrace{(Z_k^{\boldsymbol{\nu}})^\top \partial_{\boldsymbol{\theta}} \nabla_z L_{\boldsymbol{\theta}}}_{\boldsymbol{\theta} \text{ in the cost}} + \underbrace{(Z_k^{\boldsymbol{\nu}})^\top \nabla_{zz}^2 L_{\boldsymbol{\theta}}\, Z_k^{\boldsymbol{\theta}}}_{\text{the path moves with } \boldsymbol{\theta}} + \underbrace{\nabla_z L_{\boldsymbol{\theta}}\, Z_k^{\boldsymbol{\nu}\boldsymbol{\theta}}}_{\boldsymbol{\theta} \text{ bends the sensitivities}} \Big) \Big]$$

$Z_k^{\boldsymbol{\nu}\boldsymbol{\nu}}$, $Z_k^{\boldsymbol{\nu}\boldsymbol{\theta}}$: second-order sensitivities; they bring in $\nabla^2 f_{\boldsymbol{\theta}}$, $\nabla^2 g_{\boldsymbol{\nu}}$, $\partial_{\boldsymbol{\theta}} \nabla f_{\boldsymbol{\theta}}$

A. S. Anand et al., *Gradient-Based Co-Design of Reinforcement Learning Policies with Differentiable Simulators*, TMLR 2026

Computing A and B for **PD** policies (SVG)

Recall: the noise ω has a **fixed** distribution, so the Markov chain changes only by **moving the path** $z_k = (\mathrm{s}_k, \mathrm{a}_k)$, smoothly in $(\boldsymbol{\nu}, \boldsymbol{\theta})$:

$$\mathbf{F} = \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}_\omega \Big[\sum_k \gamma^k \nabla_z L_{\boldsymbol{\theta}}(z_k)\, Z_k^{\boldsymbol{\nu}} \Big], \qquad Z_k^{\boldsymbol{\nu}} = \frac{\mathrm{d}z_k}{\mathrm{d}\boldsymbol{\nu}}, \quad Z_k^{\boldsymbol{\theta}} = \frac{\mathrm{d}z_k}{\mathrm{d}\boldsymbol{\theta}} \quad \text{(forward sensitivities)}$$

Differentiate once more along the same path (A.S. Anand, 2026)

$$A = \mathbb{E}_\omega \Big[\sum_k \gamma^k \Big(\underbrace{(Z_k^{\boldsymbol{\nu}})^\top \nabla_{zz}^2 L_{\boldsymbol{\theta}}\, Z_k^{\boldsymbol{\nu}}}_{\text{curvature of the cost}} + \underbrace{\nabla_z L_{\boldsymbol{\theta}}\, Z_k^{\boldsymbol{\nu}\boldsymbol{\nu}}}_{\text{curvature of the path}} \Big) \Big]$$

$$B = \mathbb{E}_\omega \Big[\sum_k \gamma^k \Big(\underbrace{(Z_k^{\boldsymbol{\nu}})^\top \partial_{\boldsymbol{\theta}} \nabla_z L_{\boldsymbol{\theta}}}_{\boldsymbol{\theta} \text{ in the cost}} + \underbrace{(Z_k^{\boldsymbol{\nu}})^\top \nabla_{zz}^2 L_{\boldsymbol{\theta}}\, Z_k^{\boldsymbol{\theta}}}_{\text{the path moves with } \boldsymbol{\theta}} + \underbrace{\nabla_z L_{\boldsymbol{\theta}}\, Z_k^{\boldsymbol{\nu}\boldsymbol{\theta}}}_{\boldsymbol{\theta} \text{ bends the sensitivities}} \Big) \Big]$$

$Z_k^{\boldsymbol{\nu}\boldsymbol{\nu}}$, $Z_k^{\boldsymbol{\nu}\boldsymbol{\theta}}$: second-order sensitivities; they bring in $\nabla^2 f_{\boldsymbol{\theta}}$, $\nabla^2 g_{\boldsymbol{\nu}}$, $\partial_{\boldsymbol{\theta}} \nabla f_{\boldsymbol{\theta}}$

- $A\,v =$ `jvp` of $\mathbf{F}$, $B^\top w =$ `vjp` of $\mathbf{F}$ in $\boldsymbol{\theta}$ (a few rollouts each)

A. S. Anand et al., *Gradient-Based Co-Design of Reinforcement Learning Policies with Differentiable Simulators*, TMLR 2026

Computing A and B for **PD** policies (SVG)

Recall: the noise ω has a **fixed** distribution, so the Markov chain changes only by **moving the path** $z_k = (\mathrm{s}_k, \mathrm{a}_k)$, smoothly in $(\boldsymbol{\nu}, \boldsymbol{\theta})$:

$$\mathbf{F} = \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}} = \mathbb{E}_\omega \Big[\sum_k \gamma^k \nabla_z L_{\boldsymbol{\theta}}(z_k)\, Z_k^{\boldsymbol{\nu}} \Big], \qquad Z_k^{\boldsymbol{\nu}} = \frac{\mathrm{d}z_k}{\mathrm{d}\boldsymbol{\nu}}, \quad Z_k^{\boldsymbol{\theta}} = \frac{\mathrm{d}z_k}{\mathrm{d}\boldsymbol{\theta}} \quad \text{(forward sensitivities)}$$

Differentiate once more along the same path (A.S. Anand, 2026)

$$A = \mathbb{E}_\omega \Big[\sum_k \gamma^k \Big(\underbrace{(Z_k^{\boldsymbol{\nu}})^\top \nabla_{zz}^2 L_{\boldsymbol{\theta}}\, Z_k^{\boldsymbol{\nu}}}_{\text{curvature of the cost}} + \underbrace{\nabla_z L_{\boldsymbol{\theta}}\, Z_k^{\boldsymbol{\nu}\boldsymbol{\nu}}}_{\text{curvature of the path}} \Big) \Big]$$

$$B = \mathbb{E}_\omega \Big[\sum_k \gamma^k \Big(\underbrace{(Z_k^{\boldsymbol{\nu}})^\top \partial_{\boldsymbol{\theta}} \nabla_z L_{\boldsymbol{\theta}}}_{\boldsymbol{\theta} \text{ in the cost}} + \underbrace{(Z_k^{\boldsymbol{\nu}})^\top \nabla_{zz}^2 L_{\boldsymbol{\theta}}\, Z_k^{\boldsymbol{\theta}}}_{\text{the path moves with } \boldsymbol{\theta}} + \underbrace{\nabla_z L_{\boldsymbol{\theta}}\, Z_k^{\boldsymbol{\nu}\boldsymbol{\theta}}}_{\boldsymbol{\theta} \text{ bends the sensitivities}} \Big) \Big]$$

$Z_k^{\boldsymbol{\nu}\boldsymbol{\nu}}$, $Z_k^{\boldsymbol{\nu}\boldsymbol{\theta}}$: second-order sensitivities; they bring in $\nabla^2 f_{\boldsymbol{\theta}}$, $\nabla^2 g_{\boldsymbol{\nu}}$, $\partial_{\boldsymbol{\theta}} \nabla f_{\boldsymbol{\theta}}$

- $A\,v =$ `jvp` of $\mathbf{F}$, $B^\top w =$ `vjp` of $\mathbf{F}$ in $\boldsymbol{\theta}$ (a few rollouts each)
- **In practice** A may be ill-conditioned or indefinite: use $A + \varepsilon I$ or a truncated eigendecomposition

A. S. Anand et al., *Gradient-Based Co-Design of Reinforcement Learning Policies with Differentiable Simulators*, TMLR 2026

Where do we use κ? (1) Sim-to-real: tune the simulator

Motivation: a policy is trained in a simulator, then deployed on a real system.

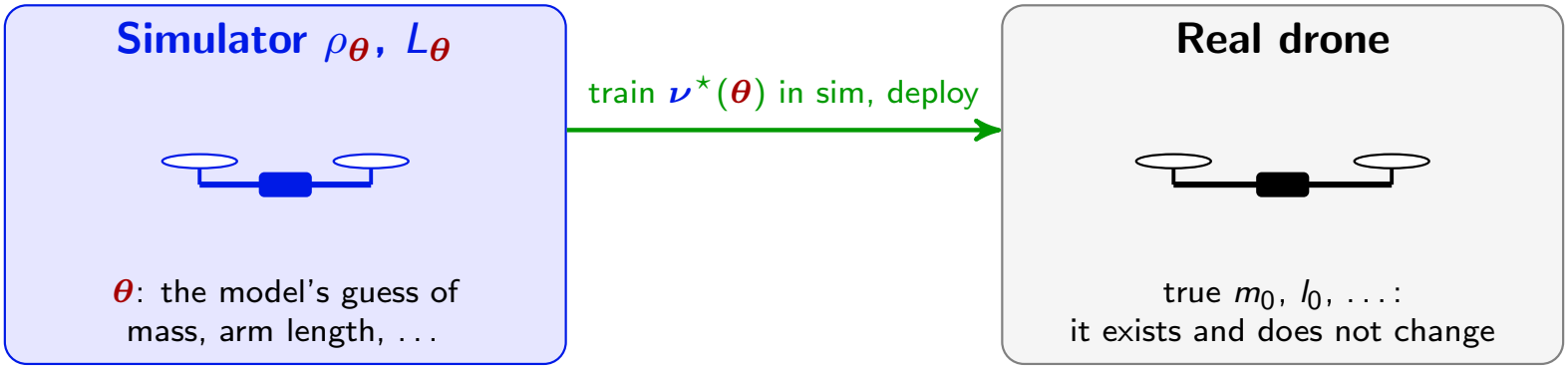
Simulator ρ_{θ}, L_{θ}



θ: the model's guess of mass, arm length, ...

Where do we use κ? (1) Sim-to-real: tune the simulator

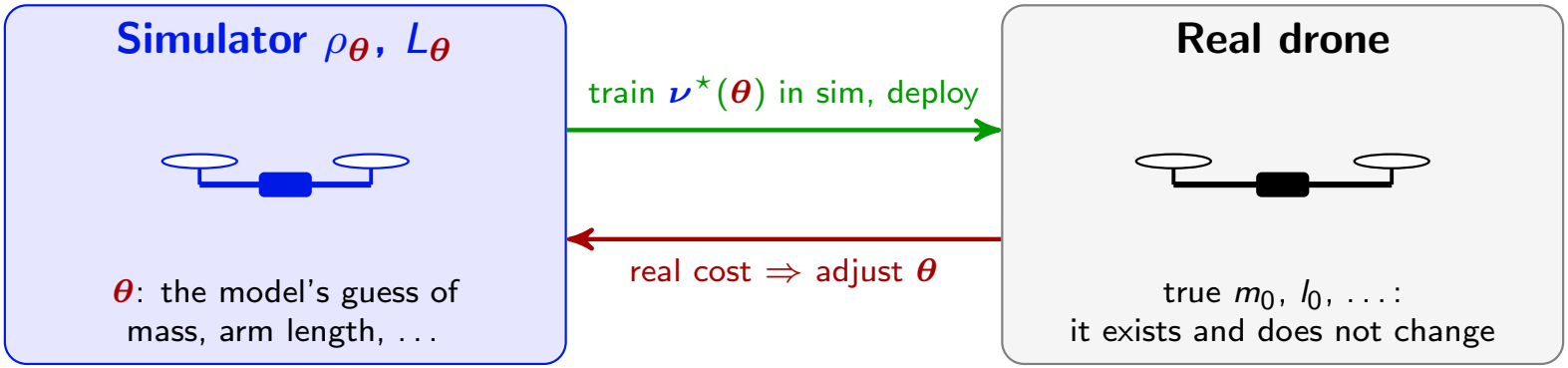
Motivation: a policy is trained in a simulator, then deployed on a real system.



- Changing θ changes the **simulator**, our model of it

Where do we use κ? (1) Sim-to-real: tune the simulator

Motivation: a policy is trained in a simulator, then deployed on a real system.

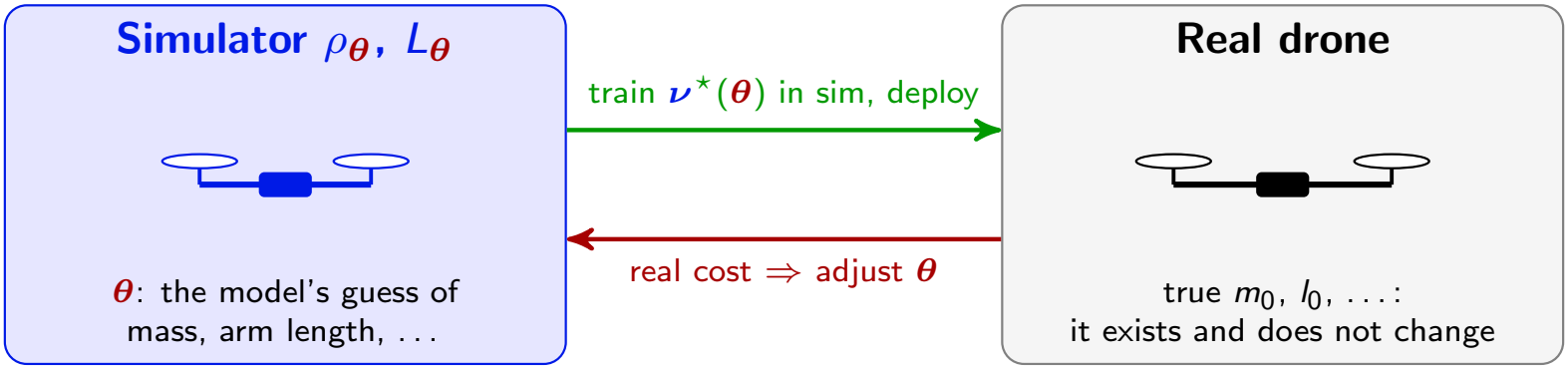


- Changing θ changes the **simulator**, our model of it
- Goal: the policy trained in the simulator should do **best on the real robot**

$$\min_{\theta} \; J_{\text{real}}\big(\nu^{\star}(\theta)\big), \qquad \nu^{\star}(\theta) \text{ trained in the simulator with parameters } \theta$$

Where do we use κ? (1) Sim-to-real: tune the simulator

Motivation: a policy is trained in a simulator, then deployed on a real system.



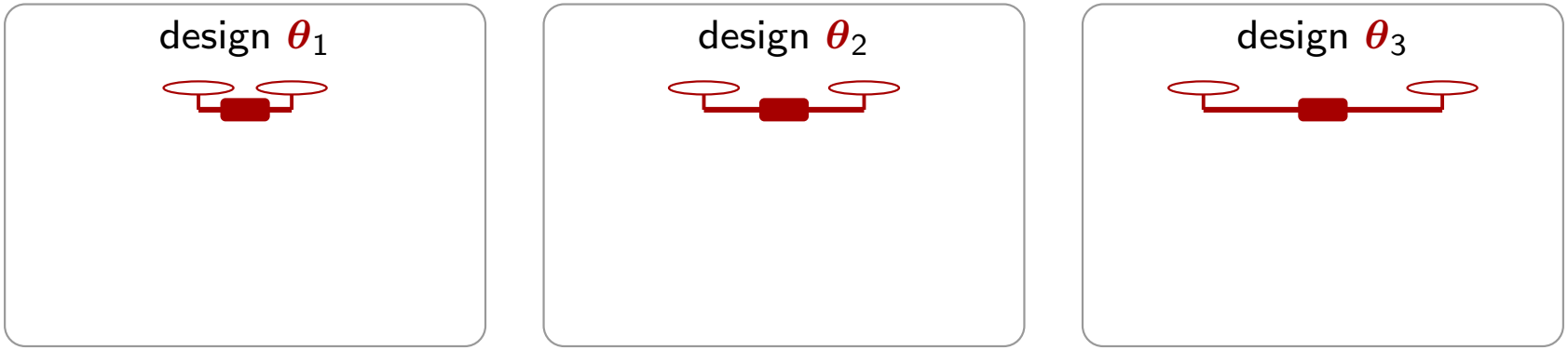
- Changing θ changes the **simulator**, our model of it
- Goal: the policy trained in the simulator should do **best on the real robot**

$$\min_{\theta} \; J_{\text{real}}\big(\nu^\star(\theta)\big), \qquad \nu^\star(\theta) \text{ trained in the simulator with parameters } \theta$$

- Not system identification: the best θ for **decisions** need not be the most accurate model

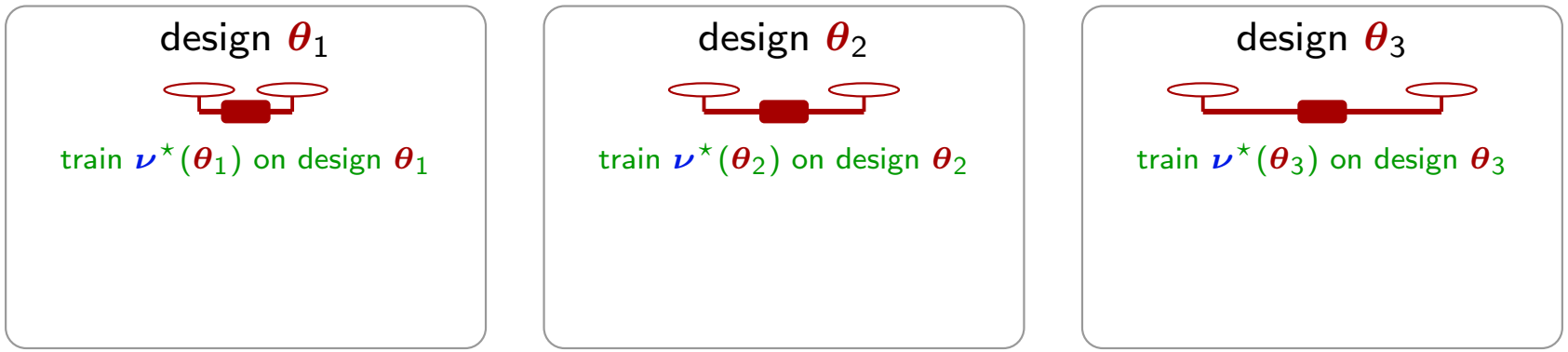
Where do we use κ? (2) Co-design: design the system and its policy

The system is **to be designed**: $\boldsymbol{\theta}$ as the **design space** (arm length, motor size; battery size, ...)



Where do we use κ? (2) Co-design: design the system and its policy

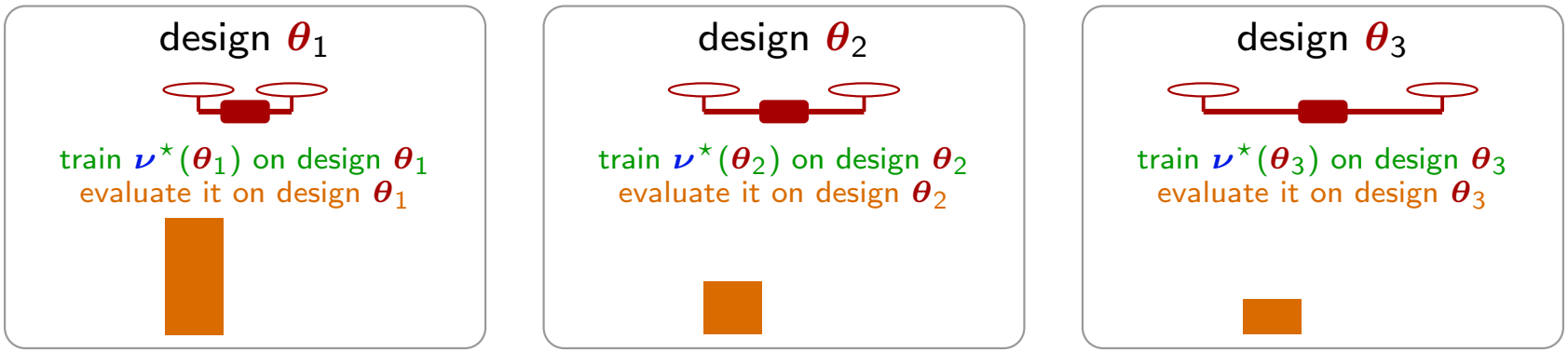
The system is **to be designed**: $\boldsymbol{\theta}$ as the **design space** (arm length, motor size; battery size, ...)



- Changing $\boldsymbol{\theta}$ changes the **real system** itself, hence its MDP: the policy must be **re-trained**, $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$

Where do we use κ? (2) Co-design: design the system and its policy

The system is **to be designed**: $\boldsymbol{\theta}$ as the **design space** (arm length, motor size; battery size, ...)

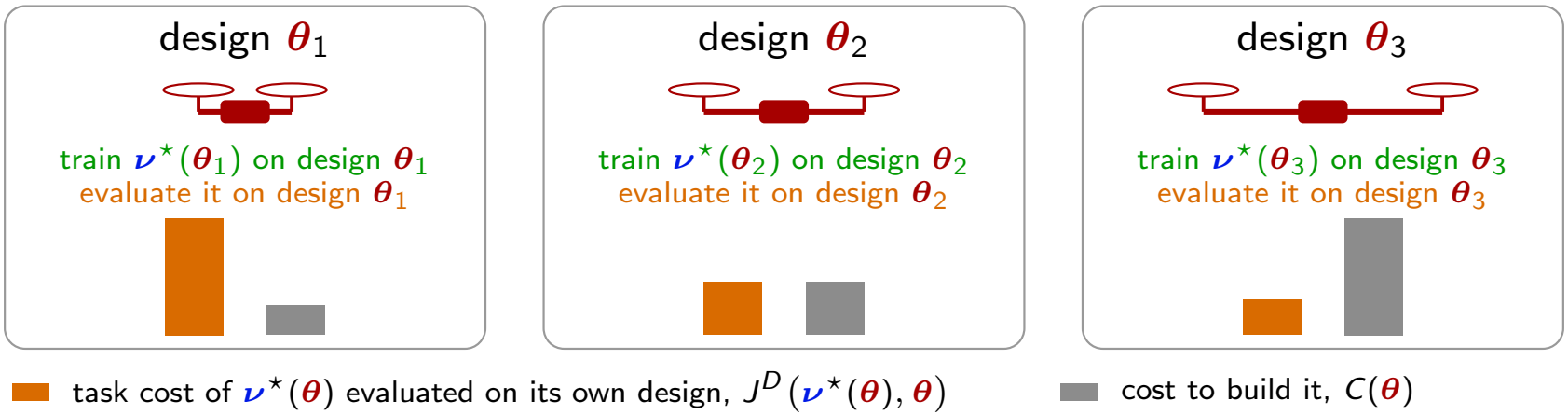


task cost of $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$ evaluated on its own design, $J^D(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta})$

- Changing $\boldsymbol{\theta}$ changes the **real system** itself, hence its MDP: the policy must be **re-trained**, $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$
- Each trained policy is **evaluated on its own design**: the system it is judged on changes with $\boldsymbol{\theta}$ ...

Where do we use κ? (2) Co-design: design the system and its policy

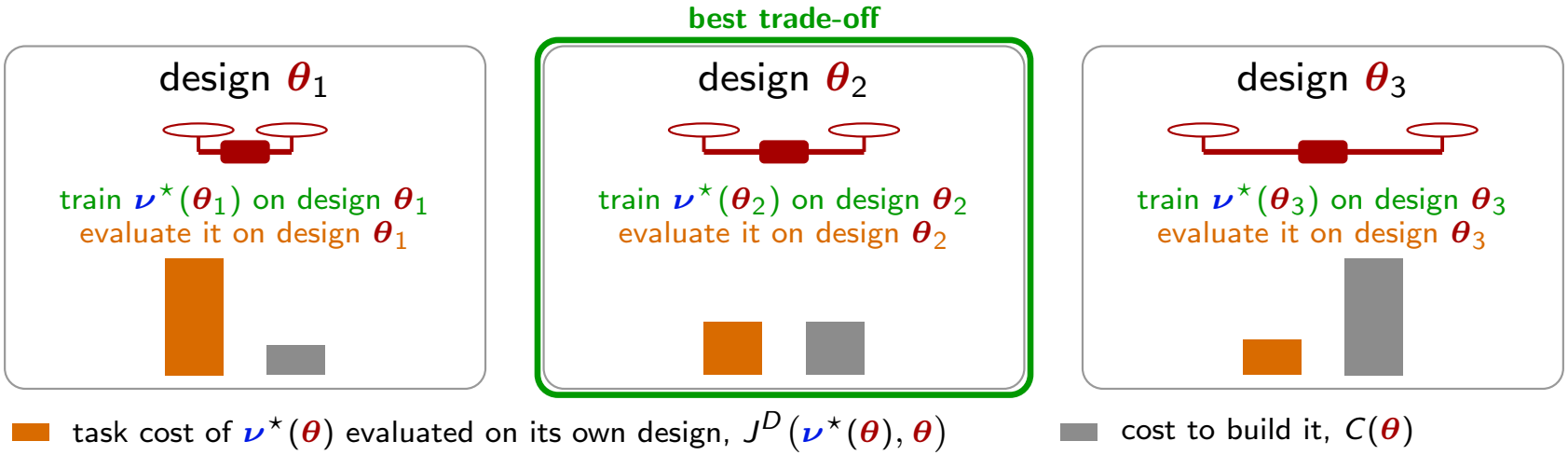
The system is **to be designed**: $\boldsymbol{\theta}$ as the **design space** (arm length, motor size; battery size, ...)



- Changing $\boldsymbol{\theta}$ changes the **real system** itself, hence its MDP: the policy must be **re-trained**, $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$
- Each trained policy is **evaluated on its own design**: the system it is judged on changes with $\boldsymbol{\theta}$ ...
- ... plus what it costs to build: $\min_{\boldsymbol{\theta}} \; J^D\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right) + C(\boldsymbol{\theta})$

Where do we use κ? (2) Co-design: design the system and its policy

The system is **to be designed**: $\boldsymbol{\theta}$ as the **design space** (arm length, motor size; battery size, . . .)

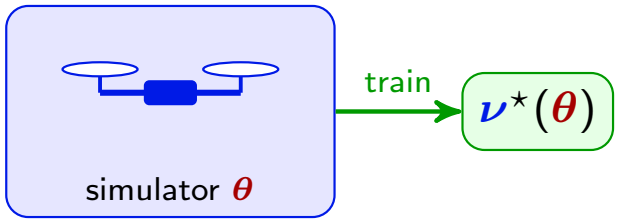


- Changing $\boldsymbol{\theta}$ changes the **real system** itself, hence its MDP: the policy must be **re-trained**, $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$
- Each trained policy is **evaluated on its own design**: the system it is judged on changes with $\boldsymbol{\theta}$ . . .
- . . . plus what it costs to build: $\min_{\boldsymbol{\theta}} \; J^D\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right) + C(\boldsymbol{\theta})$

Sim-to-real: $\boldsymbol{\theta}$ changes only the **model**. Co-design: $\boldsymbol{\theta}$ changes the **robot itself**.

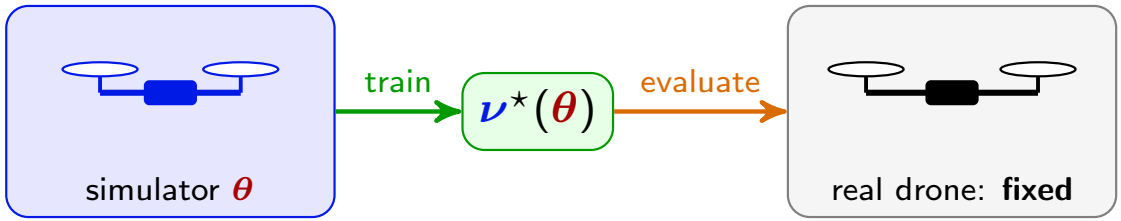
Where is the trained policy evaluated?

Sim-to-real



Where is the trained policy evaluated?

Sim-to-real

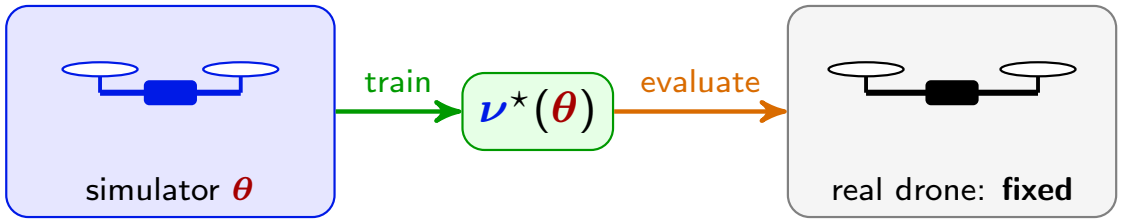


$J_{\text{real}}\left(\nu^\star(\theta)\right)$

the judge does not change

Where is the trained policy evaluated?

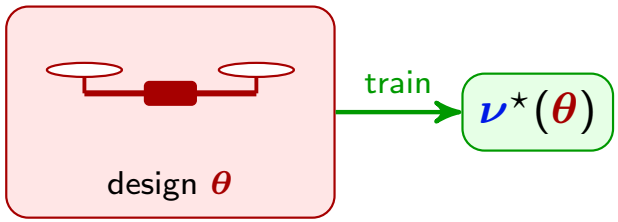
Sim-to-real



$J_{\text{real}}\left(\nu^\star(\theta)\right)$

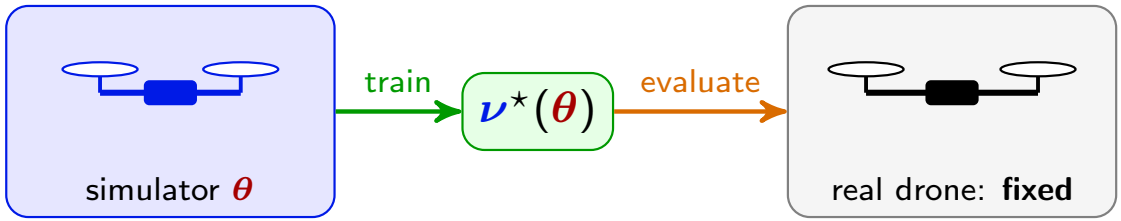
the judge does not change

Co-design



Where is the trained policy evaluated?

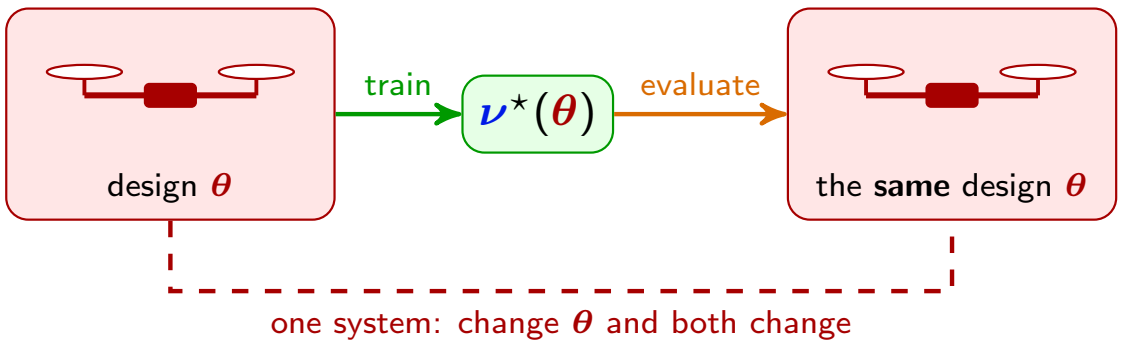
Sim-to-real



$J_{\text{real}}\left(\nu^\star(\theta)\right)$

the judge does not change

Co-design

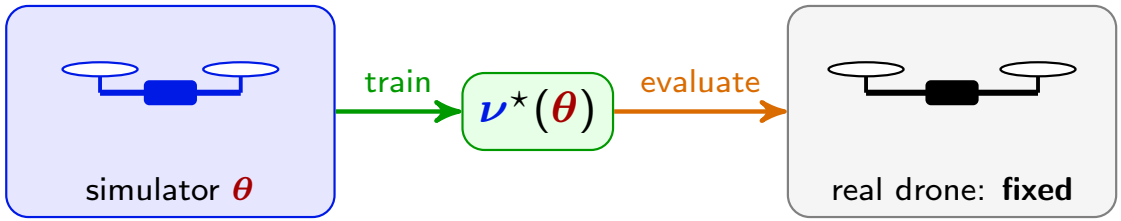


$J^D\left(\nu^\star(\theta), \theta\right)$

the judge changes with θ

Where is the trained policy evaluated?

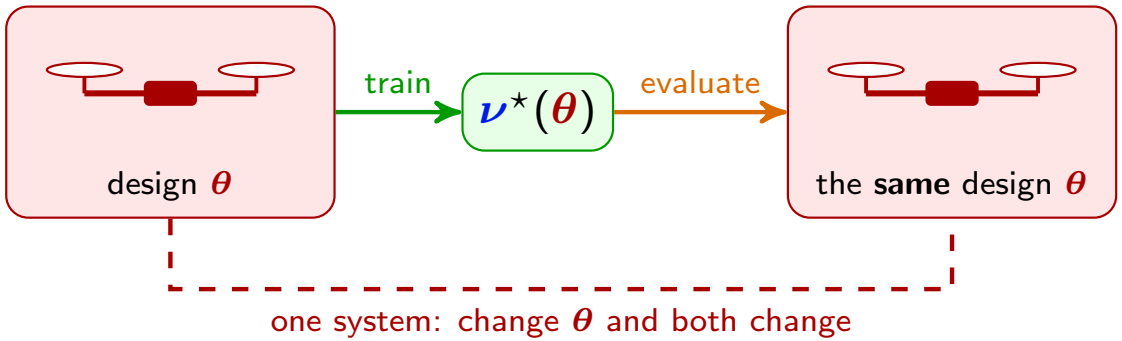
Sim-to-real



$J_{\text{real}}\left(\nu^\star(\theta)\right)$

the judge does not change

Co-design



$J^D\left(\nu^\star(\theta), \theta\right)$

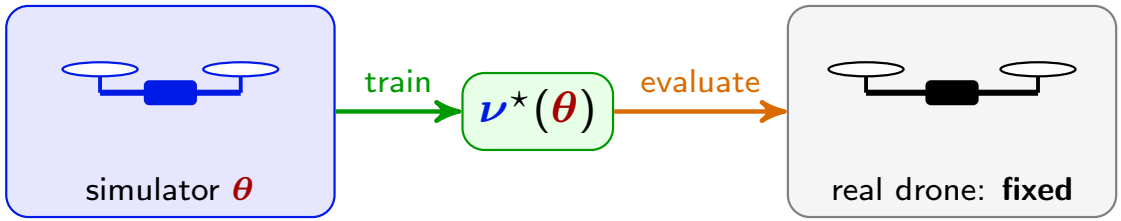
the judge changes with θ

sim-to-real: $J_{\text{real}}\left(\nu^\star(\theta)\right)$ θ acts only through the **policy**

co-design: $J^D\left(\nu^\star(\theta), \theta\right)$ θ acts through the **policy** *and* the **evaluating system**

Where is the trained policy evaluated?

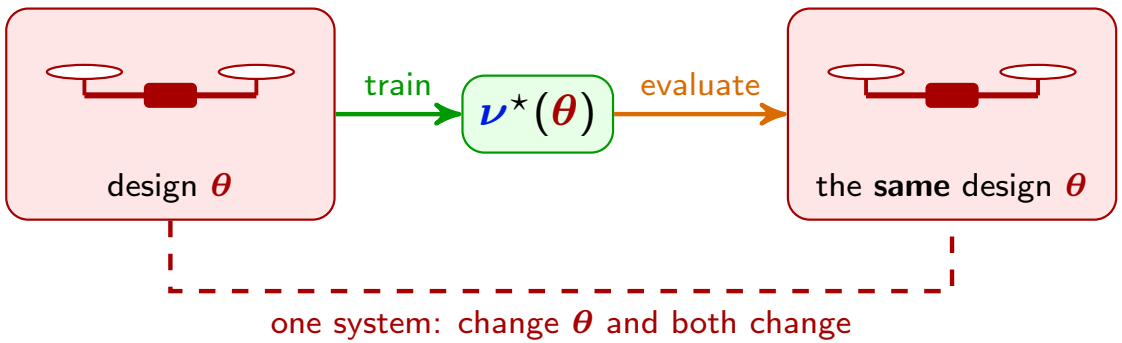
Sim-to-real



$J_{\text{real}}\left(\nu^\star(\theta)\right)$

the judge does not change

Co-design



$J^D\left(\nu^\star(\theta), \theta\right)$

the judge changes with θ

sim-to-real: $J_{\text{real}}\left(\nu^\star(\theta)\right)$ θ acts only through the **policy**

co-design: $J^D\left(\nu^\star(\theta), \theta\right)$ θ acts through the **policy** *and* the **evaluating system**

Co-design: θ acts **twice**, through the re-trained policy *and* the system that evaluates it. So only co-design has a **direct** term $\partial_\theta J^D$ (policy fixed, system changed).

Using κ: the gradient of the outer objective

In both problems we tune θ with the gradient of an **outer objective** $J^{\text{out}}\left(\nu^\star(\theta), \theta\right)$ that **judges** the trained policy. The blocker of slide 13 was $\nabla_\theta \pi_\theta = \nabla_\nu \pi_\nu \, \kappa$: now computable.

$$\nabla_\theta J^{\text{out}}\left(\nu^\star(\theta), \theta\right) = \underbrace{\partial_\theta J^{\text{out}}}_{\text{direct: } \nu \text{ fixed}} + \underbrace{\nabla_\nu J^{\text{out}} \, \kappa}_{\text{indirect: the policy re-adapts}}, \qquad \kappa = -A^{-1} B$$

	Sim-to-real	**Co-design**

Using κ: the gradient of the outer objective

In both problems we tune θ with the gradient of an **outer objective** $J^{\text{out}}\left(\nu^\star(\theta), \theta\right)$ that **judges** the trained policy. The blocker of slide 13 was $\nabla_\theta \pi_\theta = \nabla_\nu \pi_\nu \, \kappa$: now computable.

$$\nabla_\theta J^{\text{out}}\left(\nu^\star(\theta), \theta\right) = \underbrace{\partial_\theta J^{\text{out}}}_{\text{direct: } \nu \text{ fixed}} + \underbrace{\nabla_\nu J^{\text{out}} \, \kappa}_{\text{indirect: the policy re-adapts}}, \qquad \kappa = -A^{-1} B$$

	Sim-to-real	**Co-design**
θ	simulator parameters	design of the system

Using κ: the gradient of the outer objective

In both problems we tune θ with the gradient of an **outer objective** $J^{\text{out}}\left(\nu^\star(\theta), \theta\right)$ that **judges** the trained policy. The blocker of slide 13 was $\nabla_\theta \pi_\theta = \nabla_\nu \pi_\nu \, \kappa$: now computable.

$$\nabla_\theta J^{\text{out}}\left(\nu^\star(\theta), \theta\right) = \underbrace{\partial_\theta J^{\text{out}}}_{\text{direct: } \nu \text{ fixed}} + \underbrace{\nabla_\nu J^{\text{out}} \, \kappa}_{\text{indirect: the policy re-adapts}} , \qquad \kappa = -A^{-1} B$$

	Sim-to-real	**Co-design**
θ	simulator parameters	design of the system
J^{out}	$J_{\text{real}}(\nu^\star)$, on the real robot	$J^D(\nu^\star, \theta) + C(\theta)$, in the simulator

Using $\boldsymbol{\kappa}$: the gradient of the outer objective

In both problems we tune $\boldsymbol{\theta}$ with the gradient of an **outer objective** $J^{\text{out}}\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right)$ that **judges** the trained policy. The blocker of slide 13 was $\nabla_{\boldsymbol{\theta}} \pi_{\boldsymbol{\theta}} = \nabla_{\boldsymbol{\nu}} \pi_{\boldsymbol{\nu}}\, \boldsymbol{\kappa}$: now computable.

$$\nabla_{\boldsymbol{\theta}} J^{\text{out}}\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right) = \underbrace{\partial_{\boldsymbol{\theta}} J^{\text{out}}}_{\text{direct: } \boldsymbol{\nu} \text{ fixed}} + \underbrace{\nabla_{\boldsymbol{\nu}} J^{\text{out}}\, \boldsymbol{\kappa}}_{\text{indirect: the policy re-adapts}}, \qquad \boldsymbol{\kappa} = -A^{-1} B$$

	Sim-to-real	**Co-design**
$\boldsymbol{\theta}$	simulator parameters	design of the system
J^{out}	$J_{\text{real}}(\boldsymbol{\nu}^\star)$, on the real robot	$J^D(\boldsymbol{\nu}^\star, \boldsymbol{\theta}) + C(\boldsymbol{\theta})$, in the simulator
direct term	0 (the real robot ignores $\boldsymbol{\theta}$)	$\neq 0$ ($\boldsymbol{\theta}$ changes the system)

Using κ: the gradient of the outer objective

In both problems we tune θ with the gradient of an **outer objective** $J^{\text{out}}\left(\nu^\star(\theta), \theta\right)$ that **judges** the trained policy. The blocker of slide 13 was $\nabla_\theta \pi_\theta = \nabla_\nu \pi_\nu \, \kappa$: now computable.

$$\nabla_\theta J^{\text{out}}\left(\nu^\star(\theta), \theta\right) = \underbrace{\partial_\theta J^{\text{out}}}_{\text{direct: } \nu \text{ fixed}} + \underbrace{\nabla_\nu J^{\text{out}} \, \kappa}_{\text{indirect: the policy re-adapts}} , \qquad \kappa = -A^{-1} B$$

	Sim-to-real	**Co-design**
θ	simulator parameters	design of the system
J^{out}	$J_{\text{real}}(\nu^\star)$, on the real robot	$J^D(\nu^\star, \theta) + C(\theta)$, in the simulator
direct term	0 (the real robot ignores θ)	$\neq 0$ (θ changes the system)
$\nabla_\nu J^{\text{out}}$	**LR**, from real rollouts	**PD** (AD), in the simulator

Using κ: the gradient of the outer objective

In both problems we tune θ with the gradient of an **outer objective** $J^{\text{out}}\left(\nu^\star(\theta), \theta\right)$ that **judges** the trained policy. The blocker of slide 13 was $\nabla_\theta \pi_\theta = \nabla_\nu \pi_\nu \, \kappa$: now computable.

$$\nabla_\theta J^{\text{out}}\left(\nu^\star(\theta), \theta\right) = \underbrace{\partial_\theta J^{\text{out}}}_{\text{direct: } \nu \text{ fixed}} + \underbrace{\nabla_\nu J^{\text{out}} \, \kappa}_{\text{indirect: the policy re-adapts}}, \qquad \kappa = -A^{-1} B$$

	Sim-to-real	**Co-design**
θ	simulator parameters	design of the system
J^{out}	$J_{\text{real}}(\nu^\star)$, on the real robot	$J^D(\nu^\star, \theta) + C(\theta)$, in the simulator
direct term	0 (the real robot ignores θ)	$\neq 0$ (θ changes the system)
$\nabla_\nu J^{\text{out}}$	**LR**, from real rollouts	**PD** (AD), in the simulator

Adjoint trick (never form κ): with $g = \nabla_\nu J^{\text{out}}$, $g^\top \kappa = -g^\top A^{-1} B$, so

1. solve $A^\top \lambda = g$: one matrix-free linear solve in $\dim \nu$ (CG, MINRES, GMRES)

Using κ: the gradient of the outer objective

In both problems we tune θ with the gradient of an **outer objective** $J^{\text{out}}\big(\nu^\star(\theta), \theta\big)$ that **judges** the trained policy. The blocker of slide 13 was $\nabla_\theta \pi_\theta = \nabla_\nu \pi_\nu \, \kappa$: now computable.

$$\nabla_\theta J^{\text{out}}\big(\nu^\star(\theta), \theta\big) = \underbrace{\partial_\theta J^{\text{out}}}_{\text{direct: } \nu \text{ fixed}} + \underbrace{\nabla_\nu J^{\text{out}} \, \kappa}_{\text{indirect: the policy re-adapts}} , \qquad \kappa = -A^{-1} B$$

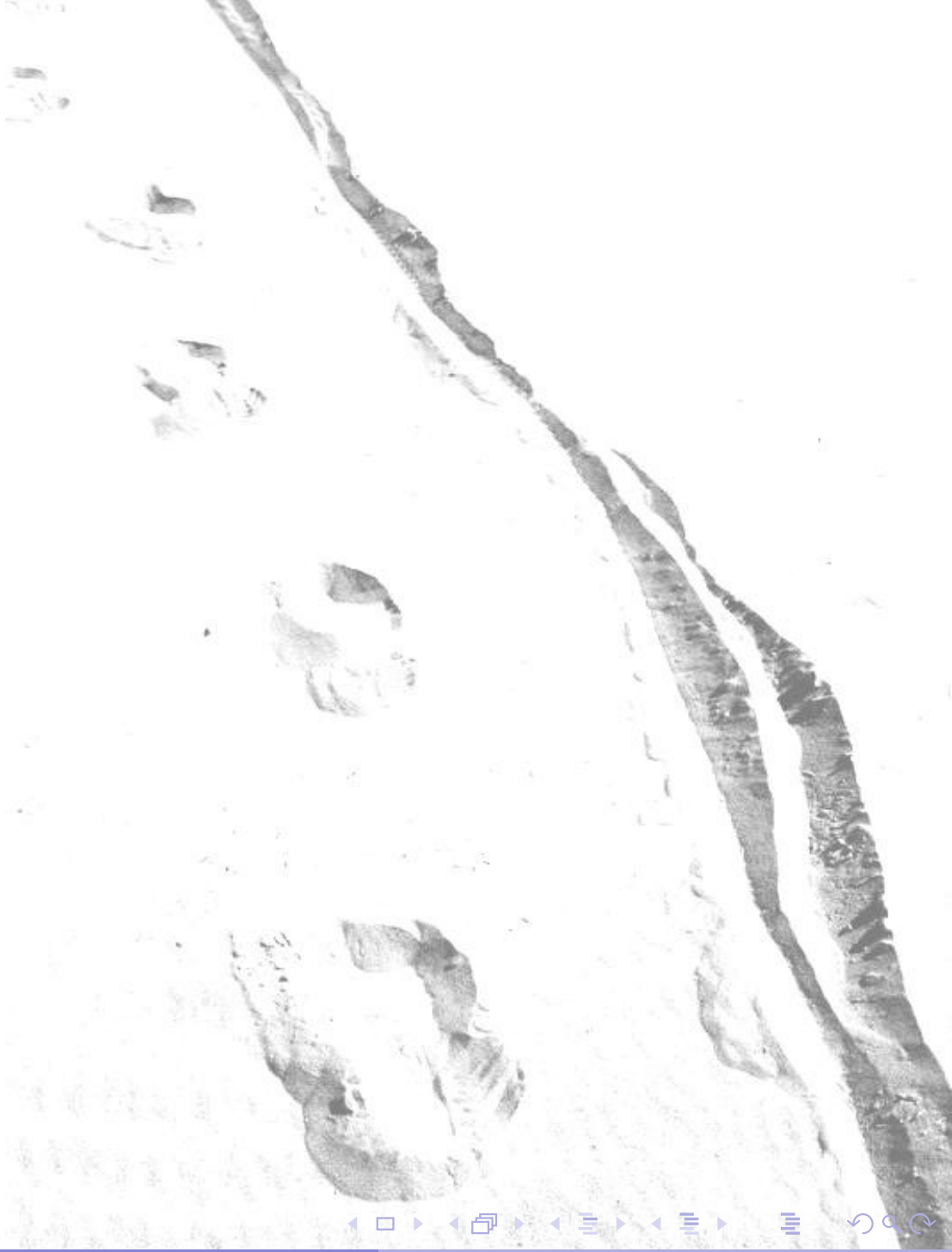
	Sim-to-real	**Co-design**
θ	simulator parameters	design of the system
J^{out}	$J_{\text{real}}(\nu^\star)$, on the real robot	$J^D(\nu^\star, \theta) + C(\theta)$, in the simulator
direct term	0 (the real robot ignores θ)	$\neq 0$ (θ changes the system)
$\nabla_\nu J^{\text{out}}$	**LR**, from real rollouts	**PD** (AD), in the simulator

Adjoint trick (never form κ): with $g = \nabla_\nu J^{\text{out}}$, $g^\top \kappa = -g^\top A^{-1} B$, so

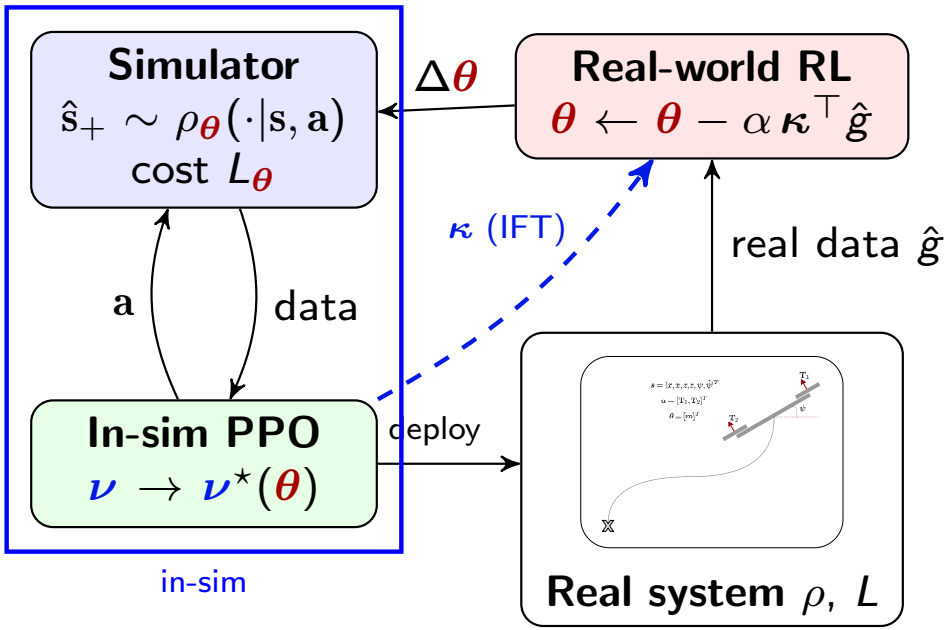
1. solve $A^\top \lambda = g$: one matrix-free linear solve in $\dim \nu$ (CG, MINRES, GMRES)
2. indirect term $= -B^\top \lambda$: one vector–Jacobian product ($> 100\times$ faster)

Outline

1. Decisions from data
2. AI models for Decision
3. Differentiating Policy Training
4. Examples



Example 1: Sim-to-real RL (and Dyna-style MBRL)



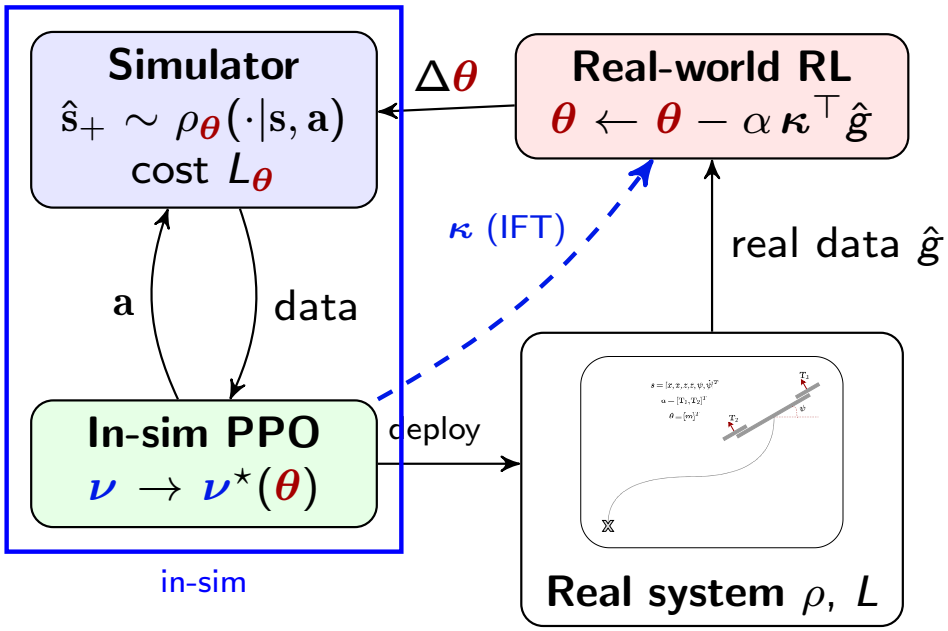
Recall (slide 26): tune the simulator $\boldsymbol{\theta}$ so that the policy trained in it is best on the real robot.

Bi-level RL

$$\min_{\boldsymbol{\theta}} \quad J_{\text{real}}(\pi_{\boldsymbol{\nu}^\star}) = \mathbb{E}_{\rho,\, \pi_{\boldsymbol{\nu}^\star}} \left[\sum_k \gamma^k L(\mathbf{s}_k, \mathbf{a}_k) \right]$$

$$\text{s.t.} \quad \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

Example 1: Sim-to-real RL (and Dyna-style MBRL)



Recall (slide 26): tune the simulator $\boldsymbol{\theta}$ so that the policy trained in it is best on the real robot.

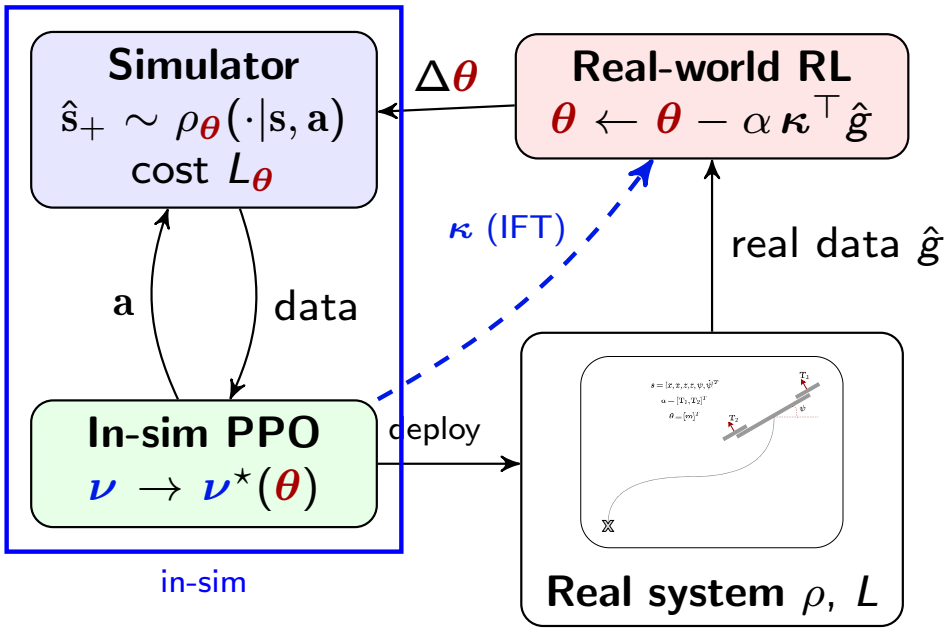
Bi-level RL

$$\min_{\boldsymbol{\theta}} \quad J_{\text{real}}(\pi_{\boldsymbol{\nu}^\star}) = \mathbb{E}_{\rho,\,\pi_{\boldsymbol{\nu}^\star}}\left[\sum_k \gamma^k L(\mathbf{s}_k, \mathbf{a}_k)\right]$$

$$\text{s.t.} \quad \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

- **In-sim**: standard RL in the simulator (PPO, **LR**)

Example 1: Sim-to-real RL (and Dyna-style MBRL)



Recall (slide 26): tune the simulator $\boldsymbol{\theta}$ so that the policy trained in it is best on the real robot.

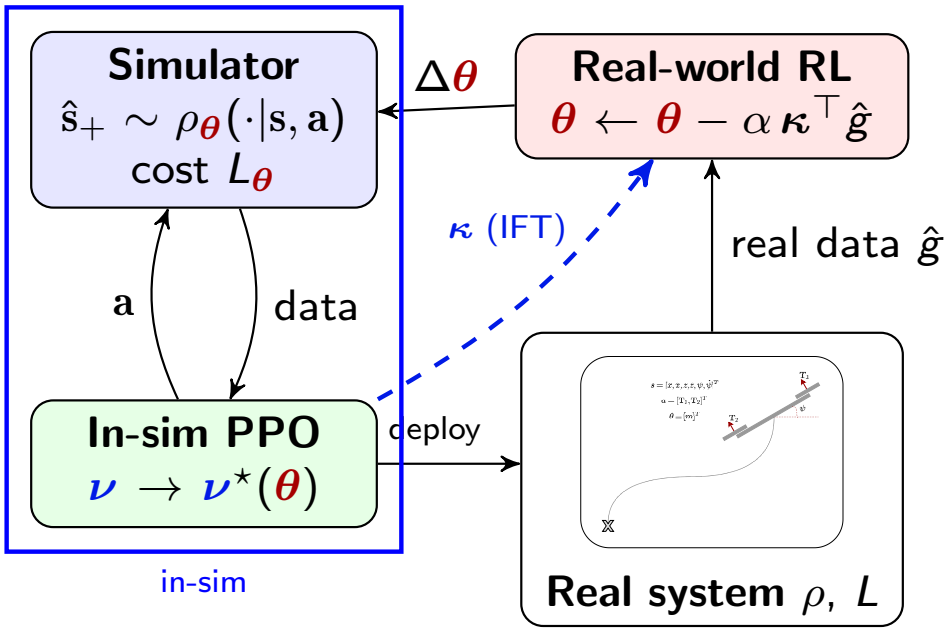
Bi-level RL

$$\min_{\boldsymbol{\theta}} \quad J_{\text{real}}(\pi_{\boldsymbol{\nu}^\star}) = \mathbb{E}_{\rho,\,\pi_{\boldsymbol{\nu}^\star}}\left[\sum_k \gamma^k L(\mathbf{s}_k, \mathbf{a}_k)\right]$$

$$\text{s.t.} \quad \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

- **In-sim**: standard RL in the simulator (PPO, **LR**)
- **Real world**: **LR** policy gradient on real data using $\boldsymbol{\kappa}$

Example 1: Sim-to-real RL (and Dyna-style MBRL)



Converges almost surely on three timescales: critics **fast**, policy $\boldsymbol{\nu}$ **medium**, simulator $\boldsymbol{\theta}$ **slow**

Recall (slide 26): tune the simulator $\boldsymbol{\theta}$ so that the policy trained in it is best on the real robot.

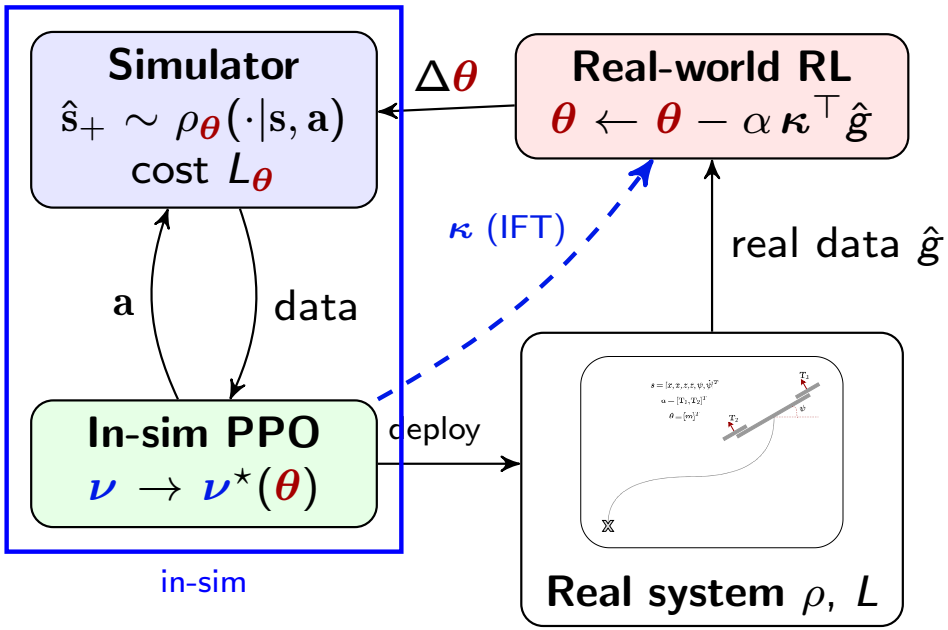
Bi-level RL

$$\min_{\boldsymbol{\theta}} \quad J_{\text{real}}(\pi_{\boldsymbol{\nu}^\star}) = \mathbb{E}_{\rho,\,\pi_{\boldsymbol{\nu}^\star}}\left[\sum_k \gamma^k L(\mathbf{s}_k, \mathbf{a}_k)\right]$$

$$\text{s.t.} \quad \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

- **In-sim**: standard RL in the simulator (PPO, **LR**)
- **Real world**: **LR** policy gradient on real data using $\boldsymbol{\kappa}$

Example 1: Sim-to-real RL (and Dyna-style MBRL)



Converges almost surely on three timescales: critics **fast**, policy $\boldsymbol{\nu}$ **medium**, simulator $\boldsymbol{\theta}$ **slow**

Recall (slide 26): tune the simulator $\boldsymbol{\theta}$ so that the policy trained in it is best on the real robot.

Bi-level RL

$$\min_{\boldsymbol{\theta}} \quad J_{\text{real}}(\pi_{\boldsymbol{\nu}^\star}) = \mathbb{E}_{\rho,\,\pi_{\boldsymbol{\nu}^\star}}\left[\sum_k \gamma^k L(\mathbf{s}_k, \mathbf{a}_k)\right]$$

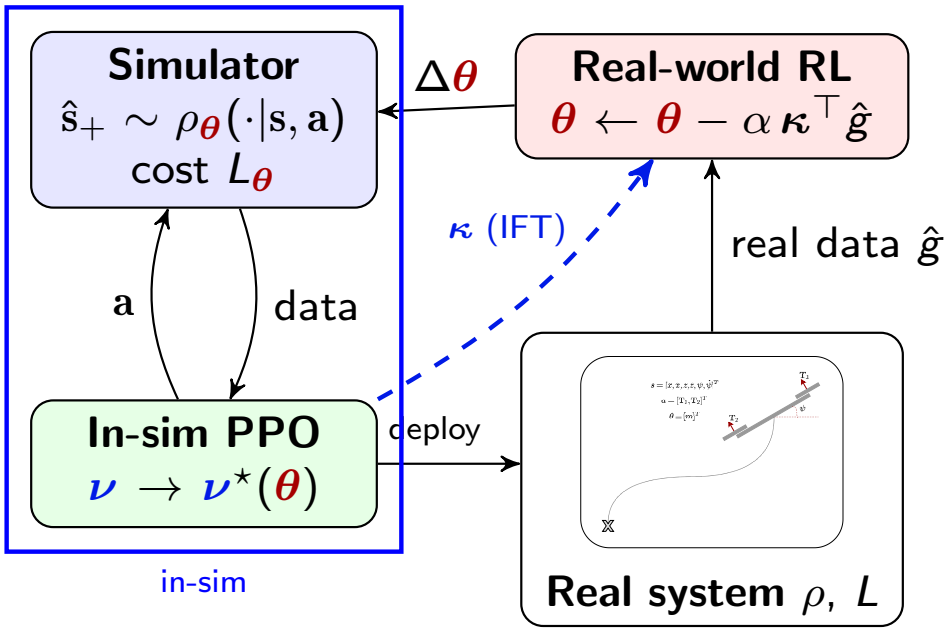
$$\text{s.t.} \quad \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

- **In-sim**: standard RL in the simulator (PPO, **LR**)
- **Real world**: **LR** policy gradient on real data using $\boldsymbol{\kappa}$

Remarks

- Optimal $\boldsymbol{\theta}$ are not unique: pick the most "accurate" one (fitting loss as regularizer)

Example 1: Sim-to-real RL (and Dyna-style MBRL)



Converges almost surely on three timescales: critics **fast**, policy $\boldsymbol{\nu}$ **medium**, simulator $\boldsymbol{\theta}$ **slow**

Recall (slide 26): tune the simulator $\boldsymbol{\theta}$ so that the policy trained in it is best on the real robot.

Bi-level RL

$$\min_{\boldsymbol{\theta}} \quad J_{\text{real}}(\pi_{\boldsymbol{\nu}^\star}) = \mathbb{E}_{\rho,\,\pi_{\boldsymbol{\nu}^\star}}\left[\sum_k \gamma^k L(\mathbf{s}_k, \mathbf{a}_k)\right]$$

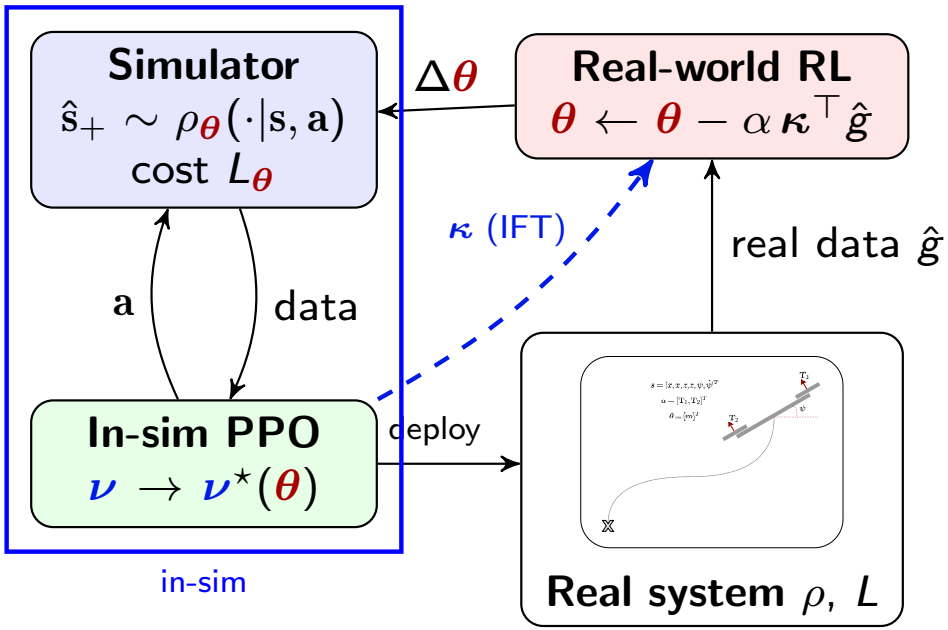
$$\text{s.t.} \quad \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

- **In-sim**: standard RL in the simulator (PPO, **LR**)
- **Real world**: **LR** policy gradient on real data using $\boldsymbol{\kappa}$

Remarks

- Optimal $\boldsymbol{\theta}$ are not unique: pick the most "accurate" one (fitting loss as regularizer)
- **Dyna-style MBRL**: $\rho_{\boldsymbol{\theta}}$ is learned; the real-world gradient complements MLE

Example 1: Sim-to-real RL (and Dyna-style MBRL)



Converges almost surely on three timescales: critics **fast**, policy $\boldsymbol{\nu}$ **medium**, simulator $\boldsymbol{\theta}$ **slow**

Recall (slide 26): tune the simulator $\boldsymbol{\theta}$ so that the policy trained in it is best on the real robot.

Bi-level RL

$$\min_{\boldsymbol{\theta}} \quad J_{\text{real}}(\pi_{\boldsymbol{\nu}^\star}) = \mathbb{E}_{\rho,\,\pi_{\boldsymbol{\nu}^\star}}\left[\sum_k \gamma^k L(\mathbf{s}_k, \mathbf{a}_k)\right]$$

$$\text{s.t.} \quad \nabla_{\boldsymbol{\nu}} \hat{J}_{\boldsymbol{\theta}}(\boldsymbol{\nu}^\star) = 0$$

- **In-sim**: standard RL in the simulator (PPO, **LR**)
- **Real world**: **LR** policy gradient on real data using $\boldsymbol{\kappa}$

Remarks

- Optimal $\boldsymbol{\theta}$ are not unique: pick the most "accurate" one (fitting loss as regularizer)
- **Dyna-style MBRL**: $\rho_{\boldsymbol{\theta}}$ is learned; the real-world gradient complements MLE
- vs. fine-tuning $\boldsymbol{\nu}$ on hardware: $\boldsymbol{\theta}$ is small, physical and explainable

Bi-level PPO and a quadrotor example

Algorithm: Bi-level PPO (costs, minimization)

for *real-world iteration* $j = 1, 2, \ldots$ **do**
- **for** N_{in} *steps* **do** PPO update of $\boldsymbol{\nu}$ in $\rho_{\boldsymbol{\theta}}, L_{\boldsymbol{\theta}}$ (warm start);
- Roll out $\pi_{\boldsymbol{\nu}}$ in sim: estimate A, B;
- Roll out $\pi_{\boldsymbol{\nu}}$ on the real system: $\hat{g} = \nabla_{\boldsymbol{\nu}} J$;
- Solve $(A^\top + \varepsilon I)\,\lambda = \hat{g}$;
- $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\, B^\top \lambda$; // $= \boldsymbol{\theta} - \alpha \boldsymbol{\kappa}^\top \hat{g}$

2D quadrotor (safe-control-gym): reach a goal from random initial states

Bi-level PPO and a quadrotor example

Algorithm: Bi-level PPO (costs, minimization)

for *real-world iteration* $j = 1, 2, \ldots$ **do**
  for N_{in} *steps* **do** PPO update of $\boldsymbol{\nu}$ in $\rho_{\boldsymbol{\theta}}, L_{\boldsymbol{\theta}}$ (warm start);
  Roll out $\pi_{\boldsymbol{\nu}}$ in sim: estimate A, B;
  Roll out $\pi_{\boldsymbol{\nu}}$ on the real system: $\hat{g} = \nabla_{\boldsymbol{\nu}} J$;
  Solve $(A^\top + \varepsilon I)\,\lambda = \hat{g}$;
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\, B^\top \lambda$; // $= \boldsymbol{\theta} - \alpha \boldsymbol{\kappa}^\top \hat{g}$

2D quadrotor (safe-control-gym): reach a goal from random initial states

- simulator intialized with wrong (mass, length)

Bi-level PPO and a quadrotor example

Algorithm: Bi-level PPO (costs, minimization)

for *real-world iteration* $j = 1, 2, \ldots$ **do**
  for N_{in} *steps* **do** PPO update of $\boldsymbol{\nu}$ in $\rho_{\boldsymbol{\theta}}, L_{\boldsymbol{\theta}}$ (warm start);
  Roll out $\pi_{\boldsymbol{\nu}}$ in sim: estimate A, B;
  Roll out $\pi_{\boldsymbol{\nu}}$ on the real system: $\hat{g} = \nabla_{\boldsymbol{\nu}} J$;
  Solve $(A^\top + \varepsilon I)\,\lambda = \hat{g}$;
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\, B^\top \lambda$; // $= \boldsymbol{\theta} - \alpha \boldsymbol{\kappa}^\top \hat{g}$

2D quadrotor (safe-control-gym): reach a goal from random initial states

- simulator intialized with wrong (mass, length)
- policy and critics: neural networks

Bi-level PPO and a quadrotor example

Algorithm: Bi-level PPO (costs, minimization)

for *real-world iteration* $j = 1, 2, \ldots$ **do**
  for N_{in} *steps* **do** PPO update of $\boldsymbol{\nu}$ in $\rho_{\boldsymbol{\theta}}, L_{\boldsymbol{\theta}}$ (warm start);
  Roll out $\pi_{\boldsymbol{\nu}}$ in sim: estimate A, B;
  Roll out $\pi_{\boldsymbol{\nu}}$ on the real system: $\hat{g} = \nabla_{\boldsymbol{\nu}} J$;
  Solve $(A^\top + \varepsilon I)\,\lambda = \hat{g}$;
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\, B^\top \lambda$; // $= \boldsymbol{\theta} - \alpha \boldsymbol{\kappa}^\top \hat{g}$

2D quadrotor (safe-control-gym): reach a goal from random initial states

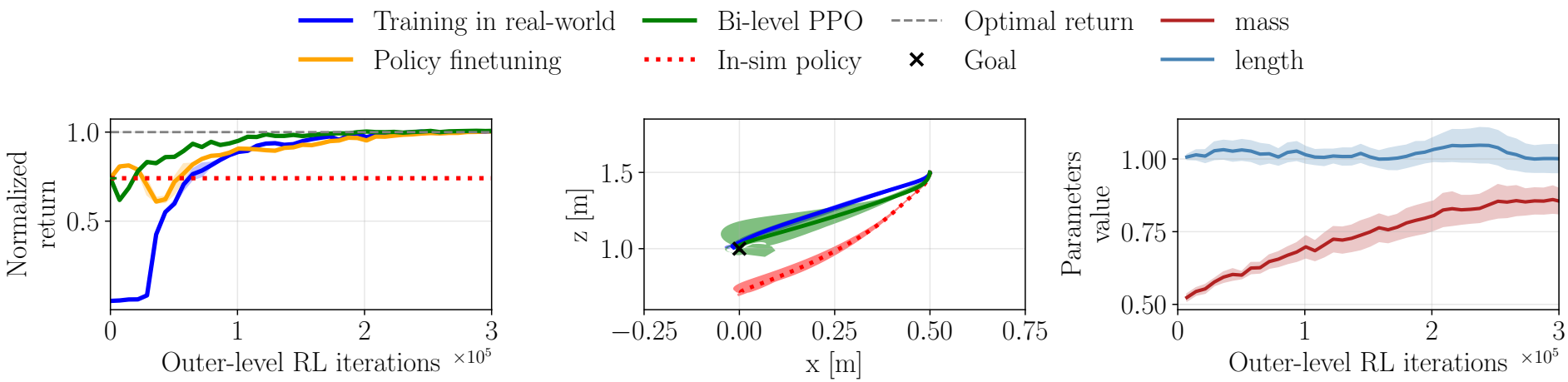
- simulator intialized with wrong (mass, length)
- policy and critics: neural networks

Bi-level PPO and a quadrotor example

Algorithm: Bi-level PPO (costs, minimization)

for *real-world iteration* $j = 1, 2, \ldots$ **do**
  for N_{in} *steps* **do** PPO update of $\boldsymbol{\nu}$ in $\rho_{\boldsymbol{\theta}}, L_{\boldsymbol{\theta}}$ (warm start);
  Roll out $\pi_{\boldsymbol{\nu}}$ in sim: estimate A, B;
  Roll out $\pi_{\boldsymbol{\nu}}$ on the real system: $\hat{g} = \nabla_{\boldsymbol{\nu}} J$;
  Solve $(A^\top + \varepsilon I)\,\lambda = \hat{g}$;
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\, B^\top \lambda$; // $= \boldsymbol{\theta} - \alpha \boldsymbol{\kappa}^\top \hat{g}$

- in-sim policy (red dotted) misses the goal: the sim-to-real gap

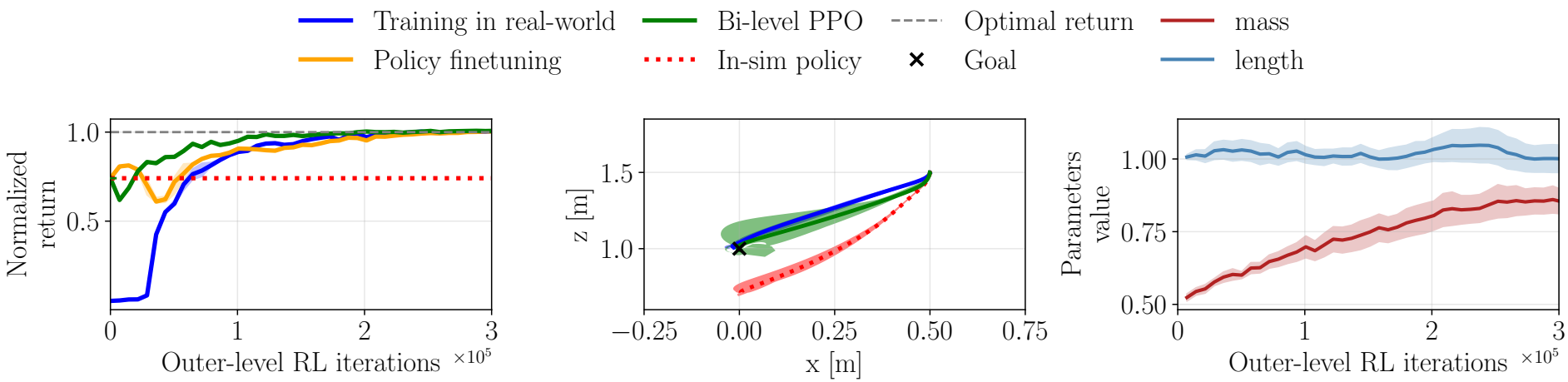


Bi-level PPO and a quadrotor example

Algorithm: Bi-level PPO (costs, minimization)

for *real-world iteration* $j = 1, 2, \ldots$ **do**
  for N_{in} *steps* **do** PPO update of $\boldsymbol{\nu}$ in $\rho_{\boldsymbol{\theta}}, L_{\boldsymbol{\theta}}$ (warm start);
  Roll out $\pi_{\boldsymbol{\nu}}$ in sim: estimate A, B;
  Roll out $\pi_{\boldsymbol{\nu}}$ on the real system: $\hat{g} = \nabla_{\boldsymbol{\nu}} J$;
  Solve $(A^\top + \varepsilon I)\,\lambda = \hat{g}$;
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\, B^\top \lambda$; // $= \boldsymbol{\theta} - \alpha \boldsymbol{\kappa}^\top \hat{g}$

- in-sim policy (red dotted) misses the goal: the sim-to-real gap
- bi-level PPO closes the gap

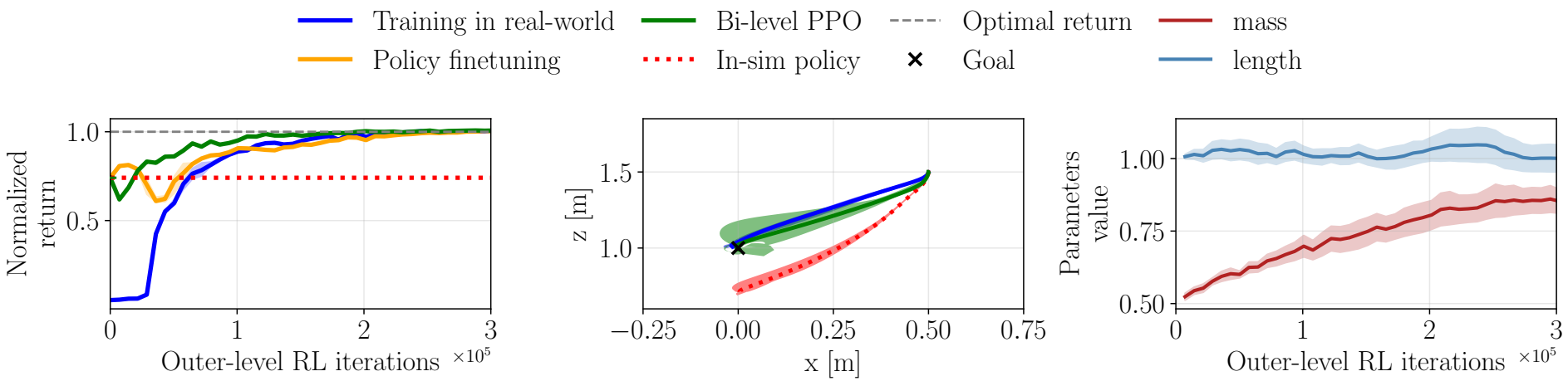


Bi-level PPO and a quadrotor example

Algorithm: Bi-level PPO (costs, minimization)

for *real-world iteration* $j = 1, 2, \ldots$ **do**
  for N_{in} *steps* **do** PPO update of $\boldsymbol{\nu}$ in $\rho_{\boldsymbol{\theta}}, L_{\boldsymbol{\theta}}$ (warm start);
  Roll out $\pi_{\boldsymbol{\nu}}$ in sim: estimate A, B;
  Roll out $\pi_{\boldsymbol{\nu}}$ on the real system: $\hat{g} = \nabla_{\boldsymbol{\nu}} J$;
  Solve $(A^\top + \varepsilon I)\,\lambda = \hat{g}$;
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\, B^\top \lambda$; // $= \boldsymbol{\theta} - \alpha \boldsymbol{\kappa}^\top \hat{g}$

- in-sim policy (red dotted) misses the goal: the sim-to-real gap
- bi-level PPO closes the gap
- **mass** $\rightarrow \approx 0.85\, m_0$: wrong model, optimal policy



Example 2: Co-design of a system and its policy

Recall (slide 27): $\boldsymbol{\theta}$ is the design; each design needs its own trained policy.

Bi-level problem

$$\begin{aligned} \min_{\boldsymbol{\theta}} \quad & J^D\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right) + C(\boldsymbol{\theta}) \\ \text{s.t.} \quad & \boldsymbol{\nu}^\star(\boldsymbol{\theta}) \in \arg\min_{\boldsymbol{\nu}} J^\pi(\boldsymbol{\nu}, \boldsymbol{\theta}) \end{aligned}$$

Example 2: Co-design of a system and its policy

Recall (slide 27): $\boldsymbol{\theta}$ is the design; each design needs its own trained policy.

Bi-level problem

$$\begin{aligned} \min_{\boldsymbol{\theta}} \quad & J^D\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right) + C(\boldsymbol{\theta}) \\ \text{s.t.} \quad & \boldsymbol{\nu}^\star(\boldsymbol{\theta}) \in \arg\min_{\boldsymbol{\nu}} J^\pi(\boldsymbol{\nu}, \boldsymbol{\theta}) \end{aligned}$$

- J^π: **operational** cost, trains the policy in-sim (the role of $\hat{J}_{\boldsymbol{\theta}}$)

Example 2: Co-design of a system and its policy

Recall (slide 27): $\boldsymbol{\theta}$ is the design; each design needs its own trained policy.

Bi-level problem

$$\begin{aligned} \min_{\boldsymbol{\theta}} \quad & J^D\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right) + C(\boldsymbol{\theta}) \\ \text{s.t.} \quad & \boldsymbol{\nu}^\star(\boldsymbol{\theta}) \in \arg\min_{\boldsymbol{\nu}} J^\pi(\boldsymbol{\nu}, \boldsymbol{\theta}) \end{aligned}$$

- J^π: **operational** cost, trains the policy in-sim (the role of $\hat{J}_{\boldsymbol{\theta}}$)
- J^D **judges** the design (the role of J^{out}), $C(\boldsymbol{\theta})$: cost to build it

Example 2: Co-design of a system and its policy

Recall (slide 27): $\boldsymbol{\theta}$ is the design; each design needs its own trained policy.

Bi-level problem

$$\begin{aligned} \min_{\boldsymbol{\theta}} \quad & J^D\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right) + C(\boldsymbol{\theta}) \\ \text{s.t.} \quad & \boldsymbol{\nu}^\star(\boldsymbol{\theta}) \in \arg\min_{\boldsymbol{\nu}} J^\pi(\boldsymbol{\nu}, \boldsymbol{\theta}) \end{aligned}$$

- J^π: **operational** cost, trains the policy in-sim (the role of $\hat{J}_{\boldsymbol{\theta}}$)
- J^D **judges** the design (the role of J^{out}), $C(\boldsymbol{\theta})$: cost to build it
- Usual practice: Bayesian optimization (BO) or evolution strategies (ES) around full RL runs; scales poorly with $\dim \boldsymbol{\theta}$

Example 2: Co-design of a system and its policy

Recall (slide 27): $\boldsymbol{\theta}$ is the design; each design needs its own trained policy.

Bi-level problem

$$\begin{aligned} \min_{\boldsymbol{\theta}} \quad & J^D\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right) + C(\boldsymbol{\theta}) \\ \text{s.t.} \quad & \boldsymbol{\nu}^\star(\boldsymbol{\theta}) \in \arg\min_{\boldsymbol{\nu}} J^\pi(\boldsymbol{\nu}, \boldsymbol{\theta}) \end{aligned}$$

- J^π: **operational** cost, trains the policy in-sim (the role of $\hat{J}_{\boldsymbol{\theta}}$)
- J^D **judges** the design (the role of J^{out}), $C(\boldsymbol{\theta})$: cost to build it
- Usual practice: Bayesian optimization (BO) or evolution strategies (ES) around full RL runs; scales poorly with dim $\boldsymbol{\theta}$
- Here: differentiable simulator, so **PD** everywhere

Example 2: Co-design of a system and its policy

Recall (slide 27): $\boldsymbol{\theta}$ is the design; each design needs its own trained policy.

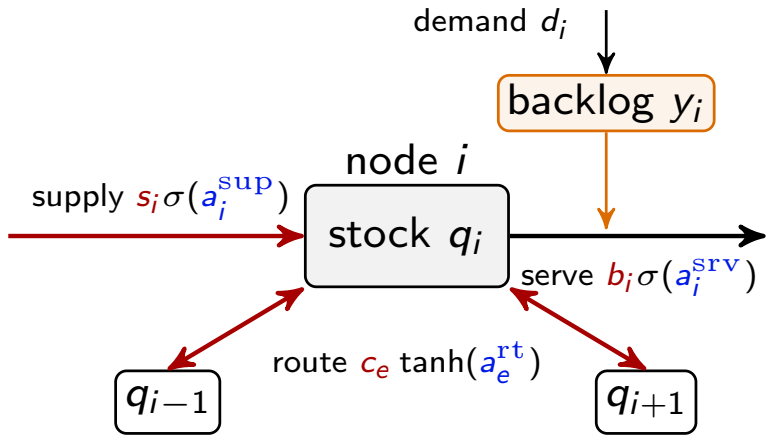
Bi-level problem

$$\min_{\boldsymbol{\theta}} \quad J^D\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right) + C(\boldsymbol{\theta})$$

$$\text{s.t.} \quad \boldsymbol{\nu}^\star(\boldsymbol{\theta}) \in \arg\min_{\boldsymbol{\nu}} J^\pi(\boldsymbol{\nu}, \boldsymbol{\theta})$$

- J^π: **operational** cost, trains the policy in-sim (the role of $\hat{J}_{\boldsymbol{\theta}}$)
- J^D **judges** the design (the role of J^{out}), $C(\boldsymbol{\theta})$: cost to build it
- Usual practice: Bayesian optimization (BO) or evolution strategies (ES) around full RL runs; scales poorly with dim $\boldsymbol{\theta}$
- Here: differentiable simulator, so **PD** everywhere

Example: inventory–routing network instead of a drone: $\boldsymbol{\theta}$ = capacities, not arm lengths



- state: stock q_i, backlog y_i, AR(1) demand d_i

Example 2: Co-design of a system and its policy

Recall (slide 27): $\boldsymbol{\theta}$ is the design; each design needs its own trained policy.

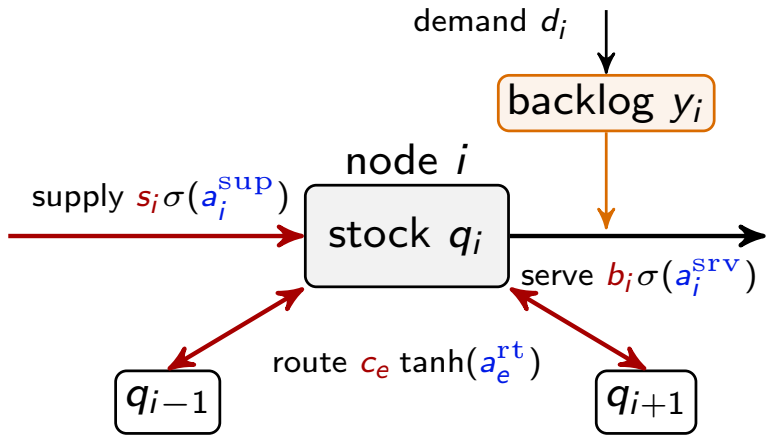
Bi-level problem

$$\min_{\boldsymbol{\theta}} \quad J^D\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right) + C(\boldsymbol{\theta})$$

$$\text{s.t.} \quad \boldsymbol{\nu}^\star(\boldsymbol{\theta}) \in \arg\min_{\boldsymbol{\nu}} J^\pi(\boldsymbol{\nu}, \boldsymbol{\theta})$$

- J^π: **operational** cost, trains the policy in-sim (the role of $\hat{J}_{\boldsymbol{\theta}}$)
- J^D **judges** the design (the role of J^{out}), $C(\boldsymbol{\theta})$: cost to build it
- Usual practice: Bayesian optimization (BO) or evolution strategies (ES) around full RL runs; scales poorly with $\dim \boldsymbol{\theta}$
- Here: differentiable simulator, so **PD** everywhere

Example: inventory–routing network instead of a drone: $\boldsymbol{\theta}$ = capacities, not arm lengths



- state: stock q_i, backlog y_i, AR(1) demand d_i
- $\boldsymbol{\theta}$: supply, service, routing capacities, $\dim \boldsymbol{\theta}$ from 15 to 150

Example 2: Co-design of a system and its policy

Recall (slide 27): $\boldsymbol{\theta}$ is the design; each design needs its own trained policy.

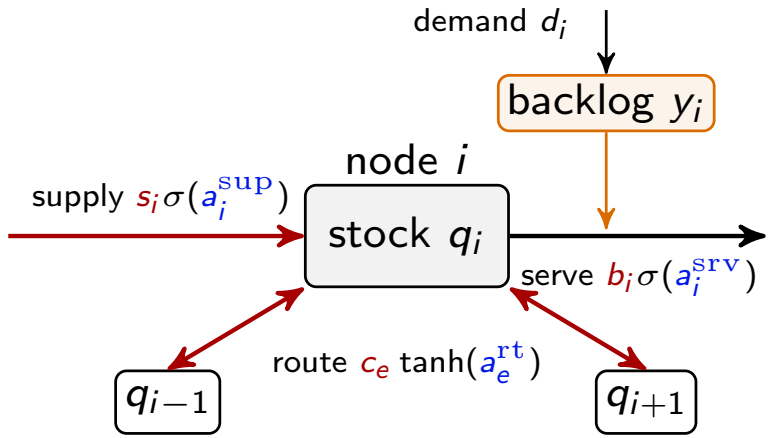
Bi-level problem

$$\min_{\boldsymbol{\theta}} \quad J^D\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right) + C(\boldsymbol{\theta})$$

$$\text{s.t.} \quad \boldsymbol{\nu}^\star(\boldsymbol{\theta}) \in \arg\min_{\boldsymbol{\nu}} J^\pi(\boldsymbol{\nu}, \boldsymbol{\theta})$$

- J^π: **operational** cost, trains the policy in-sim (the role of $\hat{J}_{\boldsymbol{\theta}}$)
- J^D **judges** the design (the role of J^{out}), $C(\boldsymbol{\theta})$: cost to build it
- Usual practice: Bayesian optimization (BO) or evolution strategies (ES) around full RL runs; scales poorly with $\dim \boldsymbol{\theta}$
- Here: differentiable simulator, so **PD** everywhere

Example: inventory–routing network instead of a drone: $\boldsymbol{\theta}$ = capacities, not arm lengths



- state: stock q_i, backlog y_i, AR(1) demand d_i
- $\boldsymbol{\theta}$: supply, service, routing capacities, $\dim \boldsymbol{\theta}$ from 15 to 150
- the design bounds the actions; softplus-smoothed min, max

Example 2: Co-design of a system and its policy

Recall (slide 27): $\boldsymbol{\theta}$ is the design; each design needs its own trained policy.

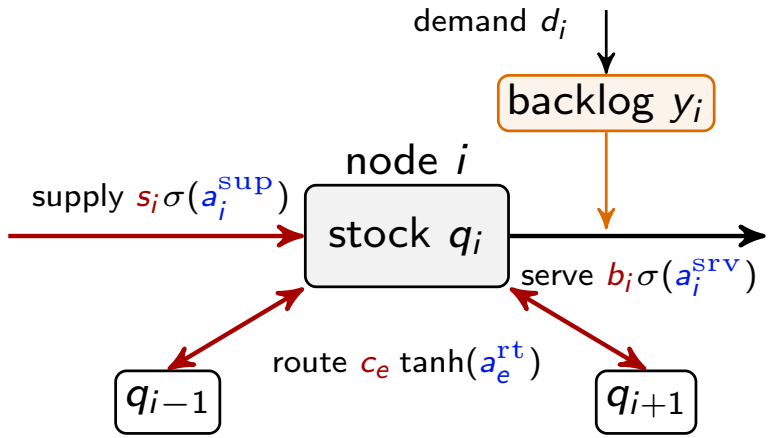
Bi-level problem

$$\min_{\boldsymbol{\theta}} \quad J^D\left(\boldsymbol{\nu}^\star(\boldsymbol{\theta}), \boldsymbol{\theta}\right) + C(\boldsymbol{\theta})$$

$$\text{s.t.} \quad \boldsymbol{\nu}^\star(\boldsymbol{\theta}) \in \arg\min_{\boldsymbol{\nu}} J^\pi(\boldsymbol{\nu}, \boldsymbol{\theta})$$

- J^π: **operational** cost, trains the policy in-sim (the role of $\hat{J}_{\boldsymbol{\theta}}$)
- J^D **judges** the design (the role of J^{out}), $C(\boldsymbol{\theta})$: cost to build it
- Usual practice: Bayesian optimization (BO) or evolution strategies (ES) around full RL runs; scales poorly with $\dim \boldsymbol{\theta}$
- Here: differentiable simulator, so **PD** everywhere

Example: inventory–routing network instead of a drone: $\boldsymbol{\theta}$ = capacities, not arm lengths



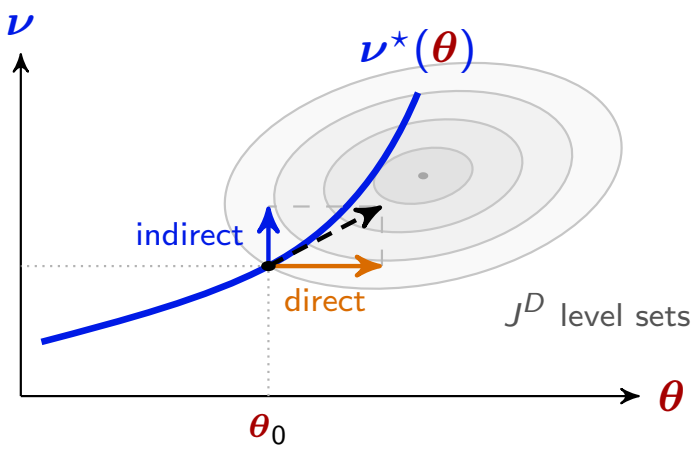
- state: stock q_i, backlog y_i, AR(1) demand d_i
- $\boldsymbol{\theta}$: supply, service, routing capacities, $\dim \boldsymbol{\theta}$ from 15 to 150
- the design bounds the actions; softplus-smoothed min, max
- $J^D = J^\pi + \lambda_y \cdot \text{backlog}$, $C(\boldsymbol{\theta})$ quadratic

Co-design gradient with pathwise (SVG) in-sim training

$$\nabla_{\boldsymbol{\theta}}\left(J^D + C\right) = \underbrace{\partial_{\boldsymbol{\theta}} J^D}_{\text{direct}} + \underbrace{\nabla_{\boldsymbol{\nu}} J^D \, \boldsymbol{\kappa}}_{\text{indirect}} + \nabla_{\boldsymbol{\theta}} C$$

Co-design gradient with pathwise (SVG) in-sim training

$$\nabla_{\boldsymbol{\theta}}\left(J^D + C\right) = \underbrace{\partial_{\boldsymbol{\theta}} J^D}_{\text{direct}} + \underbrace{\nabla_{\boldsymbol{\nu}} J^D \, \boldsymbol{\kappa}}_{\text{indirect}} + \nabla_{\boldsymbol{\theta}} C$$



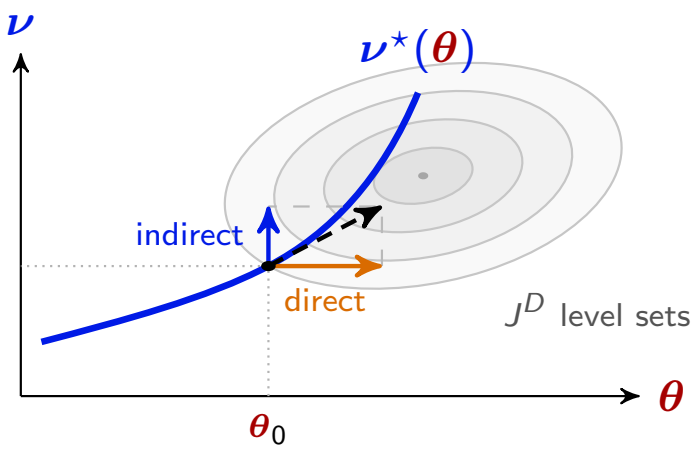
A design step = a move at fixed policy
+ the induced policy shift along $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$

Co-design gradient with pathwise (SVG) in-sim training

$$\nabla_{\boldsymbol{\theta}}\left(J^D + C\right) = \underbrace{\partial_{\boldsymbol{\theta}} J^D}_{\text{direct}} + \underbrace{\nabla_{\boldsymbol{\nu}} J^D \, \boldsymbol{\kappa}}_{\text{indirect}} + \nabla_{\boldsymbol{\theta}} C$$

Along a sampled path, $\boldsymbol{\kappa}$ enters through the actions:

$$A_k = \nabla_{\mathrm{s}} g_{\boldsymbol{\nu}} \, S_k + \nabla_{\boldsymbol{\nu}} g_{\boldsymbol{\nu}} \, \boldsymbol{\kappa}$$
$$S_{k+1} = \partial_{\boldsymbol{\theta}} f_{\boldsymbol{\theta}} + \nabla_{\mathrm{s}} f_{\boldsymbol{\theta}} \, S_k + \nabla_{\mathrm{a}} f_{\boldsymbol{\theta}} \, A_k$$



A design step = a move at fixed policy + the induced policy shift along $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$

Co-design gradient with pathwise (SVG) in-sim training

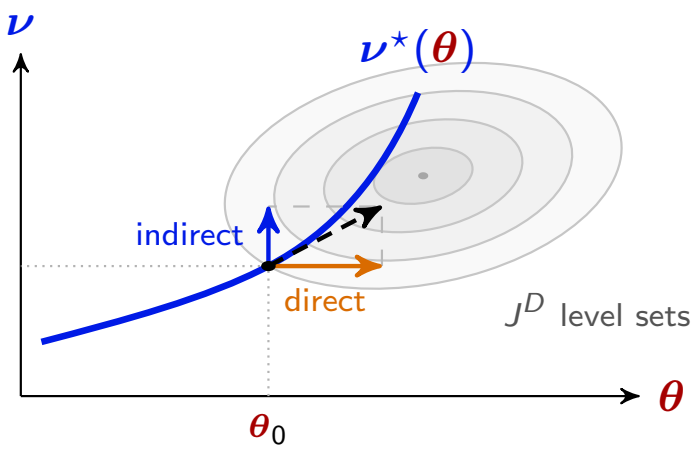
$$\nabla_{\boldsymbol{\theta}}\left(J^D + C\right) = \underbrace{\partial_{\boldsymbol{\theta}} J^D}_{\text{direct}} + \underbrace{\nabla_{\boldsymbol{\nu}} J^D \, \boldsymbol{\kappa}}_{\text{indirect}} + \nabla_{\boldsymbol{\theta}} C$$

Along a sampled path, $\boldsymbol{\kappa}$ enters through the actions:

$$A_k = \nabla_{\mathrm{s}} g_{\boldsymbol{\nu}} \, S_k + \nabla_{\boldsymbol{\nu}} g_{\boldsymbol{\nu}} \, \boldsymbol{\kappa}$$

$$S_{k+1} = \partial_{\boldsymbol{\theta}} f_{\boldsymbol{\theta}} + \nabla_{\mathrm{s}} f_{\boldsymbol{\theta}} \, S_k + \nabla_{\mathrm{a}} f_{\boldsymbol{\theta}} \, A_k$$

Envelope: indirect term $= 0$ if $J^D = J^\pi$; the backlog penalty ($\lambda_y > 0$) makes $J^D \neq J^\pi$.



A design step = a move at fixed policy + the induced policy shift along $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$

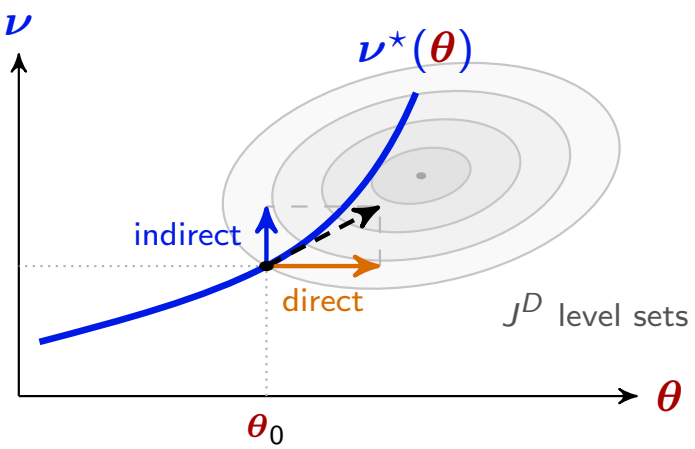
Co-design gradient with pathwise (SVG) in-sim training

$$\nabla_{\boldsymbol{\theta}}\left(J^D + C\right) = \underbrace{\partial_{\boldsymbol{\theta}} J^D}_{\text{direct}} + \underbrace{\nabla_{\boldsymbol{\nu}} J^D\, \boldsymbol{\kappa}}_{\text{indirect}} + \nabla_{\boldsymbol{\theta}} C$$

Along a sampled path, $\boldsymbol{\kappa}$ enters through the actions:

$$A_k = \nabla_{\mathrm{s}} g_{\boldsymbol{\nu}}\, S_k + \nabla_{\boldsymbol{\nu}} g_{\boldsymbol{\nu}}\, \boldsymbol{\kappa}$$
$$S_{k+1} = \partial_{\boldsymbol{\theta}} f_{\boldsymbol{\theta}} + \nabla_{\mathrm{s}} f_{\boldsymbol{\theta}}\, S_k + \nabla_{\mathrm{a}} f_{\boldsymbol{\theta}}\, A_k$$

Envelope: indirect term $= 0$ if $J^D = J^\pi$; the backlog penalty ($\lambda_y > 0$) makes $J^D \neq J^\pi$.



A design step = a move at fixed policy + the induced policy shift along $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$

Algorithm: Gradient-based co-design (costs, minimization)

for $k = 0, 1, 2, \ldots$ **do**

A few warm-started SVG steps on $J^\pi(\cdot, \boldsymbol{\theta}_k)$: they only have to **track** $\boldsymbol{\nu}^\star(\boldsymbol{\theta}_k)$;

$g_{\mathrm{dir}} \leftarrow \partial_{\boldsymbol{\theta}} J^D$, one `jax.grad` with $\boldsymbol{\nu}$ fixed;

if $J^D \neq J^\pi$ **then** $u \leftarrow \nabla_{\boldsymbol{\nu}} J^D$, $g_{\mathrm{ind}} \leftarrow -B^\top (A + \varepsilon I)^{-1} u$; **else** $g_{\mathrm{ind}} \leftarrow 0$;

$\boldsymbol{\theta}_{k+1} \leftarrow \Pi_\Theta\left(\boldsymbol{\theta}_k - \alpha\left(g_{\mathrm{dir}} + g_{\mathrm{ind}} + \nabla_{\boldsymbol{\theta}} C\right)\right)$;

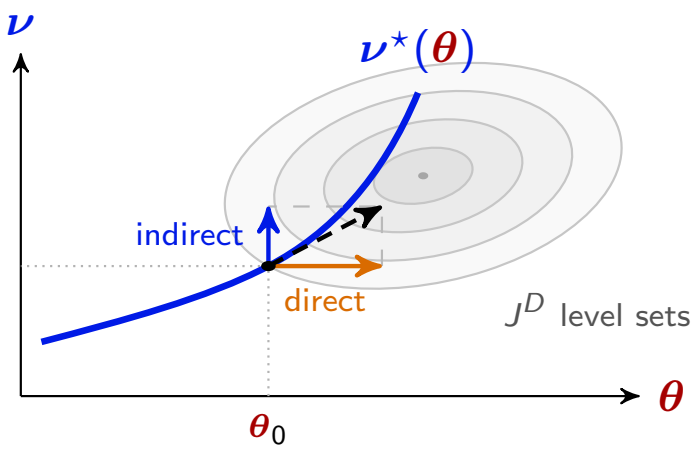
Co-design gradient with pathwise (SVG) in-sim training

$$\nabla_{\boldsymbol{\theta}}\left(J^D + C\right) = \underbrace{\partial_{\boldsymbol{\theta}} J^D}_{\text{direct}} + \underbrace{\nabla_{\boldsymbol{\nu}} J^D\, \boldsymbol{\kappa}}_{\text{indirect}} + \nabla_{\boldsymbol{\theta}} C$$

Along a sampled path, $\boldsymbol{\kappa}$ enters through the actions:

$$A_k = \nabla_{\mathrm{s}} g_{\boldsymbol{\nu}}\, S_k + \nabla_{\boldsymbol{\nu}} g_{\boldsymbol{\nu}}\, \boldsymbol{\kappa}$$
$$S_{k+1} = \partial_{\boldsymbol{\theta}} f_{\boldsymbol{\theta}} + \nabla_{\mathrm{s}} f_{\boldsymbol{\theta}}\, S_k + \nabla_{\mathrm{a}} f_{\boldsymbol{\theta}}\, A_k$$

Envelope: indirect term $= 0$ if $J^D = J^\pi$; the backlog penalty ($\lambda_y > 0$) makes $J^D \neq J^\pi$.



A design step = a move at fixed policy + the induced policy shift along $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$

Algorithm: Gradient-based co-design (costs, minimization)

for $k = 0, 1, 2, \ldots$ **do**

A few warm-started SVG steps on $J^\pi(\cdot, \boldsymbol{\theta}_k)$: they only have to **track** $\boldsymbol{\nu}^\star(\boldsymbol{\theta}_k)$;

$g_{\mathrm{dir}} \leftarrow \partial_{\boldsymbol{\theta}} J^D$, one `jax.grad` with $\boldsymbol{\nu}$ fixed;

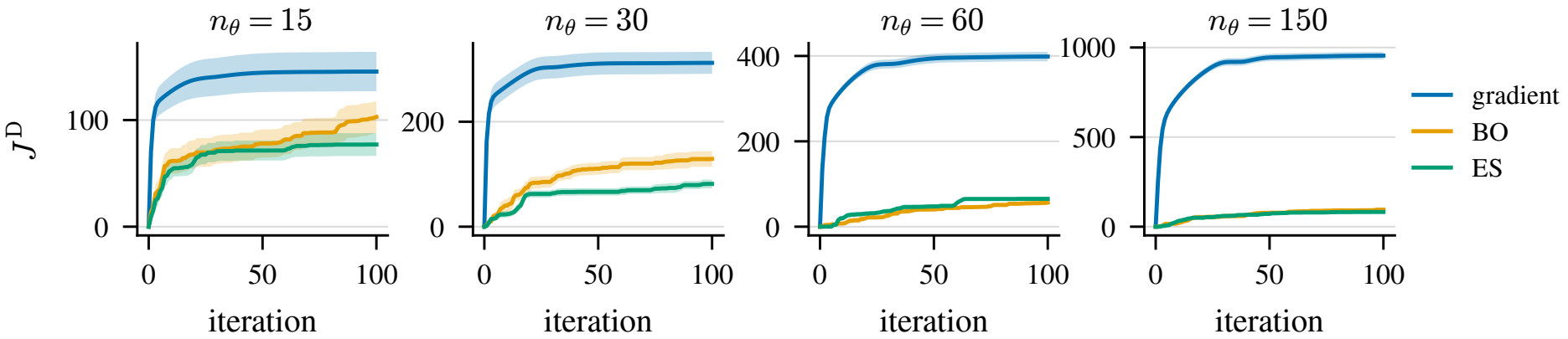
if $J^D \neq J^\pi$ **then** $u \leftarrow \nabla_{\boldsymbol{\nu}} J^D$, $g_{\mathrm{ind}} \leftarrow -B^\top (A + \varepsilon I)^{-1} u$; **else** $g_{\mathrm{ind}} \leftarrow 0$;

$\boldsymbol{\theta}_{k+1} \leftarrow \Pi_\Theta\left(\boldsymbol{\theta}_k - \alpha\left(g_{\mathrm{dir}} + g_{\mathrm{ind}} + \nabla_{\boldsymbol{\theta}} C\right)\right)$;

Converges to an approximately stationary design for small α: error $\mathcal{O}\left(\frac{1}{\alpha N}\right) + \mathcal{O}(\alpha\sigma^2)$+ bias

Co-design results: inventory–routing network

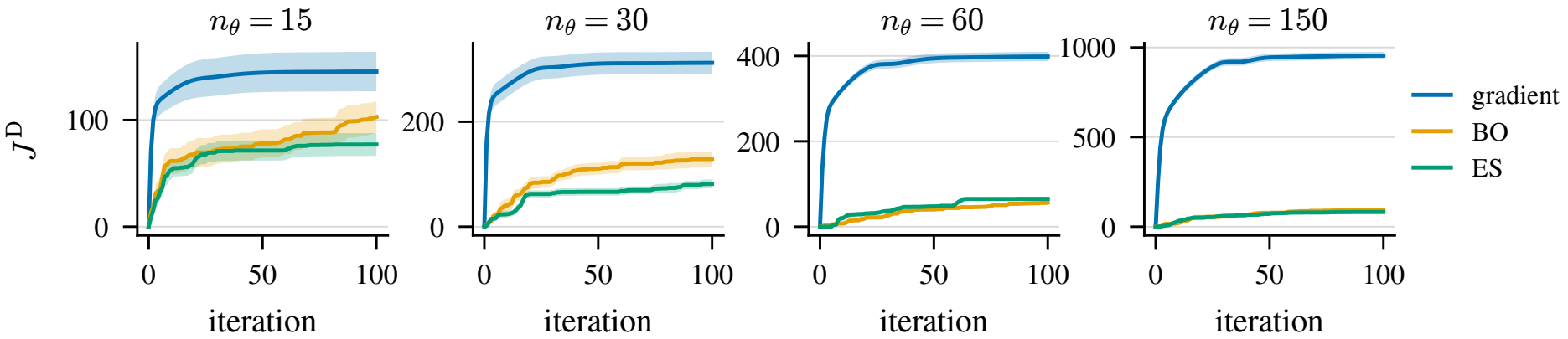
Against black-box search: 100 design evaluations each, matched objectives ($\lambda_y = 0$, so the κ-term is 0 and only the direct gradient is used)



Plots show the design *return* $-(J^D + C)$: higher is better.

Co-design results: inventory–routing network

Against black-box search: 100 design evaluations each, matched objectives ($\lambda_y = 0$, so the κ-term is 0 and only the direct gradient is used)

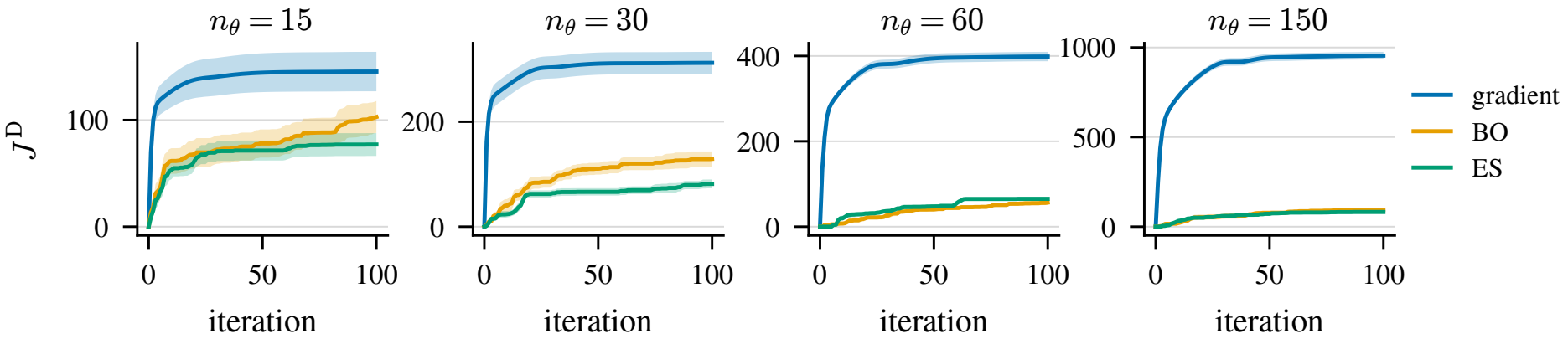


- the gradient approach outperforms BO and ES at every size, and the gap grows with dim $\boldsymbol{\theta}$ (15 to 150)

Plots show the design *return* $-(J^D + C)$: higher is better.

Co-design results: inventory–routing network

Against black-box search: 100 design evaluations each, matched objectives ($\lambda_y = 0$, so the κ-term is 0 and only the direct gradient is used)

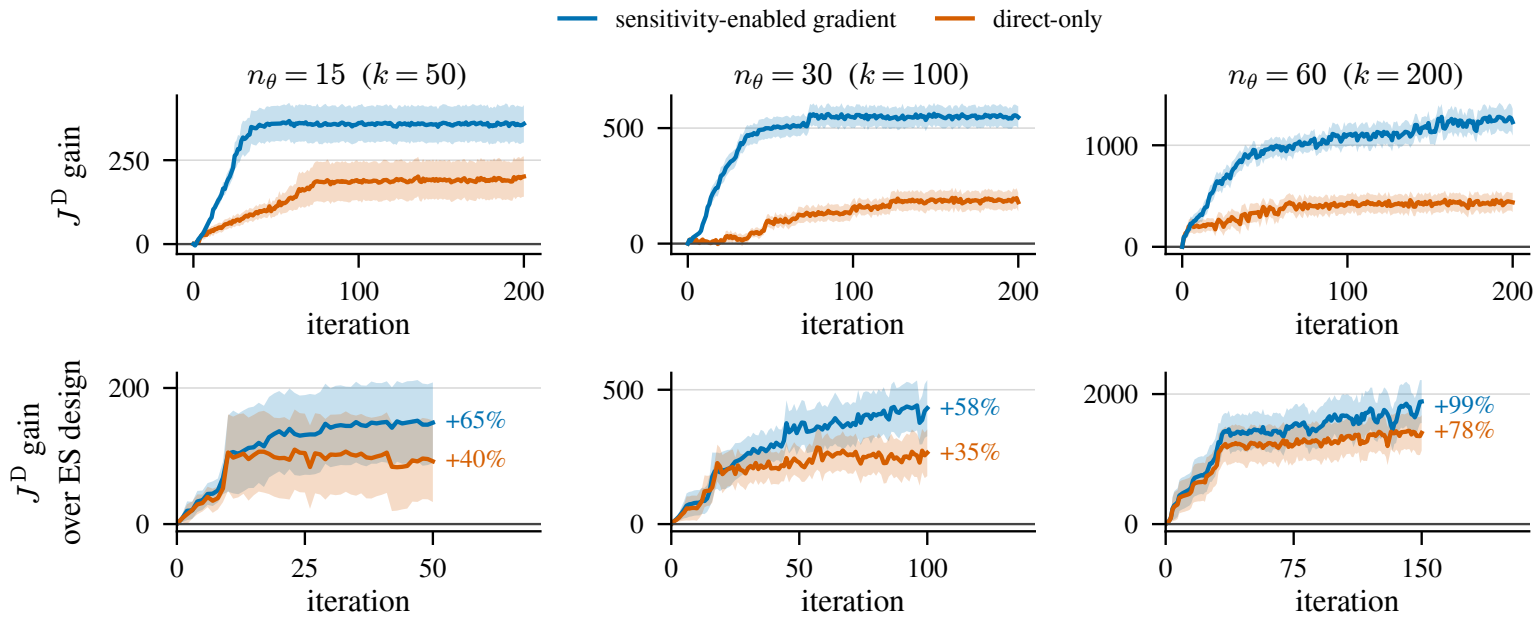


- the gradient approach outperforms BO and ES at every size, and the gap grows with dim $\boldsymbol{\theta}$ (15 to 150)
- cost per design evaluation: $\approx 2\,\mathrm{s}$ for the gradient, $\approx 2\,\mathrm{s}$ ES, $\approx 3\,\mathrm{s}$ BO

Plots show the design *return* $-(J^D + C)$: higher is better.

Co-design results: inventory–routing network

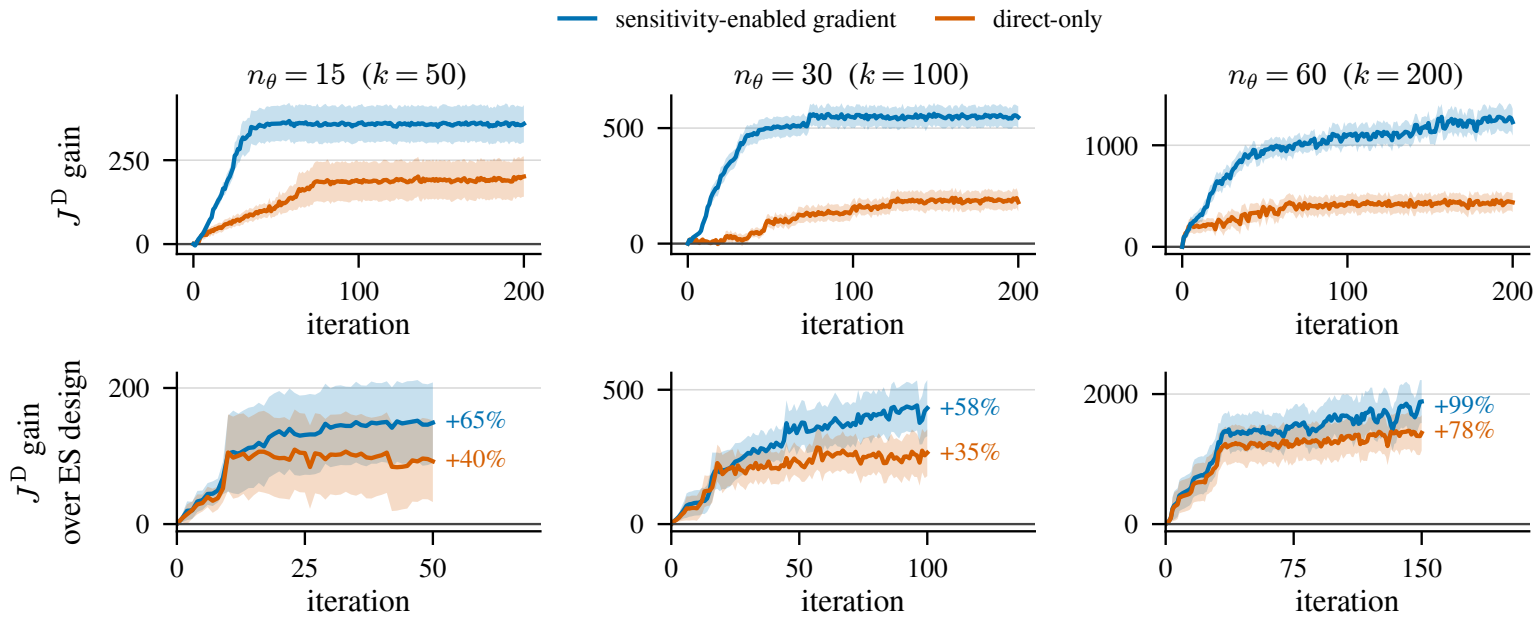
Effect of the κ-term: mismatched objectives ($\lambda_y = 0.1$, $J^D \neq J^\pi$)



Plots show the design *return* $-(J^D + C)$: higher is better.

Co-design results: inventory–routing network

Effect of the κ-term: mismatched objectives ($\lambda_y = 0.1$, $J^D \neq J^\pi$)

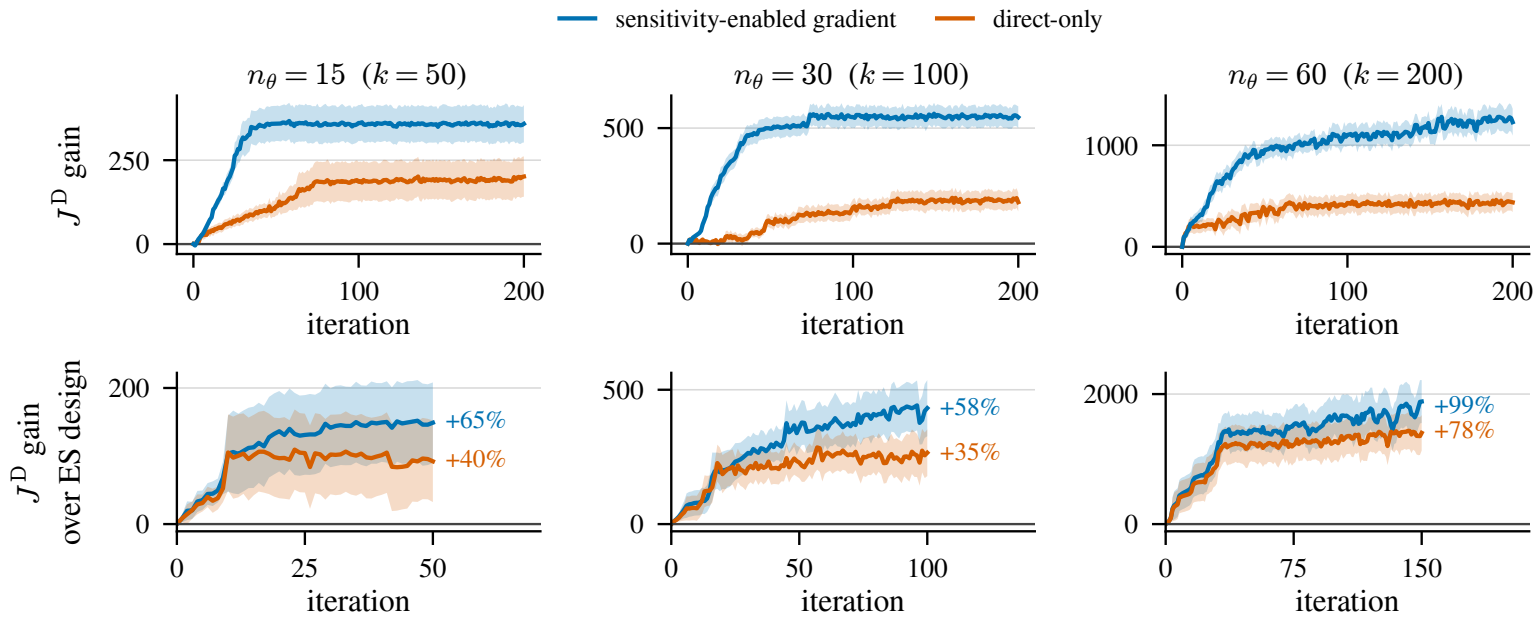


- **top**: sensitivity-enabled vs direct-only gradient, from the same initial design: the κ-term helps, more in higher dimension

Plots show the design *return* $-(J^D + C)$: higher is better.

Co-design results: inventory–routing network

Effect of the κ-term: mismatched objectives ($\lambda_y = 0.1$, $J^D \neq J^\pi$)

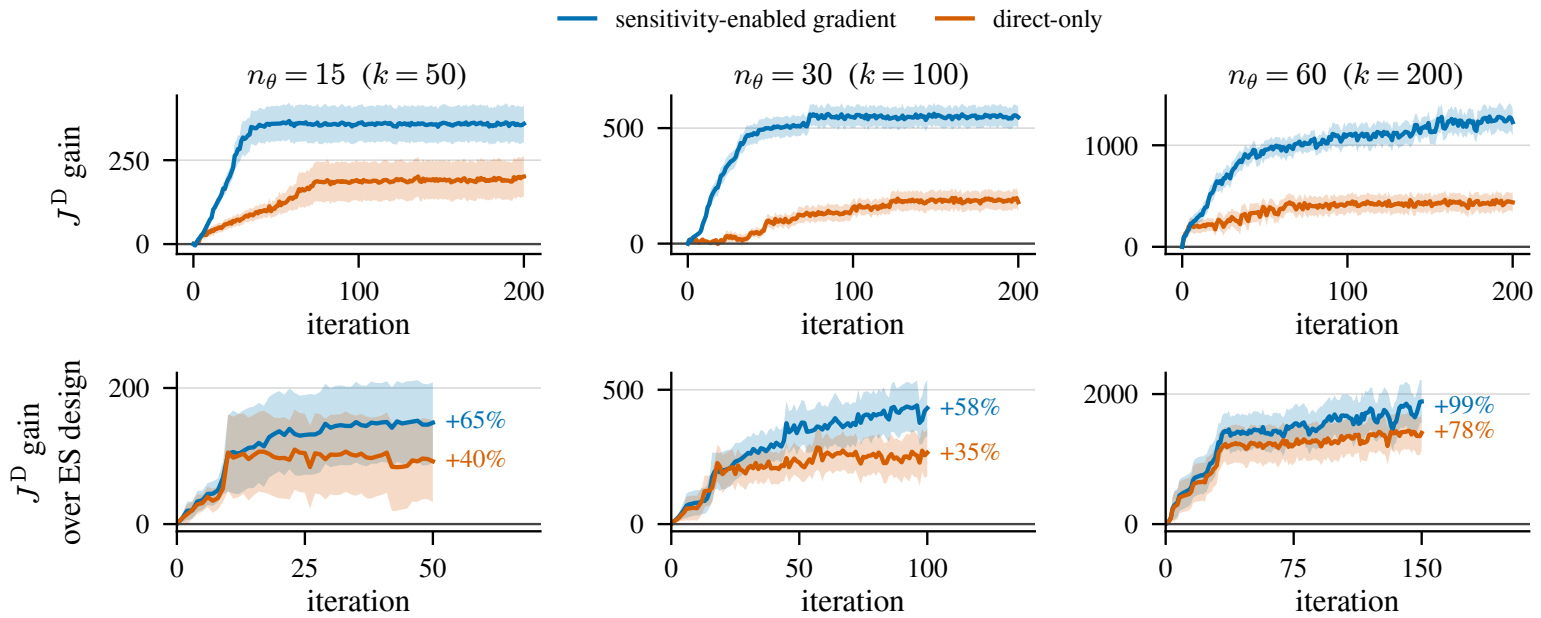


- **top**: sensitivity-enabled vs direct-only gradient, from the same initial design: the κ-term helps, more in higher dimension
- **bottom**: starting from the best design of a converged ES search: the gradient still improves it

Plots show the design *return* $-(J^D + C)$: higher is better.

Co-design results: inventory–routing network

Effect of the κ-term: mismatched objectives ($\lambda_y = 0.1$, $J^D \neq J^\pi$)



- **top**: sensitivity-enabled vs direct-only gradient, from the same initial design: the κ-term helps, more in higher dimension
- **bottom**: starting from the best design of a converged ES search: the gradient still improves it
- cost: $\approx 6.5\,\mathrm{s}$ per design evaluation with a matrix-free κ-solve (vs $\approx 2\,\mathrm{s}$ direct-only)

Plots show the design *return* $-(J^D + C)$: higher is better.

Take-aways

1. A policy trained in a model is an **implicit function** of the MDP: $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$

Take-aways

1. A policy trained in a model is an **implicit function** of the MDP: $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$
2. **Bellman tree**: $\boldsymbol{\nu}$ lives on the action edges, $\boldsymbol{\theta}$ on the transition edges and the costs
   - **LR**: an edge passes $\partial \log P(\text{edge}) \times (\text{value below it})$: no derivatives in $(\mathbf{s}, \mathbf{a})$, real data; variance grows with $|Q|$
   - **PD**: chain rule along each sampled path: differentiable simulators; low variance if the dynamics are smooth; H steps + critic

Take-aways

1. A policy trained in a model is an **implicit function** of the MDP: $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$
2. **Bellman tree**: $\boldsymbol{\nu}$ lives on the action edges, $\boldsymbol{\theta}$ on the transition edges and the costs
   - **LR**: an edge passes $\partial \log P(\text{edge}) \times (\text{value below it})$: no derivatives in $(\mathbf{s}, \mathbf{a})$, real data; variance grows with $|Q|$
   - **PD**: chain rule along each sampled path: differentiable simulators; low variance if the dynamics are smooth; H steps + critic
3. **IFT** on the stationarity of training: $\boldsymbol{\kappa} = -A^{-1}B$. Only the end point of training matters

Take-aways

1. A policy trained in a model is an **implicit function** of the MDP: $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$
2. **Bellman tree**: $\boldsymbol{\nu}$ lives on the action edges, $\boldsymbol{\theta}$ on the transition edges and the costs
   - **LR**: an edge passes $\partial \log P(\text{edge}) \times (\text{value below it})$: no derivatives in $(\mathbf{s}, \mathbf{a})$, real data; variance grows with $|Q|$
   - **PD**: chain rule along each sampled path: differentiable simulators; low variance if the dynamics are smooth; H steps + critic
3. **IFT** on the stationarity of training: $\boldsymbol{\kappa} = -A^{-1}B$. Only the end point of training matters
4. **Outer gradient** = direct + $\nabla_{\boldsymbol{\nu}} J^{\text{out}}\, \boldsymbol{\kappa}$: sim-to-real tunes the *model* (direct = 0), co-design the *system* (both terms)

Take-aways

1. A policy trained in a model is an **implicit function** of the MDP: $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$
2. **Bellman tree**: $\boldsymbol{\nu}$ lives on the action edges, $\boldsymbol{\theta}$ on the transition edges and the costs
   - **LR**: an edge passes $\partial \log P(\text{edge}) \times (\text{value below it})$: no derivatives in $(\mathbf{s}, \mathbf{a})$, real data; variance grows with $|Q|$
   - **PD**: chain rule along each sampled path: differentiable simulators; low variance if the dynamics are smooth; H steps + critic
3. **IFT** on the stationarity of training: $\boldsymbol{\kappa} = -A^{-1}B$. Only the end point of training matters
4. **Outer gradient** = direct + $\nabla_{\boldsymbol{\nu}} J^{\text{out}}\, \boldsymbol{\kappa}$: sim-to-real tunes the *model* (direct = 0), co-design the *system* (both terms)
5. **In JAX**: `grad`, `jvp`, `vjp` + one Krylov solve; never form $\boldsymbol{\kappa}$; **LR** needs a DiCE-type surrogate

Take-aways

1. A policy trained in a model is an **implicit function** of the MDP: $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$
2. **Bellman tree**: $\boldsymbol{\nu}$ lives on the action edges, $\boldsymbol{\theta}$ on the transition edges and the costs
   - **LR**: an edge passes $\partial \log P(\text{edge}) \times (\text{value below it})$: no derivatives in $(\mathbf{s}, \mathbf{a})$, real data; variance grows with $|Q|$
   - **PD**: chain rule along each sampled path: differentiable simulators; low variance if the dynamics are smooth; H steps + critic
3. **IFT** on the stationarity of training: $\boldsymbol{\kappa} = -A^{-1}B$. Only the end point of training matters
4. **Outer gradient** = direct + $\nabla_{\boldsymbol{\nu}} J^{\text{out}}\, \boldsymbol{\kappa}$: sim-to-real tunes the *model* (direct = 0), co-design the *system* (both terms)
5. **In JAX**: `grad`, `jvp`, `vjp` + one Krylov solve; never form $\boldsymbol{\kappa}$; **LR** needs a DiCE-type surrogate
6. **Applications**: simulators tuned for *decisions* (sim-to-real, MBRL) and system design (co-design)

Take-aways

1. A policy trained in a model is an **implicit function** of the MDP: $\boldsymbol{\nu}^\star(\boldsymbol{\theta})$
2. **Bellman tree**: $\boldsymbol{\nu}$ lives on the action edges, $\boldsymbol{\theta}$ on the transition edges and the costs
   - **LR**: an edge passes $\partial \log P(\text{edge}) \times (\text{value below it})$: no derivatives in $(\mathbf{s}, \mathbf{a})$, real data; variance grows with $|Q|$
   - **PD**: chain rule along each sampled path: differentiable simulators; low variance if the dynamics are smooth; H steps + critic
3. **IFT** on the stationarity of training: $\boldsymbol{\kappa} = -A^{-1}B$. Only the end point of training matters
4. **Outer gradient** = direct + $\nabla_{\boldsymbol{\nu}} J^{\text{out}}\, \boldsymbol{\kappa}$: sim-to-real tunes the *model* (direct = 0), co-design the *system* (both terms)
5. **In JAX**: `grad`, `jvp`, `vjp` + one Krylov solve; never form $\boldsymbol{\kappa}$; **LR** needs a DiCE-type surrogate
6. **Applications**: simulators tuned for *decisions* (sim-to-real, MBRL) and system design (co-design)

Open questions: scaling the critic sensitivities, conditioning of A for deep networks, offline real-world updates, partial observability

Papers

- Anand et al., *Bi-level RL pathway for sim-to-real optimality*, RLC 2026
- Anand et al., *Gradient-based co-design of RL policies with differentiable simulators*, TMLR 2026